基于稳健表示、共表达网络与风格条件生成的啤酒化合物—感官强度建模

摘要

针对啤酒化学谱与感官强度之间的高维、非线性关联，以及风格样本不均衡和缺陷控制要求，本文以 250 款啤酒的 231 个化学及理化变量与 50 个标准化感官标签为基础，建立预测、关联识别与风格条件生成的统一框架，并在每一问内先固定评价协议、再比较候选。

问题一在训练折内完成缺失处理与稳健表示，比较逐目标中位数、普通 Ridge、图 Ridge 与图正则低秩四模型（同一嵌套交叉验证、预处理与截距约定）。主候选 Aroma/Flavour 30 维等权 MAE 为 0.3593、同协议中位数基线 0.3982；全 50 维 0.3505、基线 0.3919。误差都在所给标签尺度上、且均为折外误差，不是 0–6 原始等级误差；0–6 等级由训练折标签的中点经验分布标定，只是训练分布内的相对推断等级。图 Ridge 与普通 Ridge 几乎相同，低秩相对两者的改善按**低秩收缩**归因，不读成“加图收益”。

问题二把候选宇宙拆为 216 个严格单体化合物与 15 个扩展指标两张表，用总体 Spearman、风格内残差化 Spearman、图低秩系数与三者百分位均值四种方法，经风格分层 bootstrap 给出逐维 Top-3。方法间 Top-3 平均两两 Jaccard 0.2091，是集合一致性而非识别准确率。方向稳定性取“与总体 Spearman 点估计同方向的一致率”0.9817；旧口径 $P(\rho > 0)$ 0.7226 只作历史对照。附件 2 描述性网络（231/645/52）与训练折内网络是两个不同产物（图 4、图 5），前者只作结构解释、不进入验证。

问题三把生成与迁移分开。生成侧用含编码质量点的层次收缩边际与收缩高斯 Copula，投影后给出非负浓度设计谱，归一化经验 V 能量距离 0.8727，仍高于同量源样本重采样参照，故不宣称优于该参照。化学反-2-壬烯醛不增（0 条），感官 t2n 代理仍增加 366 条（0.488），两命题分列。迁移侧 198 个证书均最优、源状态分层核验通过（3465 / 分母 3663），交付策略转移能量 2.0799，优于同一完整可行域的均值投影基线，而两者综合不增率都是 1。

稳健性按问题分列：问题一 3 个执行块（参考 + 种子块 + 内层正则扰动）；问题二、三为**冻结种子块参数扰动**、计划内各 2 块，另有登记后补跑的补充块单列未并入计划记录。所有相对极差在预注册限内，但每组只覆盖 3 个冻结种子中的 2 个，固定种子复跑只证确定性重现。问题一没有独立外部测试集，问题三的迁移只是声明模型下的最优，相应边界在各章分开声明。

关键词：图正则低秩回归；嵌套交叉验证；风格混杂；分层 bootstrap；收缩 Copula；凸二次规划证书

目录

摘要

问题重述与分析 1

数据核实与统一字典 2

模型假设、符号与总体框架 3

问题一：图正则低秩预测与相对分级 4

问题二：共表达网络、风格混杂与稳定 Top-3 9

问题三：风格条件生成与声明模型下的迁移 14

稳健性与限制 27

组件作用与负结果汇总 28

结论、建议与局限 29

参考文献 29

AI 工具使用声明 30

附录 A　支撑材料长表索引与两张概率表 31

附录 B　复现要点 32

附录 C　核心源程序代码 33

问题重述与分析

题目要求与建模对象

赛题要求三项相互关联的任务：估计评分卡定义的 Aroma 与 Flavour 感官强度并输出 0–6 分级；用化合物相关矩阵构建共表达图，识别每个感官维度的 Top-3 并论证稳定性与化学合理性；学习以 `tasting_category_fine` 为条件的化合物谱生成模型，展示 Tripel、Kriek、Lambic 的感官画像，并把 Pils/Lager 化学谱迁移至 Tripel、同时控制指定缺陷 [1]。

共同难点：特征数接近样本量、化合物高度相关；标准化感官值与原始整数评分之间缺少可逆标定；风格差异可能使总体相关偏离风格内关系；小样本风格难以独立估计高维联合分布。本文先核实数据尺度与编码规则，再为每问固定评价协议、最后比较候选。

任务	建模对象	核心输出	主要评价
感官评分	$Y = f(X) + E$	折外连续预测与 0–6 相对分级	30 维与 50 维等权平均 MAE、平均 R^2、Skill
关键因素	化学成分与感官的同期关联	每个维度 3 个候选及其稳定性	入选频率、方法间 Jaccard、方向一致率
生成	$p(X \mid s)$	风格谱、感官代理画像、缺陷对照	归一化能量距离、约束分层核验
迁移	声明模型下的迁移算子	迁移前后化学与感官对照	转移能量、完整可行域基线、约束分层与求解证书

生成与迁移在第 6 章分列成两个对象，评价对象不同，不能合并成一个 “问题三成功率”。

三问的评价对象与证据边界

本文把每条陈述的证据强度分成四层并逐处标注：**协议已固定**、**已真实执行**（产物文件存在且可复算）、**数值可复核**（可追溯到支撑材料中的具体文件与列）、**结论适用边界已声明**。四层不互相替代：一个数字可复核，不等于它支持一个更宽的结论；一次执行成功，也不等于该协议覆盖了留出泛化。

按此口径，三问的证据位置如下，正文与文末限制按实际状态书写，不做统一化表述：

- 问题一：3 次重复 × 3 外层折 × 每折 3 内层折的嵌套交叉验证完整覆盖 250 行，750 行折外预测可复算；但没有独立外部测试集，也没有在 0–6 原始人工评分上校准。
- 问题二：固定 200 行训练集、风格分层 500 次 bootstrap 的 Top-3 与方向一致率可复算；但集合一致性与方向一致率都不是识别准确率，附件交叉相关表与本研究同源。
- 问题三：生成侧与迁移侧的产物、求解证书与约束分层均可复算；但生成未优于同量源样本重采样参照，迁移只是声明模型下的最优，且没有迁移后真实标签。

与附件实际结构不符的三处修正

第一，评分工作簿是空白评分卡。附件 `3.xlsx` 含 `beer_1` 至 `beer_12` 与一张汇总页，各页均为带 0–6 刻度的空白模板，不含已填写的原始整数评分，也不含该量表的标定参数。因此本文的 0–6 等级只能定义为可复核的相对等级，而不能声称恢复了人工评分。评分卡上还印有一项 **Acetaldehyde (青苹果、果皮)**，但标签表 `4.csv` 没有它的列：该项列入维度映射说明，不预测、不分级、不造真值。

第二，交叉相关表的感官列顺序与主表不一致。附件 `6.csv` 尾部的列序为 Carbonation、Aftertaste、Body、Overall，而目标表为 aftertaste、body、co2、overall 的顺序，按位置对齐会错位三列，并连带使化学行与候选索引错配。本文改为按名称一对一对齐：化学行经化学元数据映射到特征名，感官列经 50 条字面字典映射到目标名，任何重复、歧义或未匹配的行按名报告并中止，不做静默丢弃。

第三，先导轮实现存在三处名实不符，本轮全部重做。其一是 “普通 Ridge” 实际带有图正则，不是普通 Ridge；其二是低秩拟合每轮刷新截距而 Ridge 参照固定标签均值，两者不在同一截距约定下可比；其三是 “折外” 表把样本内预测与留出预测混在同一张表里。确认轮把四模型放在同一嵌套协议、同一预处理规则与同一截距公式下重算，并对 “秩 50 且 $\gamma = 0$ 应还原普通 Ridge” 作逐折数值核对。

数据核实与统一字典

样本对齐与主键

化学表、感官表与跨来源描述符表均按 `beer` 序号连接：`beer` 是 1..250 的置换，可整行对齐 1.csv 与 4.csv；后两表的 `beer_id` 只有 96 个唯一值，存在截断，不能作为可靠连接键。化学表规模为 250 行、231 列，感官表 50 列，风格变量共 22 类。

尺度、编码点与物理浓度

利用附件校准表覆盖的 70 个变量逐列回算，37 列与 $\log_{10} c$ 一致，33 列与原始浓度 c 一致，其余 161 列缺少直接校准依据。在已确认的对数列中，$x < 0$、$x = 0$、$x > 0$ 分别对应以指定单位计量的 $0 < c < 1$、$c = 1$、$c > 1$；该逻辑不适用于未经确认的列。

37 个对数列中有 755 个单元格对应校准表记录的零值，其存储值为该列最小正浓度一半的 $\log_{10}$。另有 142 个未校准列共享约 -1.8178 的地板编码，合计 6807 个单元格。这些数值首先表示观测编码，不能据此断言真实浓度为零或已知低于某个仪器检出限。生成模型因此显式保留地板质量点，物理阈值比较只在尺度与单位已确认时进行。

校准附件与主输入的 80 格冲突（本轮新增披露）。把 `1.csv` 的存储值与 `8.xlsx` 校准表按同尺度逐格比较（对数校准列比 $\log_{10} c$，线性校准列比 c，容差 10^{-7}）：可比单元格共 16613 个，其中 80 个单元格不一致，79 个相对偏差超过 1%。冲突**全部落在 33 个线性校准列**（24 款酒、32 个字段，含反-2-壬烯醛、双乙酰、4VG 相关列），37 个对数校准列零冲突。本文的处理规则是先声明、不修补：`1.csv` 是主输入，`8.xlsx` 只用于确定尺度与单位，冲突单元格不替换、不插补，也不宣称两侧一致；冲突成因**未知**，本文不把它解释成任一方有误。

数据类别	数量	用途及处理
已校准 log10 变量	37	预测时使用存储值；物理比较时反变换 $c = 10^x$
已校准线性变量	33	保留原单位；物理比较时施加非负下界
未直接校准变量	161	以存储尺度建模，只在声明的观测支撑内取值
其中派生汇总量	5（含于上两行，不另计）	热量、总酸、总酯、酒花香气及硫化物汇总；属子集，合计仍为 231
感官目标	50	用于预测与评价，不作为生成器的化学输入

缺失处理与三套评价协议

化学表有 73 个缺失单元格，分布于 9 列、63 行，其中乙酸戊酯 27 个、酒花香气汇总 13 个、甲基吡嗪 12 个、双乙酰 9 个、3-甲基丁醛 7 个，其余 4 列合计 5 个。预测模型在当前训练折内部进行中位数填补，并增加 9 个缺失指示列；验证折与评价行沿用训练折参数。生成模型估计边际时只使用有效观测，不把填补值的堆积当作真实概率质量点。

本轮存在三套并行的评价协议，正文不把它们混为一谈：

- **问题一采用全样本嵌套交叉验证（评价协议）**：3 次重复 × 3 个外层折 × 每折 3 个内层折，覆盖全部 250 款啤酒，每次重复的折外预测合起来恰好覆盖 250 行，三次重复合计 750 行预测。该协议没有独立外部测试集；本文以嵌套交叉验证的外折测试结果评估本数据分布内的表现，“留出泛化” 类陈述不成立。
- **问题一的 250 行评分表是展示协议，不是评价协议**：它由冻结候选在全部 250 行上重拟合一次得到，只用于给出连续预测与 0–6 相对等级，其样本内拟合误差不作为性能估计，也不参与任何 MAE、R^2、基线或结论计算。评价侧与展示侧分别是 `oof_predictions_750.csv`（750 行，250 款酒 × 3 次重复）与 `beer_scores_250.csv` / `grades_250.csv`（各 250 行）；维度字典 `dimension_map.csv` 共 51 行 = 50 个有标签维度 + 1 个无标签卡面项。两侧不可互换。
- **问题二使用注册的 200 行训练集**：保留的 50 行为控制重复曝光而不参与拟合，也不用于评价。问题二的 bootstrap 只在该 200 行内进行。

三套协议的对象不同，其误差与距离数值不可横向比较。

防止信息泄露

缺失处理、稳健表示、目标中心化、图估计、超参数选择与分级切点全部在训练折内拟合。任一啤酒的折外预测都来自不含该啤酒的训练集：外层测试行不参与填补中位数、不参与标准化参数、不参与图估计、不参与内层选参、不参与分级切点。附件 `2.csv` 的全样本相关矩阵只用于构造第 5 章“两个用途不同的网络”一节的描述性网络，本轮把它作为独立产物重算并登记；它不参与任何 Top-3 选择，也不进入预测或关联模型的验证。预测与关联模型的正则化图必须由当前训练折重新计算。跨来源感官描述符与附件提供的既有模型重要性均不作为化学输入。

模型假设、符号与总体框架

主要假设

第一，已对齐的同期样本可用于估计本数据分布内的化学—感官关系；该关系不直接代表未来批次或其他市场。第二，可确认尺度的变量按校准字典解释，未校准变量仅作为存储尺度上的观测量。第三，风格内秩残差可减少风格组间差异的影响，但不能消除未观测混杂。第四，条件生成模型近似描述既有风格的观测分布；缺陷约束是数学筛查规则，不能单独保证真实啤酒的工艺可实现性。第五，迁移结论只在声明的变量空间、目标函数与约束集合下成立。

符号说明

符号	含义
$X \in \mathbb{R}^{n\times p}$	存储尺度化学及理化特征，$p = 231$
Z	当前训练折拟合的稳健表示，含 9 个缺失指示列，共 240 维
$Y \in \mathbb{R}^{n\times q}$	标准化感官标签，$q = 50$；Aroma 与 Flavour 各 15 维记作 $q_{AF} = 30$
s_i	第 i 款啤酒的细粒度风格，共 22 类
$G = (V, E, W), L_G$	训练折内估计的带符号关联图及其拉普拉斯矩阵
B, r	多输出回归系数及秩约束
$\rho^{\text{total}}, \rho^{\text{within}}$	总体及风格内残差化秩相关
S_{jt}	候选 j 在维度 t 的综合分
$F_{j,s}, R_s$	风格条件边际分布与 Copula 相关矩阵
c_j, T_j	已确认单位的物理浓度与感官阈值
z	迁移二次规划的变量，为折叠标准化存储空间中的 231 维向量
P, D, C	迁移目标风格的主成分载荷、得分标准差对角阵与其收缩相关矩阵 $C = 0.9\widehat{R}_T + 0.1I$
V, U	经验能量距离（非负）与带符号能量诊断量（可负，不截零）
π_{jt}, sign_stability$_{jt}$	候选入选频率与“与点估计同方向”的复制一致率

总体框架

统一框架以字典为基础：问题一估计多输出感官预测器并给出相对分级；问题二使用同族图低秩回归的系数，与总体关联和风格内关联共同识别候选；问题三先条件生成化学谱，再在声明的凸二次规划下做风格迁移。三问共享标签体系与化学字典，但评价对象不同：问题一评价逐维误差，问题二评价集合一致性，问题三评价分布距离与求解证书。任何跨问的“综合得分”在本文中都不使用。

问题一：图正则低秩预测与相对分级

训练折内稳健表示

231 个化学特征与 9 个缺失指示列构成 240 维输入。对化学变量比较两种表示：存储尺度 z 分数与 Rank-Gauss 表示。

$$z_{ij}^{\mathrm{zs}} = \mathrm{clip}\left(\frac{x_{ij}-\mu_{j,\mathrm{train}}}{s_{j,\mathrm{train}}}, -8, 8\right), \qquad u_{ij} = \widehat{F}_{j,\mathrm{train}}(x_{ij}), \quad z_{ij}^{\mathrm{rg}} = \Phi^{-1}\{\mathrm{clip}(u_{ij}, \varepsilon, 1-\varepsilon)\}, \quad \varepsilon = (2n_{\mathrm{train}})^{-1}.$$

z 分数按冻结规则截断在 ± 8，避免单个极端值主导最小二乘；截断只发生在表示层，不改变原始浓度。重复观测使用中秩，验证样本沿用训练经验分布。目标只在所给标签尺度上中心化：$\widetilde{y}_{it} = y_{it} - \mu_t$，$\mu_t$ 为该折训练行的标签均值；代码不把目标除以训练标准差，也没有任何目标尺度字段参与求解。

图正则低秩多输出回归

模型同时利用感官维度之间的共享结构及化学变量的共线性，求解

$$\min_{B,b} \frac{1}{n_{\mathrm{train}}}\|\widetilde{Y} - ZB - \mathbf{1}b^{\top}\|_F^2 + \lambda\|B\|_F^2 + \gamma\,\mathrm{tr}(B^{\top}L_G B), \qquad \mathrm{rank}(B) \le r.$$

训练图的边权为 $w_{jk} = \widehat{\rho}_{jk}\mathbf{1}\{|\widehat{\rho}_{jk}| \ge \tau\}$，$w_{jj} = 0$，其中 $\tau = 0.5$ 在任何拟合之前冻结；$\widehat{\rho}$ 由训练折的中秩 Spearman 计算。取 $L_G = D_{|W|} - W$，其惩罚可写为

$$\mathrm{tr}(B^{\top}L_G B) = \frac{1}{2}\sum_{j,k}|w_{jk}|\|B_{j\cdot} - \mathrm{sign}(w_{jk})B_{k\cdot}\|_2^2.$$

正相关变量倾向具有相近系数，负相关变量倾向具有反向系数。$D_{|W|} - W$ 由 Gershgorin 圆盘定理保证半正定（特征值位于 $[0,\, 2\max_i \sum_j |w_{ij}|]$），因此实现中不再对特征值作截断；缺失指示列的图惩罚块设为零。

低秩解取闭式形式而非迭代近似：令 $Z_c, \widetilde{Y}_c$ 为训练块的中心化矩阵，

$$A = \frac{1}{n}Z_c^{\top}Z_c + \lambda I + \gamma L_G, \qquad C = \frac{1}{n}Z_c^{\top}\widetilde{Y}_c, \qquad B = A^{-1}CV_rV_r^{\top},$$

其中 V_r 为 $C^{\top}A^{-1}C$ 的前 r 个特征向量。该解是有秩约束问题的精确最小化点，$r = q$ 时退化为满秩惩罚解；实现只做 Cholesky 分解与三角回代，不显式求逆。

四个线性模型统一使用同一截距公式

$$b = \bar{y} - \bar{z}\,B, \qquad \widehat{y} = \bar{y} + (z - \bar{z})B,$$

其中 $\bar{z}, \bar{y}$ 为训练块均值。该约定使 “秩 50 且 $\gamma = 0$” 必须还原普通 Ridge；逐折核对的最大预测差为 4.00×10^{-15}，机器精度通过。

四个同协议模型与内层选参

同一进程中运行四个模型，预处理规则、截距约定、外层划分与分级切点完全一致：

模型	定义	内层搜索
逐目标训练中位数	每维取外层训练标签的中位数	无参数
普通 Ridge	$\gamma = 0$，$r = 50$（无秩约束）	表示、λ
图 Ridge	$\gamma > 0$，$r = 50$	表示、λ、$\gamma \in \{0.1, 1\}$
图正则低秩	$\gamma \ge 0$，$r \in \{3, 6, 10\}$	表示、λ、$\gamma \in \{0, 0.1, 1\}$、r

λ 的网格为 $\{0.1, 1, 10\}$，表示在 z 分数与 Rank-Gauss 之间选择。每个外层训练集内部再切 3 个内层折选参，内层网格的每一个单元都写入实验目录，使参数选择可逐格复核。外层评价使用 3 次重复 × 3 个外层折，重复种子在协议中冻结。

预测结果：同协议四模型对照

表 1　同一嵌套协议下四种模型的预测误差

模型	A/F30 等权 MAE	全 50 维 MAE	宏平均 R^2（A/F30）	Skill（A/F30）	相对中位数差值（A/F30）	相对中位数差值（全 50 维）
最小中位数基线	0.3982	0.3919	-0.0584	0.0000	0.0000	0.0000
普通 Ridge	0.3667	0.3582	0.1584	0.0791	-0.0791	-0.0860
图 Ridge	0.3664	0.3575	0.1576	0.0798	-0.0798	-0.0879
图正则低秩	0.3593	0.3505	0.1854	0.0977	-0.0977	-0.1056

注：三次重复 × 三次外折的折外池化结果；A/F30 为香气 15 维与风味 15 维的等权平均，“全 50 维”另加其余 20 维。Skill（A/F30）= 1 − MAE / 中位数基线 MAE；相对中位数差值 =（本模型 MAE − 中位数基线 MAE）/ 中位数基线 MAE，负值表示优于基线。普通 Ridge 固定无图惩罚（γ=0）且不含低秩项；图 Ridge 不含低秩项；图正则低秩的秩在 {3, 6, 10} 中由内层验证选参，图惩罚强度 γ 与正则强度 λ 的选参网格为 $\gamma \in \{0, 0.1, 1\}$、$\lambda \in \{0.1, 1, 10\}$。本表只描述同一协议下的相对拟合差异，不构成对任一分量的显著性结论。

同一协议下，中位数基线的 Aroma/Flavour 30 维等权 MAE 为 0.3982，全部 50 维为 0.3919。图正则低秩模型把 30 维 MAE 降到 0.3593，50 维降到 0.3505；平均 R^2 由基线的负值转为 0.1854，汇总 Skill 为 0.0977。

本文使用两个 Skill 口径，二者不可混用：逐维 $\text{Skill}_t = 1 - \text{MAE}_{t,\text{model}}/\text{MAE}_{t,\text{median}}$，以及汇总 $\text{Skill} = 1 - \overline{\text{MAE}}_{\text{model}}/\overline{\text{MAE}}_{\text{median}}$。前者正负混杂时后者仍可为正，因此逐维 Skill 图与汇总值必须分别解读。

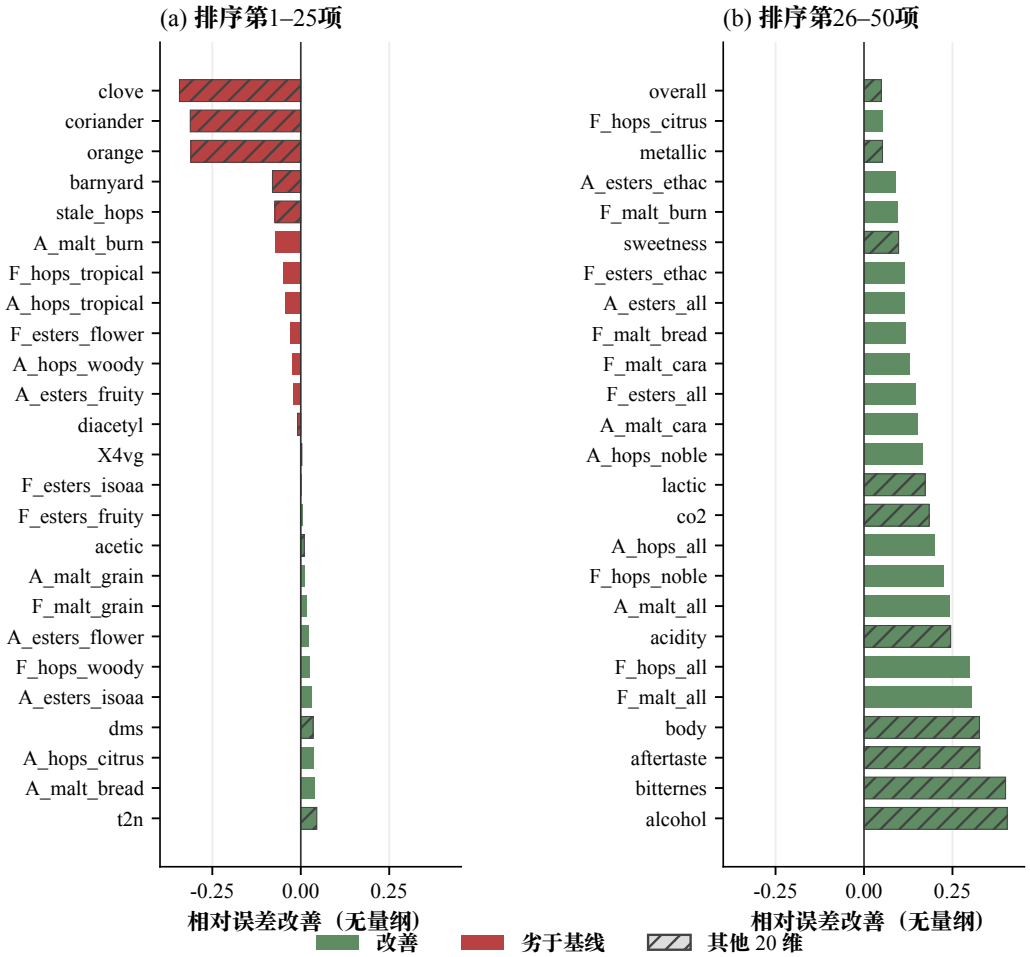


图 1　图低秩模型逐目标相对误差改善，按数值从小到大排序后分为两列。改善为 $1 - \text{MAE}_{\text{模型}}/\text{MAE}_{\text{中位数}}$，均取同一内部嵌套验证的折外预测；斜线为其他 20 维，12 维劣于中位数基线。

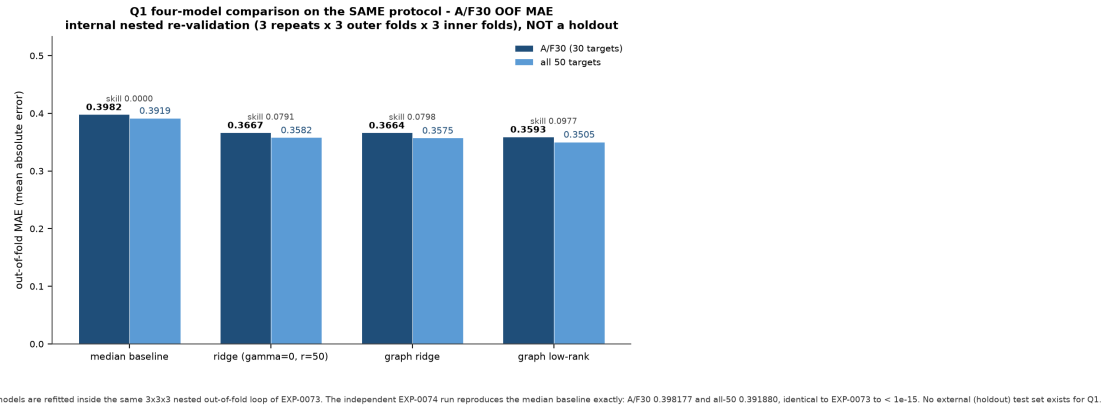


图 **2**　问题一：四模型同协议折外 MAE（内部 3×3×3 嵌套重验证，非 holdout）：中位数 0.398、岭 0.367、图岭 0.366、图低秩 0.359；全 50 维另列。

值得注意的是三个**负结果**，它们共同决定了本文对组件作用的措辞。

第一，普通 Ridge 的 30 维 MAE 为 0.3667，图 Ridge 为 0.3664，两者的差在小数点后第 4 位（表 1），远小于它们与中位数基线的差距。也就是说：在本协议下，**仅加入图正则几乎不改变误差**。

第二，图低秩相对普通 Ridge 的改善（0.3667 → 0.3593）同样不大。这里必须把归因写对：这一改善发生在“秩约束 $r \in \{3, 6, 10\}$ 被激活”的模型里，因此本文把它归因为**低秩收缩**；而“普通 Ridge → 图 Ridge”的差值属于图正则项的作用，两者都不是可以单独摘出来宣称的“加图收益”。我们没有跑“同协议、无图、但同样加低秩”的消融，所以**不保留“图结构带来预测增益”的主张**；图与低秩组件的作用被定位为“提供多输出共享结构、共线性去噪与可解释系数”。

第三，由于三条线性模型的误差都与中位数基线相差一个量级以内的改善，逐折波动与折间差异不能忽略：9 个折的 MAE 并不落在同一水平上（表 2），组间与折间差异的量级与模型间差异相当。因此任何“某组件稳定领先”的说法在本协议下都缺乏支撑。

表 **2**　指标分组误差与折间波动

对象	口径	MAE	样本数	同折中位数基线 MAE
香气 15 维（指标组）	指标组	0.3646	15	—
风味 15 维（指标组）	指标组	0.3540	15	—
其余 20 维（指标组）	指标组	0.3373	20	—
第 1 次重复 · 第 1 折	折外分段	0.3549	91	0.4024
第 1 次重复 · 第 2 折	折外分段	0.3450	83	0.3794
第 1 次重复 · 第 3 折	折外分段	0.3635	76	0.4124
第 2 次重复 · 第 1 折	折外分段	0.3643	76	0.4039
第 2 次重复 · 第 2 折	折外分段	0.3618	91	0.3882
第 2 次重复 · 第 3 折	折外分段	0.3614	83	0.4044
第 3 次重复 · 第 1 折	折外分段	0.3492	83	0.3771
第 3 次重复 · 第 2 折	折外分段	0.3731	76	0.4165
第 3 次重复 · 第 3 折	折外分段	0.3623	91	0.4026

注：上三行为三个指标组的折外池化 MAE 与组内维数；下九行为三次重复 × 三次外折逐折的折外 MAE、折内行数与同一折的中位数基线 MAE，用于展示折间波动而非模型间比较。逐折行数不同是因为外折按风格分层切分。

分组的结论是方向性的而非量级性的：口感、缺陷与总体评价构成的 20 维组，其误差低于 Aroma 与 Flavour 两个 15 维组；Aroma 组最高。逐风格结果（支撑材料表（支撑材料`/tables/q1-styles.md|csv`））进一步显示，样本量极小的风格误差明显更大，且同一模型在不同风格上的排序并不稳定，这与其训练行数直接相关。

逐维结果见支撑材料表 支撑材料`/tables/q1-targets.md|csv`（30 维，按 Skill 升序排列以暴露恶化维度）：总体 Skill 为正并不意味着每一维都改善，把汇总值当作逐维结论是本协议下最常见的误读。

0–6 相对等级与典型评分表

由于附件未提供原始整数评分及其标准化参数，本文定义可复核的相对强度等级。以该目标外层训练标签的中点经验分布函数 $\widehat{F}_t^{\mathrm{mid}}$ 标定：

$$a_{it} = \widehat{F}_t^{\mathrm{mid}}(\widehat{y}_{it}), \qquad g_{it}^{\mathrm{cont}} = 6a_{it}, \qquad g_{it} = \min\{6, \max[0, \lfloor 6a_{it} + 1/2 \rfloor]\}.$$

对观测侧使用**同一个** $\widehat{F}_t^{\mathrm{mid}}$，因此两侧处于同一标尺，可比较的只有“相对等级一致性”，不是与原始人工评分的一致性。预测落在训练分布中位附近时等级约为 3，靠近两端时趋向 0 或 6。0 不表示该风味不存在，6 不表示恢复了人工满分，缺陷维度的高等级也不等于超出物理阈值。

本节的表 3（相对等级一致性）、图 3 以及相邻表 4–5（Beer#1–Beer#10 的核心维度等级）**均来自折外评价侧**。表 3 与图 3 汇总三次重复的 750 行折外结果（37500 个等级格）；表 4–5 只展示首轮重复。每款啤酒的预测取自不含该啤酒的外折模型，参照等级按**该外折训练标签的经验 CDF** 映射。另一侧是展示侧：冻结候选在全部 250 行上重拟合一次，产物 `beer_scores_250.csv` 与 `grades_250.csv`，其参照分布按**固定的 200 行训练标签经验 CDF** 映射。两侧因训练样本与参照分布不同，在 Beer#1–Beer#10 的 5+5 个核心维度共 100 个等级格中有 32 格不同（已逐格核对），因此各司其职、不能互换引用：评价侧只进入表 3–5、图 3 与全部误差读数，展示侧只用于交付 250 款啤酒的连续预测与整数分级。等级只能读作“训练分布内的相对推断等级”，不能读作“重新标定的人工评分”；展示侧的样本内拟合误差也不作为模型性能，任一 MAE、R^2、κ 或基线数值都只从折外行重算。

维度映射的一项缺口。评分卡上印有 Acetaldehyde（青苹果、果皮），跨 Aroma 与 Flavour 两栏各出现一次，但标签表 `4.csv` 除 50 列之外没有这一项。本文在维度映射（`dimension_map.csv`，51 行）中把它单列为“有卡面文本、无标签列”的项：不预测、不分级、不与其他维度合并计数，也**不凭空造真值**。`7.csv` 里的同名 Ratebeer 描述符是外部支持数据（该列在本数据上缺失率 100%），不属于本问标签，未被映射进来。

表 3　各模型的相对等级一致性

模型	等级 MAE	整级一致率	二次加权 κ
图正则低秩	1.4111	0.2289	0.3321
图 Ridge	1.4391	0.2150	0.2590
普通 Ridge	1.4401	0.2105	0.2390
最小中位数基线	1.5072	0.1650	0.0000

模型或参照	等级 0	等级 1	等级 2	等级 3	等级 4	等级 5	等级 6
图正则低秩	1315	1915	4151	9493	13092	7140	394
图 Ridge	412	1007	3156	11488	15347	5985	105
普通 Ridge	141	699	2787	12102	16372	5388	11
最小中位数基线	0	0	0	37500	0	0	0
参照标签	3190	6229	6227	6189	6259	6211	3195

注：等级为训练折内经验中点分布映射出的 0–6 相对强度等级，参照等级是真实标准化标签经同一映射得到的等级；本表衡量的是相对等级的一致性，不是恢复人工感官评分，也不代表等级 0/6 对应原始评分卡的 0/6 分。二次加权 κ 与整级一致率均在 37500 个等级格（750 行 × 50 维）上计算；最小中位数基线的预测恒为中间等级，其等级分布只有一档，因此 κ 为 0。预测等级向中间收缩，说明离散等级仍有改善空间。

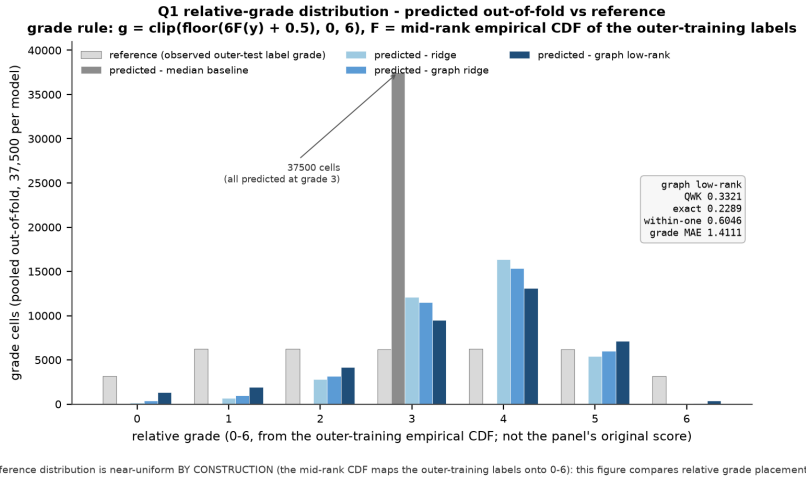


图 3　问题一：相对等级 0–6 的折外预测分布与参照分布对照（内部嵌套 CV、中秩经验 CDF；属相对等级，未恢复人工评分）。

表 4、表 5 依据 `beer` 的真实序号提取 Beer#1–Beer#10，与评分卡中的 Beer# 对齐，列出核心维度的相对等级，取**自首轮重复的折外预测**（评价侧）。全部 250 款啤酒 × 50 维的连续预测与整数分级另见展示侧 `beer_scores_250.csv` 与 `grades_250.csv`（全部 250 行重拟合、固定 200 行参

照 CDF），维度字典见 `dimension_map.csv`。两侧在这 100 个等级格中有 32 格不同，身份不同、不能互换引用。

表 4　Beer#1–Beer#10 的 Aroma 核心维度相对等级

样本	麦芽总体	酒花总体	柑橘	酯香总体	乙酸异戊酯
Beer#1	4/5	3/5	3/1	4/2	3/3
Beer#2	3/3	1/0	2/0	1/0	4/0
Beer#3	3/2	2/2	3/4	4/4	4/4
Beer#4	4/4	3/3	4/4	4/5	4/3
Beer#5	1/5	5/5	5/6	3/4	3/5
Beer#6	4/1	3/1	3/4	4/5	4/5
Beer#7	1/1	5/6	5/6	4/1	3/2
Beer#8	3/4	5/6	4/5	4/4	4/4
Beer#9	3/3	5/5	4/5	3/3	3/4
Beer#10	1/1	1/3	4/6	1/5	2/5

注：每个单元为“预测/参照”，顺序固定。等级为训练折内经验分布映射得到的 0–6 相对强度等级，预测列为折外预测、参照列为真实标签经同一映射的等级；样本编号与题面评分卡的 Beer# 对齐（第 n 号样本即 beer_n 评分卡对应的啤酒）；表内为首轮重复的折外结果，五维为核心香气维度（麦芽总体、酒花总体、柑橘、酯香总体、乙酸异戊酯）；等级是相对强度，不是人工评分恢复，也不表示风味存在与否。

表 5　Beer#1–Beer#10 的 Flavour 核心维度相对等级

样本	麦芽总体	酒花总体	柑橘	酯香总体	乙酸异戊酯
Beer#1	3/5	3/4	3/1	4/5	3/6
Beer#2	3/4	1/0	2/3	1/0	4/0
Beer#3	3/2	2/2	3/4	4/3	4/5
Beer#4	4/5	3/4	3/3	4/6	4/4
Beer#5	2/5	5/6	5/4	2/4	3/3
Beer#6	4/2	3/2	3/4	4/4	4/4
Beer#7	2/2	5/6	5/5	4/3	4/1
Beer#8	4/2	5/6	4/4	4/6	4/4
Beer#9	4/3	5/5	4/6	3/3	3/1
Beer#10	0/0	0/3	4/6	1/6	3/6

注：每个单元为“预测/参照”，顺序固定。等级为训练折内经验分布映射得到的 0–6 相对强度等级，预测列为折外预测、参照列为真实标签经同一映射的等级；样本编号与题面评分卡的 Beer# 对齐（第 n 号样本即 beer_n 评分卡对应的啤酒）；表内为首轮重复的折外结果，五维为核心风味维度（麦芽总体、酒花总体、柑橘、酯香总体、乙酸异戊酯）；等级是相对强度，不是人工评分恢复，也不表示风味存在与否。

样本间差异已经体现在整数等级中：同一维度在不同酒款上跨越多个等级，说明该维度在本数据内可分；但相邻等级之间没有可核对的原始评分作为标尺，等级差 1 不应读作“强度差 1 个单位”。

问题一的实际答案

1. **能预测到什么程度**：冻结 3×3×3 嵌套交叉验证下，图正则低秩候选的 Aroma/Flavour 30 维等权 MAE 为 0.3593、同协议中位数基线 0.3982，全 50 维 0.3505（基线 0.3919）；误差单位是所给标签尺度，不是 0–6 原始整数分。
2. **证据与产物**：主结论为问题一主实验（图正则低秩候选）相对同协议中位数基线实验的比较结果；稳健性覆盖三个执行块（参考块、种子块扰动、内层正则扰动）。可核对产物为 750 行折外预测、逐维/逐风格/逐折表（支撑材料表 支撑材料`/tables/q1-targets.md|csv`、`q1-styles.md|csv` 与表 2）、250 行展示评分与分级、51 行维度字典。
3. **限制**：改善主要来自低秩收缩而非图结构；0–6 只是相对等级；**没有独立外部测试集**，3×3×3 嵌套交叉验证的外折结果只评估本数据分布内的表现。

问题一的作用边界

本节只回答“本数据分布内、冻结协议下的折外表现”，有三条边界。

第一，**没有独立外部测试集**：每一行都是 3×3×3 嵌套交叉验证的外折预测，不存在从未参与拟合的外部样本；外折测试结果评价的是**本数据分布内**的表现，因此最强只到“内部折外重验证”，不宣称留出泛化。

第二，**固定种子复跑只证确定性重现**：逐位一致说明同环境可重复，但不引入新样本、不是独立实现，不能作为泛化证据。

第三，**结论只适用于当前候选、数据版本与协议**：更换候选、数据或划分后本节的误差与等级数字不可沿用。等级侧向中部收缩是结构性短板（表 3），且缺少原始人工评分作标尺。

问题二：共表达网络、风格混杂与稳定 Top-3

候选宇宙：严格单体化合物与扩展指标分表 231 个化学及理化变量并非同一类对象。按化学语义拆分为四类：单体化合物、元素或成分指数、理化参数、派生汇总。严格单体化合物构成 216 项，是“关键化合物因素”的主表候选；元素与成分指数 3 项、理化参数 7 项、派生汇总 5 项共 15 项构成扩展表。

表 6　候选宇宙构成与两张表的划分

类别	候选数	进入严格榜	进入扩展榜	说明
单体化合物	216	是	否	严格榜唯一允许的宇宙：参与 50 维 Top-3 选择
元素/指数	3	否	是	元素/指数项，非单体化合物，移出严格榜
理化参数	7	否	是	理化参数，非化合物丰度，只进扩展榜
派生和	5	否	是	派生汇总量，不是独立化合物，只进扩展榜
合计	231	—	—	候选矩阵为 231 个候选 × 50 个维度

注：严格榜只允许单体化合物，元素/指数、理化参数与派生和另一张扩展榜单独报告，两张表不得混排；同一化合物在不同维度上的候选一致。候选矩阵共 231 个候选 × 50 个维度 = 11550 个候选-维度单元。

两个用途不同的网络

本问涉及两个**用途不同、统计量不可混用**的网络，本轮各自有独立产物与图。

（一）训练折内网络。预测与关联模型使用的图每次都由当前训练行重新估计：中位数填补后的 231 列计算中秩 Spearman，取 $w_{jk} = \widehat{\rho}_{jk}\mathbf{1}\{|\widehat{\rho}_{jk}| \geq \tau\}$，$\tau = 0.5$ 在拟合前冻结，对角置零。它是正则化与结构化系数提取的输入，随 bootstrap 重抽样逐次重建；在问题二注册输入上该网络复现 732 条边、51 个连通分量（图 4）。

（二）附件 2 的描述性网络。为回答“这批样本里谁与谁共同变化”，本轮把附件 `2.csv` 的 231×231 相关矩阵按预先声明的规则（无向、$i < j$、$|\rho| \geq 0.5$、保留符号、无自环）构造成网络，并产出节点表、边表、阈值与规则文件、分量统计与图。实测节点 231、边 645、连通分量 52、最大分量 175；正边 629、负边 16；度中位 4、最大 30；孤立节点 48。该网络与附件同源，只用于本章“化学合理性”一节的模块归属与正负相关格局解释。

两者分工与禁止事项。描述性网络回答“结构长什么样”，训练网络只承担正则化；连通分量只是阈值图上的传递连通，不等于经功能验证的生物化学模块，因此模块标签不进入 Top-3 综合打分。**附件 2 的全样本矩阵不进入任何验证**：它不会作为预测或关联模型的正则化图，也不参与 Top-3 选择与 bootstrap；独立预测用的图始终在训练行内重建，避免把全样本结构泄漏到评价里。反过来，训练网络的边数与分量数也不能当成“附件 2 网络”的统计量引用。

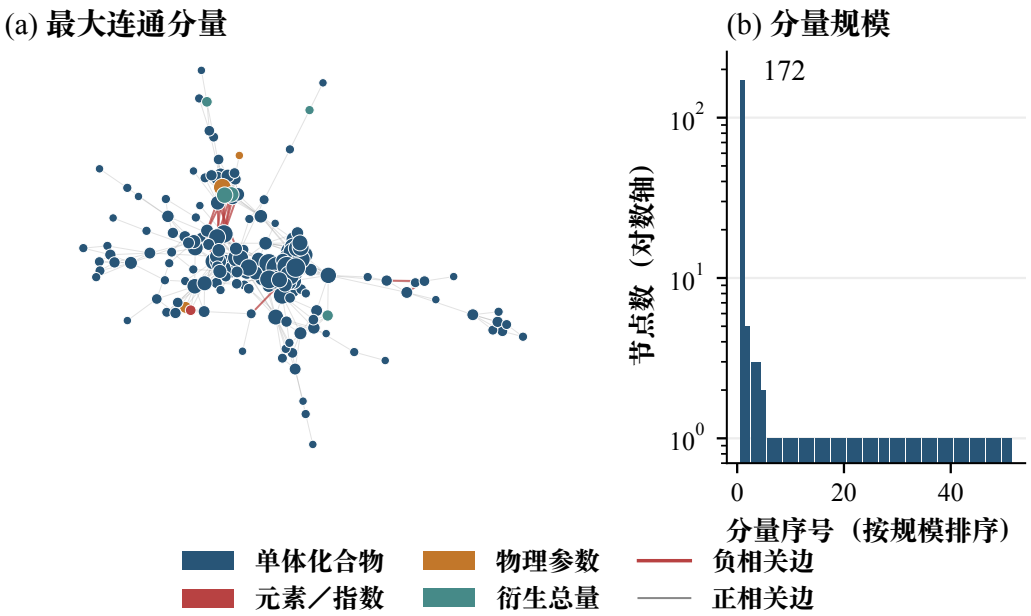


图 **4**　训练数据的 Spearman 相关网络（$|\rho| \geq 0.5$）：231 节点、732 边、51 分量。左图为最大分量（172 节点），右图为全部分量规模；颜色表示变量类别，大小表示节点度。该全训练集描述图不替代折内建图，相关边不表示因果。

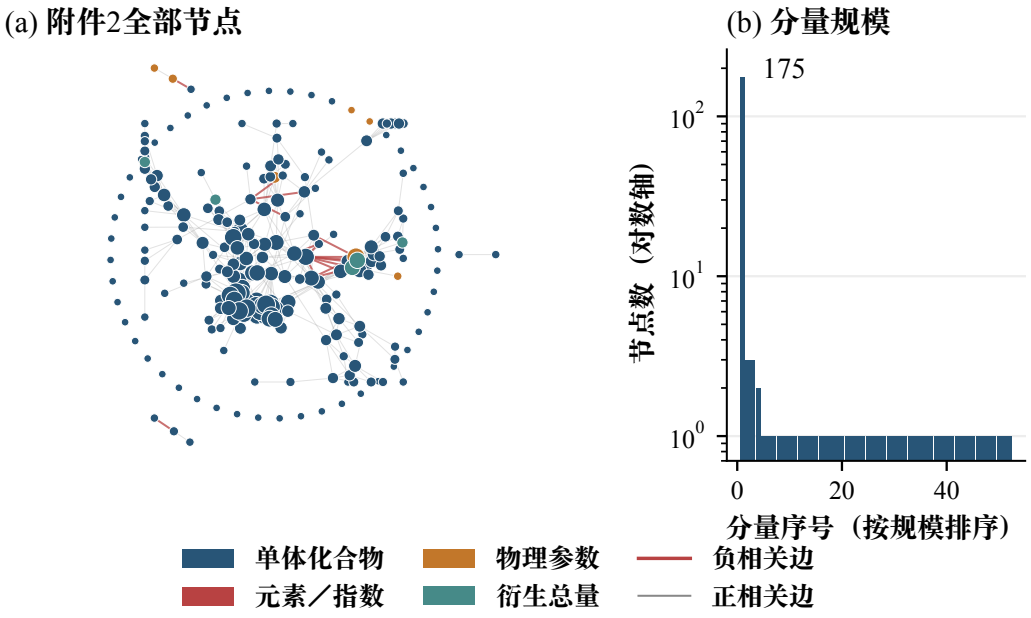


图 **5**　附件 2 的描述性 Spearman 相关网络（$|\rho| \geq 0.5$）：231 节点、645 边、52 分量，最大分量 175 节点。节点位置读取已存 `nodes.csv`；颜色表示变量类别，大小表示节点度。该全样本网络不进入预测验证。

风格混杂与残差化关联

总体关联定义为 $\rho_{jt}^{\text{total}} = \text{Spearman}(X_j, Y_t)$。为分离组间风格差异，令 H 为风格虚拟变量与截距组成的设计矩阵，对秩残差化：

$$P_H = H(H^\top H)^\dagger H^\top, \quad r_j^X = (I - P_H)\,\text{rank}(X_j), \quad r_t^Y = (I - P_H)\,\text{rank}(Y_t), \quad \rho_{jt}^{\text{within}} = \text{Corr}(r_j^X, r_t^Y).$$

该指标反映去除风格均值差异后的共同关联。风格内残差化能减少组间差异的影响，但不能消除未观测混杂；因此本文同时保留总体关联，让两种口径并列，而不是用一种替换另一种。

四种方法与综合打分

对候选 j 与目标 t 使用四种方法：总体 Spearman、风格内残差化 Spearman、图低秩回归系数 $\widehat{B}_{jt}$、以及三者的共识分

$$S_{jt} = \frac{1}{3}\left[R\Big(|\rho_{jt}^{\text{total}}|\Big) + R\Big(|\rho_{jt}^{\text{within}}|\Big) + R\Big(|\widehat{B}_{jt}|\Big)\right],$$

其中 R 为同一目标内、该表自身候选宇宙上的百分位秩；$\widehat{B}$ 来自固定 $\tau = 0.5$, $\lambda = 1$, $\gamma = 0.1$, $r = 6$ 的图低秩模型。四种方法各自给出 Top-3；交付表的排序规则是**入选频率优先**：先按 π 降序，π 并列时用原样本综合分 S 裁决，再并列时沿用固定特征名字符序。方法间差异本身作为稳定性证据保留，不通过换排序规则来消除。

稳定性用风格分层 bootstrap 度量：在注册的 200 行训练集内，按 `tasting_category_fine` 分层、每层有放回重采样到自身规模，共重采样 500 次。每个重抽样样本都重新执行中位数填补、$\tau = 0.5$ 的 Spearman 图估计、图低秩回归、风格内秩残差化与三项子分数，然后重新取 Top-3：

$$\pi_{jt} = \frac{1}{B}\sum_{b=1}^{B}\mathbf{1}\{j \in \text{Top3}_t^{(b)}\}, \qquad \text{se}_{\text{mc}} = \sqrt{\frac{\pi_{jt}(1-\pi_{jt})}{B}}.$$

最终表按 π 降序排列，并列时按原样本综合分 S 裁决，再并列时沿用固定特征名的字符序。S 表征原样本中的综合关联强度，π 表征重抽样下的入选稳定性；候选方向取总体 Spearman 的符号，只是关联方向，不代表干预效应。每个入选条目旁同时报告**首个合格同模块替代候选**：在该参考模块的点分序中剔除本维已入选 Top-3 后取第一个候选（因此它不再是固定的“模块点分第 4 名”），既让第三名的不稳定性不被隐藏，也保证替代项不与 Top-3 重复。

方向稳定性按“与点估计同方向”定义。对每个（候选，维度）单元，令点方向为该单元在 200 行训练集上的总体 Spearman 点估计符号；每一次 bootstrap 复制重算同一单元的总体 Spearman，并按预先固定的容差 10^{-12} 分桶：

$$\text{sign_stability}_{jt} = \frac{\#\{\text{复制与点方向同号}\}}{B - n_{\text{invalid}}}, \qquad p_{\text{positive}} = \frac{n_{\text{positive}}}{B - n_{\text{invalid}}}.$$

点估计为负时，“同号”就是负方向；零（$|\rho| \le 10^{-12}$）与不可计算（非有限值）都单列计数，绝不并入负方向，$1 - p_{\text{positive}}$ 也不再被当作负方向频率。这一口径描述的是**总体相关方向**的稳定性，不是低秩模型系数方向，也不是因果方向。旧口径 $P(\rho > 0)$ 把方向为负的条目一律计成“不一致”，其数值 0.7226 属定义造成的偏低，本文只作历史对照。

核心与完整 Top-3

核心 20 个 Aroma/Flavour 维度的 Top-3 见表 7；完整 50 维、每维 3 项共 150 条目（支撑材料表（支撑材料/`tables/q2-full-top3.md|csv`））以及 15 个扩展指标在 50 维上的 Top-3（支撑材料表（支撑材料/`tables/q2-extended.md|csv`））放在附录 A，同数据 CSV 与完整溯源在 支撑材料/`tables/INDEX.json`。三张表的候选集合不同，不能相互替代或合并计数；放在附录只为控制正文篇幅，数据行未删减。

表 7　核心维度 Top-3 关键化学因素、入选频率与方向

感官维度	第 1 位	第 2 位	第 3 位	第 3 位同模块替代候选
香气 · 麦芽	furfural	X2.acetylfuran	beta.phellandrene	methylpyrazine
总体	0.490/0.9767 | + · 内	0.422/0.9597 | + · 混	0.340/0.9457 | − · 混	
香气 · 谷物	terpinen.4.ol	furfural	propylene.glycol	—（无）
	0.296/0.9628 | − · 混	0.252/0.9566 | + · 混	0.200/0.9008 | − · 内	
香气 · 面包	beta.phellandrene	X4.ethyl.guaiacol	furfural	terpinen.4.ol
	0.360/0.9581 | − · 混	0.302/0.9457 | − · 内	0.224/0.9287 | + · 内	
香气 · 焦糖	X2.methyl.benzofuran	furfural	X2.ethyl.1.hexanol	beta.phellandrene
	0.634/0.9674 | − · 内	0.328/0.9674 | + · 混	0.238/0.8961 | − · 混	
香气 · 烘烤	ethyl.hydrocinnamate	selin.11.en.4.alpha.ol	SO2.mg.L.	—（无）
	0.558/0.9860 | + · 内	0.228/0.9380 | + · 内	0.194/0.9287 | − · 混	

续表 7　核心维度 Top-3 关键化学因素、入选频率与方向

感官维度	第 1 位	第 2 位	第 3 位	第 3 位同模块替代候选
香气 · 酒花总体	ethyl_octanoate 0.832/0.9829 | + · 混	cis.dihydro.beta. terpineol 0.270/0.9271 | + · 风	DMS 0.260/0.9209 | + · 风	terpinolene
香气 · 柑橘	citronellol 0.596/0.9674 | + · 混	geraniol 0.502/0.9581 | + · 混	X1.alpha.terpineol 0.238/0.9271 | + · 内	cis.dihydro.beta. terpineol
香气 · 热带水果	beta.phellandrene 0.666/0.9860 | + · 内	DMS 0.648/0.9953 | + · 混	X.E..dihydro. apofarnesal 0.202/0.9380 | − · 混	—（无）
香气 · 贵族酒花	ethyl_octanoate 0.932/0.9953 | + · 混	ethyl_decanoate 0.692/0.9721 | + · 混	cis.dihydro.beta. terpineol 0.200/0.9302 | + · 混	isopentyl.hexanoate
香气 · 木质	X.E..ortho.methoxy. cinnamaldehyde 0.306/0.8837 | + · 内	hydroxy.methyl.furfural 0.188/0.8837 | + · 内	nonan.2.ol 0.180/0.9209 | + · 内	ethyl.hydrocinnamate
香气 · 酯香总体	acetaldehyde 0.928/0.9984 | + · 内	isopentyl_acetate 0.458/0.9752 | + · 内	isoamyl_alcohol 0.416/0.9752 | + · 内	X2E.6Z.farnesal
香气 · 乙酸乙酯	X2E.6Z.farnesal 0.616/0.9922 | + · 内	acetaldehyde 0.360/0.9442 | + · 混	phenethyl_alcohol 0.348/0.9457 | + · 内	—（无）
香气 · 乙酸异戊酯	isopentyl_acetate 0.610/0.9876 | + · 内	acetaldehyde 0.448/0.9845 | + · 内	X2E.6Z.farnesal 0.362/0.9798 | + · 内	isoamyl_alcohol
香气 · 花香	beta.phellandrene 0.530/0.9907 | + · 内	acetaldehyde 0.358/0.9860 | + · 内	X.E..dihydro. apofarnesal 0.300/0.9411 | − · 内	—（无）
香气 · 果香	beta.phellandrene 0.596/0.9814 | + · 混	lactic_acid.mg.L. 0.166/0.8744 | + · 混	propylene.glycol 0.158/0.7767 | − · 内	—（无）
风味 · 麦芽总体	acetaldehyde 0.686/0.9876 | + · 风	MeSH 0.220/0.8977 | + · 风	X.E..alpha.ionone 0.204/0.9364 | − · 风	ethyl.hydrocinnamate
风味 · 酒花总体	ethyl_octanoate 0.890/0.9798 | + · 混	cis.dihydro.beta. terpineol 0.506/0.9659 | + · 混	DMS 0.294/0.9349 | + · 混	ethyl_decanoate
风味 · 柑橘	geraniol 0.600/0.9876 | + · 混	geosmin 0.260/0.9535 | + · 混	phenethyl_alcohol 0.232/0.9240 | − · 内	—（无）
风味 · 酯香总体	acetaldehyde 0.976/1.0000 | + · 混	isopentyl_acetate 0.462/0.9674 | + · 内	isoamyl_alcohol 0.336/0.9690 | + · 混	ethyl_acetate
风味 · 乙酸异戊酯	isopentyl_acetate 0.716/0.9876 | + · 内	acetaldehyde 0.534/0.9767 | + · 混	isoamyl_alcohol 0.410/0.9705 | + · 内	ambroxide

注：第 1–3 位单元格分行显示名称与“π/S \| 方向 · 类型”。π 为 500 次风格内分层 bootstrap 的入选频率；S 为**三种基础方法（总体 Spearman、风格内残差 Spearman、图低秩系数）在维度内百分位排名的均值共识分**；方向为总体 Spearman 相关的符号；类型按原脚本口径判定——“内”表示 | 风格内相关 | ≥ 0.8| 总体相关 |，“风”表示未满足前者且 | 总体相关 | ≥ 0.25、| 风格内相关 | < 0.5| 总体相关 |，其余为“混”。维度取香气 15 维全部与风味 5 个核心维度；完整 50 维见附录 A 的长表。最后一列为第 3 位的**同模块替代候选**：先剔除该维度已进入 Top-3 的候选，再取同参考模块内点估计排序最靠前的第一个未入选候选，因此它不会与同维 Top-3 重复（150 条严格共识与 600 条四方法交付表的重合数均为 0）；“无”表示该参考模块内不存在这样的候选。

表中 “类别” 依据绝对关联强度分类：风格内关联不低于总体关联绝对值的 0.8 倍记作 “内”，未满足前式且总体关联绝对值不低于 0.25、风格内关联低于总体 0.5 倍记作 “风”，其余记作 “混”。该分类依据强度变化构造，未单独检验风格间符号异质性。

稳定性、方法一致性与同源对照

逐项稳定性读数见表19。全部条目的入选频率均值为 0.4205，其中 1263 / 1800 个条目低于 0.5：多数候选在重抽样下仍与同维其他候选竞争，单维 Top-3 不是稳定不变的结论。按交付的严格共识榜（50 维每维 3 项，共 150 条）统计，其中 95 条的 $\pi < 0.5$；这 95 条里方向为负的 29 条、为正的 66 条。

三次重复之间的集合一致性为 0.8748，明显高于四方法之间的一致性 0.2091；也就是说，不稳定主要来自方法口径差异，而不是重复抽样噪声。按点估计口径（每次重复取点估计 Top-3 再平均）的平均两两 Jaccard 为 0.1977，与 π 优先口径不同，两者描述不同的集合规则，不能互相替换。

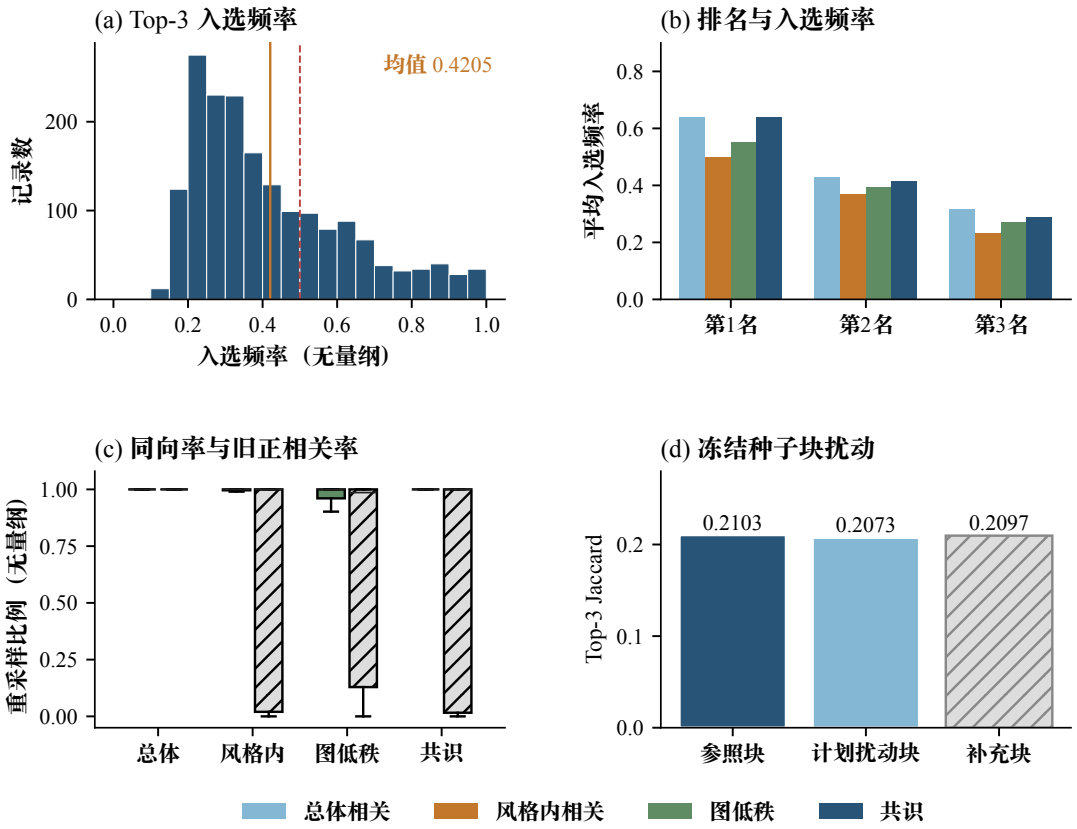


图 **6**　Top-3 的入选频率与方向稳定性（3 个冻结重复、4 种方法，共 1800 条记录；每块 500 次风格分层重采样）。虚线 0.5 为参考线。图 (c) 实色为与点估计同向的比例，斜线为旧正相关比例；两者不等价。图 (d) 前两块属于计划内扰动，第三块为补充结果。

方向一致率按三个不同的分母分别报告，三者不可混用。第一个是首轮 150 条严格共识的均值 0.9995（0.9994667）：它与入选频率同表，说明这套候选的方向在重抽样下几乎不变。第二个是单轮 600 条四方法交付格的均值 0.9820（0.9819800）：每个（维度，方法，名次）格各自算一次一致率再取均值。第三个是三次重复池化 1800 条的均值 0.9817（0.9816878），也是一般引用时应给的口径。在 $600 \times 500 \times 3 = 900{,}000$ 个 bootstrap 复制上，只有极少数复制与该单元的点估计方向相反。作为历史对照，旧口径 $P(\rho > 0)$ 为 0.7226，它把方向为负的条目一律算作“不一致”，因此天然偏低，不得再称为符号稳定性。

分项计数与均值一起读，两个分母不能混算：交付的 1800 个（四方法 × 三重复池化）条目中，点估计方向为负 503 个、方向不确定 0 个；复制层面零方向 0 次、不可计算 0 次（都单列，不并入任何方向，也不写成“负方向频率”）。150 条严格共识中负 36 条、正 114 条；这 36 条的逐条方向一致率 0.978–1.000（见 `sign_stability_summary.json`），低旧值来自定义而非方向不稳。但方向稳定不等于入选稳定：其中 29 条 $\pi < 0.5$、仅 7 条 $\pi \geq 0.5$。这些条目的替代候选见表 7 最后一列，第 3 位频率偏低时应与正选并读。

附件 `6.csv` 的 44 行化学—感官交叉相关按名称对齐后，平均秩相关为 0.9210（严格子集 0.9080）。两个限制：与本研究同源，属同源对照而非外部验证；比较用绝对关联排序，只说明排序强度一致，不说明方向正确。按位置对齐的旧值偏低正是列错位三列所致，已作历史记录。

化学合理性：机理支持与关联标志分层

酯类与果香。乙酸异戊酯、乙酸乙酯等是与酵母发酵相关的酯类，支持“酯香候选以酯类为主”的类别解释；啤酒化学—感官研究亦显示酯类与醇类共同变化，单一变量的重要性可能替代共相关的其他化合物 [2]。因此 Top-3 出现非酯类条目（如醛类）应读作代谢或工艺关联标志，不能由排序推导添加效果——该解释在本数据上属待证，本文未做加标或酿造干预。

萜烯类与酒花柑橘、花香。香茅醇与香叶醇具描述词一致性，萜烯可随酒花或香料引入，风格差异也可能改变其出现方式 [2]。表 7 中标“混”的候选说明其排序不能脱离配方与风格解释，本文不对其作用强弱排序。

高级醇、有机酸与醇厚、酸感。甘油等可与口感联系，有机酸与酸感具类别一致性 [2]；但未进

Top-3 的已知风味物质不能据此排除，共线候选间的名次差也不等于作用强弱。

硫化物、双乙酰与缺陷。双乙酰可由 α-乙酰乳酸转化、也可被酵母还原去除 [3]；DMS 既有热过程前体途径，也可由 DMSO 还原产生 [4]。这些机理支持把双乙酰与 DMS 作为缺陷监测对象，但统计排序与已知直接致味物并非一一对应，因此问题三直接约束已测得的 DMS、双乙酰与 4VG，而不以 Top-3 替代化学检测。

问题二的实际答案

1. **给出了什么**：严格单体 216 项（另有 15 项扩展指标单列）在 50 维上的逐维 Top-3，带入选频率 π、共识分 S、总体相关方向与首个合格同模块替代候选（已剔除本维 Top-3）；核心 20 维见表 7，完整 150 条与扩展表见附录 A 索引。附件 2 描述性网络（231/645/52）与训练折内网络（732 边/51 分量）是两种用途的独立产物。
2. **稳定性到什么程度**：方法间 Top-3 平均两两 Jaccard 0.2091、跨重复 0.8748；π 均值 0.4205，1800 个交付条目中 1263 个低于 0.5；方向一致率 0.9817（旧口径 0.7226）。
3. **证据归属**：主结论为问题二主实验的共识榜结果；稳健性为其预登记的种子块计划（计划内参考块与扰动块，另有登记后补跑的一块）。计划范围与执行读数见表18及“稳健性与限制”一节。
4. **限制**：Top-3 是排序假设而非已识别清单，150 条中 95 条入选频率低于 0.5；方向稳定不等于入选稳定；同期关联不是因果；同源对照不是外部验证。

问题二的作用边界

第一，**同期关联不等于因果**：任何 “某化合物决定某风味” 的表述都不成立，Top-3 是排序假设，需独立干预实验检验。

第二，**集合一致性与方向一致率都不是识别准确率**：方法间与跨重复 Jaccard 只描述排序分歧度，没有真值可对照；方向一致率只说同一单元总体相关方向在重抽样下可复现，既不涉及低秩系数方向，也不涉及因果。

第三，**同源对照不是外部验证**：`6.csv` 与描述性网络都与附件同源，只核对流程未把排序弄乱；描述性网络只是阈值图结构，不是功能模块或因果网络。

第四，**入选频率普遍偏低**：150 条严格共识中 95 条 $\pi < 0.5$，“方向可复现” 与 “名次稳定” 是两个问题，替代候选须与正选并读。

问题三：风格条件生成与声明模型下的迁移

问题三包含两个不同对象：其一是按风格生成化学谱的条件生成器，其二是把 Pils/Lager 谱在声明约束下迁移到 Tripel 的优化算子。两者的指标、失败模式与边界都不同，本文分开评价，不合并为单一的 “问题三成功率”。

生成目标与评价对象

主任务学习已出现风格的条件分布 $p(X \mid s)$，采用固定 3 折风格内留出：每次重复 22 个风格各 3 折，设计格总数 198、生成样本 750 份。能量距离只在**可估格**上算：177 个可估格、729 份可估生成；另有 21 个格因对照侧样本不足被排除并单列。可估数与总数不可互换。标签不进入化学边际或 Copula 的似然拟合，只用于独立的感官代理与评价。

原始抽样与约束设计谱的分离

本轮同时保存 `raw`（条件生成原始抽样）与 `design`（约束投影后设计谱），目的是让投影代价可测。33 个已校准线性列在抽样层即施物理非负下界，故两份谱的负浓度格数均为零——这是构造保证，不是 “生成质量好” 的证据。

表 8　原始抽样与约束设计谱的对照

指标	未约束原始生成	约束后设计谱	说明
归一化经验能量距离 V	0.8736	0.8727	按 n_gen 加权池化；越小表示生成谱越接近真实谱
无符号能量 V	16.0178	15.9999	非负的分布差异量
带符号能量 U	1.6044	1.6154	可负、不截零；U 的符号只表示带符号统计量的读数方向，不用它推断偏移方向或过度分散
约束满足率	0.1267	1.0000	同时满足全部声明约束的样本比例；设计侧由投影循环保证
负浓度格数	0	0	线性非负约束被违反的格数
约束投影轮数（均值）	—	1.3232	逐格迭代直到最大残差 ≤ 1e-9；允许轮数上限 12
约束投影轮数（最大）	—	4	允许轮数上限 12，本轮实际最大 4
约束投影最大残差	—	0.0000	投影后从写出的样本文件复核
全列平均绝对改动（标准化单位）	—	0.0089	约束投影相对未约束原始抽样的逐列平均绝对改动
改动最大的列：二甲基硫（DMS）	—	0.5720	约束投影相对未约束原始抽样的逐列平均绝对改动
改动最大的列：双乙酰（2,3-丁二酮）	—	0.2888	约束投影相对未约束原始抽样的逐列平均绝对改动
改动最大的列：反-2-壬烯醛	—	0.0513	约束投影相对未约束原始抽样的逐列平均绝对改动

注：左侧为未施加约束的原始条件生成，右侧为约束投影后的设计谱。能量距离按可估格加权池化，V 与 U 是同一能量统计量的无符号与带符号读数。约束满足率在未约束原始生成上为 0.1267，在设计谱上为 1.0000，后者是投影循环的构造性质，不能当作分布质量证据。设计谱中真实化学反-2-壬烯醛的增加条数为 0；感官 t2n 代理的增加条数另计（366 / 750），两者是不同命题，不能互相替代。改动量以标准化单位给出，只描述“投影改了多少”，不是约束残差。

两项必须分开读：能量距离从原始抽样到设计谱的变化量度量“投影让分布偏离多少”；约束满足率从 0.1267 变为 1 则是**构造性质**（投影保证），不提供真实酿造可行性信息。未约束原始抽样只有小部分满足全部约束，说明缺陷约束在生成中是有效约束，但这同样只是数学筛查。

两个缺陷命题必须分开报告。真实化学列反-2-壬烯醛在设计谱中**不增**：相对原始抽样增加 0 条（最大增量 0）。**感官代理** t2n 仍增加 366 条（0.488，分母 750 份生成）。前者是化学列检查通过，后者说明感官口径的“老化风味不增”未满足；代理不是实测感官、也未写成硬约束，本文不通过改措辞把它算作已满足——要变成硬约束需另行检查可行域并重跑。

含编码质量点的层次收缩边际

对每个变量采用风格内经验边际与池化边际的**两层收缩**，收缩权重不是风格行数，而是该风格该列的**有效观测数** $n_{j,s}^{\mathrm{eff}}$（未填补、非缺失），$\omega_{j,s}=n_{j,s}^{\mathrm{eff}}/(n_{j,s}^{\mathrm{eff}}+\kappa)$。之所以分两层，是因为含地板质量点的变量由“地板概率 π”与“非地板条件分布 H”两部分组成，两者必须分别收缩后再合成：

$$\bar{\pi}_{j,s}=\omega_{j,s}\pi_{j,s}^{\mathrm{style}}+(1-\omega_{j,s})\pi_{j}^{\mathrm{pool}},\qquad \bar{H}_{j,s}(v)=\omega_{j,s}\widehat{H}_{j,s}^{\mathrm{style}}(v)+(1-\omega_{j,s})\widehat{H}_{j}^{\mathrm{pool}}(v),$$

$$\widehat{F}_{j,s}(v)=\bar{\pi}_{j,s}+(1-\bar{\pi}_{j,s})\bar{H}_{j,s}(v)\quad(v>v_{j}^{\mathrm{floor}}),\qquad \widehat{F}_{j,s}(v_{j}^{\mathrm{floor}})\text{ 处赋质量点 }\bar{\pi}_{j,s}.$$

当风格与池化的地板比例不同（$\pi_{j,s}^{\mathrm{style}}\neq\pi_{j}^{\mathrm{pool}}$）时，上述两层收缩**不等同于**把两个完整 CDF 直接混合；两者的差异只出现在同时含地板质量点、且两侧地板率不同的列上。

经验边际只用原始有效观测：填补值即使写入输入矩阵也不进入经验样本；$n_{j,s}^{\mathrm{eff}}<3$ 时走预声明的 pooled fallback 并输出状态；连续部分与地板质量点分母分别声明。生成外层/内层、迁移目标/源边际四个入口共用同一套边际语义。

有地板编码的变量另以混合分布保留编码质量点：

$$F_{j,s}^{\mathrm{obs}}(v)=\pi_{j,s}^{\mathrm{floor}}\mathbf{1}\{v\geq v_{j}^{\mathrm{floor}}\}+(1-\pi_{j,s}^{\mathrm{floor}})F_{j,s}^{\mathrm{nonfloor}}(v).$$

按相应概率抽取编码值，其余用经验分位数插值。该结构避免把大量重复地板值平滑成虚假连续浓度：37 个对数列的 755 个校准零值与 142 列的 6807 个共享地板编码都作为质量点保留。

收缩高斯 Copula

将条件边际映射至高斯坐标，估计风格依赖结构：

$$z_{ij} = \Phi^{-1}[\widehat{F}_{j,s_i}(x_{ij})], \quad \widehat{R}_s = \eta_s \widehat{R}_s^{\text{emp}} + (1-\eta_s)\widehat{R}_{\text{shared}}, \quad \eta_s = \frac{n_s}{n_s + \kappa_R}.$$

相关矩阵特征值截断至 $\geq 10^{-6}$，归一化后向单位阵收缩 5%；采样 $z^* \sim N(0, \widehat{R}_s)$，经 $u^* = \Phi(z^*)$ 与逆边际得 X^*。小样本风格更依赖池化信息。

参数选择在每个外层折内部用该折自己的 3 个内层折完成：$\kappa \in \{0, 5, 15\}$、$\kappa_R \in \{20, 50, 100\}$ 的 9 个组合逐折选择，不跨折池化、不用外层留出误差挑参。（先导轮曾把逐折选择表平均写回，使留出信息进入超参数；本轮已改为逐折独立。）

评价量：分布距离与结构统计

归一化经验能量距离为

$$\mathcal{E}^2_{\text{norm}} = \frac{\max(\mathcal{E}^2, 0)}{D_{\text{ref}}}, \qquad \mathcal{E}^2 = 2\mathbb{E}\|X-Y\| - \mathbb{E}\|X-X'\| - \mathbb{E}\|Y-Y'\|,$$

其中 D_{ref} 是该风格训练行的典型两两距离（D_{train}），归一化只用它除一次。V 与 U 不是同一个量的两种写法，对角项与分母都不同，必须写清：

$$V = 2\overline{d(G,R)} - \frac{\sum_{i,i'} d(G_i, G_{i'})}{m^2} - \frac{\sum_{j,j'} d(R_j, R_{j'})}{n^2}, \qquad U = 2\overline{d(G,R)} - \overline{d(G,G)}^* - \overline{d(R,R)}^*,$$

其中 m, n 为生成与参照样本数，$\overline{d(\cdot,\cdot)}^*$ 在 $m(m-1)$、$n(n-1)$ 个非对角对上取平均。V 用全体有序对之和除以 m^2（含对角零项），U 用非对角均值，故 $V - U = \frac{\sum d(G,G)}{m^2(m-1)} + \frac{\sum d(R,R)}{n^2(n-1)} \geq 0$、$V \geq U$ 且 $V \geq 0$。U 可负；本文不截零、不开方，也不以其正负判断偏移方向或过度分散（那需另做检验，本文未做）。

此外分别报告边际误差、依赖谱 MAE、多样性（样本间平均距离）、最近邻距离、批内整行重复率与边饱和比例。它们各自回答一个问题，不能互相替代。

生成质量与参照对照

表 9　生成方案的分布质量与同协议参照对照

方案	归一化能量距离 V	带符号 U	多样性	最近邻距离	批内整行重复率	边饱和比例	范围说明
分层收缩混合 Copula 设计谱（主方案）	0.8727	1.6154	17.4190	15.5202	0.0000	0.1237	能量/U/多样性/最近邻按 177 格 · 729 份加权；批内重复率按 750 行；边饱和按 173250 个单元格
仅共享相关 Copula	0.8718	1.5472	17.6151	15.8771	0.0000	0.1244	能量/U/多样性/最近邻按 177 格 · 729 份加权；批内重复率按 750 行；边饱和按 173250 个单元格
风格内经验 Copula	0.8655	1.4990	17.0534	14.4323	0.0000	0.1173	能量/U/多样性/最近邻按 177 格 · 729 份加权；批内重复率按 750 行；边饱和按 173250 个单元格
独立边际	0.8697	1.4959	17.6165	16.3891	0.0000	0.1277	能量/U/多样性/最近邻按 177 格 · 729 份加权；批内重复率按 750 行；边饱和按 173250 个单元格
源样本重采样参照	0.8137	0.1806	18.3913	16.0210	0.0000	0.1197	同风格训练行重采样参照，不是生成模型；能量/U/多样性/最近邻按 177 格 · 729 份加权；批内重复率按 750 行；边饱和按 173250 个单元格
范围口径：21 个可估风格 · 177 个可估格池化（归一化 V）	0.8727	1.6154	—	—	—	—	n_gen=729；未归一化 V = 15.9999；与主方案同值，用于声明池化范围
范围口径：Tripel/Kriek/Lambic 三风格 · 27 个格单独池化（归一化 V）	1.8079	1.0279	—	—	—	—	n_gen=147；未归一化 V = 33.7829；与上一行不是同一范围，不可互换

注：前五行为同一评价协议下不同采样方案的设计谱比较，能量距离越小越好；U 可负、不截零。**分母不同，不可混读**：归一化能量距离、带符号 U、多样性与最近邻距离按 177 个可估格、729 份生成的 n_gen 加权口径计算；**批内整行重复率**的分母是 750 行生成样本（衡量单批次内部是否出现重复行），**边饱和比例**的分母是 231×750 = 173250 个存储单元格，其中包含未进入能量距离的 21 个不可估格 / 21 份生成，因此两类指标的分母不是同一批格。四个分布侧指标各自回答不同问题，不可互相解释：多样性为生成样本在标准化空间中的两两欧氏距离均值（按固定抽样次数子采样）；最近邻距离为**生成批次内部**每个样本到最近样本距离的均值（不涉及训练集）；批内整行重复率按各批次的 $1 - n_{\text{unique}}/n$ 计算，再按生成样本数加权汇总（总计 750 行）；边饱和比例为落在声明边界上的单元格占比（地板编码、观测支撑上下界、线性列零值）。源样本重采样参照直接复用同风格的训练行，其能量距离低于各生成方案；该行的作用是给出“不生成”的参照，不能与生成模型等同比较。后两行只声明池化范围：21 个可估风格 177 个可估格的池化值与三风格单独池化值不可互换，两者的带符号 U 也随范围变化，引用时必须写明范围。**跨集指标**：交付设计谱相对同折训练集的整行精确重合率为 0.0000（0/750），到最近训练样本的标准化欧氏距离均值为 13.8650（231 维折叠标准化空间，中位 13.1884）；它与批内整行重复率（分母 750 行）、边饱和比例（分母 231×750 个单元格）是三个不同口径，不能互相替代。

设计谱的归一化能量距离 0.8727、未归一化 $V = 15.9999$、带符号 $U = 1.6154$（只作并列读数，不解释符号）；多样性 17.4190、最近邻 15.5202、批内整行重复率 0、边饱和比例 0.1237。

关键对照是**源样本重采样**：同样本量、同风格、直接对训练行重采样得到的参照谱（表 9 的“源样本重采样参照”行），其归一化能量距离低于设计谱的 0.8727。也就是说，在本评价口径下，**生成器没有超过“直接重采样真实样本”这一最简参照**。这是本问最重要的负结果，本文据此明确不宣称生成模型优于基线。

设计谱既无批内重复、也未照搬训练行。除批内整行重复率 0.0000 外，另算两项跨集指标：把

750 份设计谱与各自外层折训练集（X_train，折叠填补与标准化）逐行比对，**与训练行的整行精确重合率为 0.0000（0/750）**；每份设计谱到同折最近训练行的**标准化欧氏距离均值为 13.8650**（中位 13.1884），该数值描述设计谱到训练行的距离；是否接近训练分布，仍需同尺度的真实样本距离参照。三个口径不同，不能互相替代。

同一张表还给出更不利的一条：**仅共享相关 Copula、风格内经验 Copula、独立边际三个对照方案的 V 也都略低于交付的主方案**。因此“收缩混合 Copula”这一配置在本数据上并不占优；各方案之间的差距小于它们与重采样参照的差距，本文不据此宣称本生成器结构更优，也不把并列对照压成一句“主方案最好”。

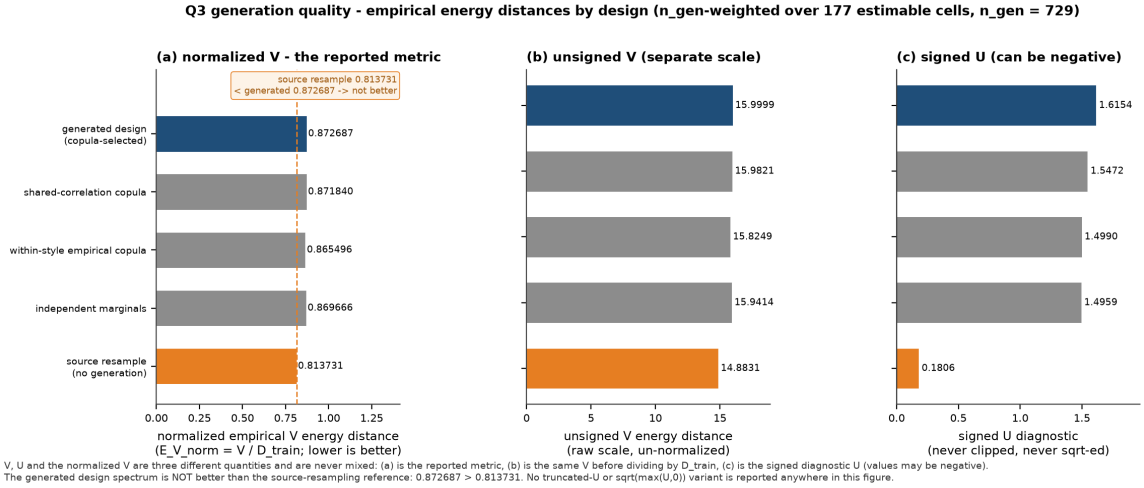


图 7　问题三：归一化经验 V 能量距离（V/D_train）对照；生成设计谱 0.8727 未优于源样本重采样 0.8137，带符号 U 单列不截零。

依赖谱 MAE（0.0056）与协方差迹比（0.8371）显示 Copula 相对独立边际有收益，但三类收缩方案之间的能量距离差异小于它们与重采样参照的差异，故“依赖结构怎么估”不是主导误差来源。边际误差 0.4933 说明单变量分布未被完全复现；感官代理画像的池化秩相关 0.9045、逐格均值 0.4710、平均绝对误差 0.2243，同样只是代理口径。

三种目标风格必须与池化结果分开

设计谱在所有可估风格上按生成样本数加权池化为 0.8727（177 格、729 份），而 Tripel/Kriek/Lambic 单独池化（27 格）明显更高：未归一化 $V = 33.7829$，归一化值与 U 见表 9。两口径不可互换或相减，未作显著性检验；三个小风格样本极少（Kriek 约 10 行、Lambic 约 11 行、Tripel 约 28 行），收缩后差异被平均，故不得用池化值代替三风格画像。

逐风格缺陷如实列出。多数风格归一化能量距离在零点几量级，但 **Kriek 明显离群**（未归一化 V 约 140、归一化约 7.4，为池化值 0.8727 的 8 倍以上），Tripel 反而最好（归一化约 0.28）。这一缺陷本轮未修好，使用 Kriek 生成结果必须按逐风格读数解读。可估风格/格的分母都小于设计格总数（198），被排除的格不填零、不并入；90 条感官 t2n 代理高于其风格内界，也是逐格诊断的一部分。

三风格生成谱与感官画像。表10 与表11 把三个目标风格的生成设计谱与感官代理直接对照同风格真实训练行，回答“生成结果长什么样、与真实样本差在哪”。

表 10　Tripel/Kriek/Lambic 关键化合物生成谱与真实训练分布对照

化合物（单位）	风格	生成中位数 [P10, P90]	实测中位数 [P10, P90]
乙醇（v/v）	Tripel	8.789 [8.070, 9.212]	8.773 [8.044, 9.262]
二氧化硫（mg/L）	Tripel	1.904 [1.167, 3.223]	1.928 [1.110, 3.180]
苦度（α-酸）（IBU）	Tripel	19.94 [13.79, 33.11]	20.04 [13.65, 34.25]
色度（EBC）	Tripel	11.71 [8.324, 22.13]	11.35 [8.090, 21.75]
乙酸乙酯（mg/L）	Tripel	41.31 [27.84, 63.39]	39.43 [27.61, 53.65]
芳樟醇（mg/L）	Tripel	0.6064 [0.1343, 4.128]	0.4851 [0.1252, 7.773]

续表 10　Tripel/Kriek/Lambic 关键化合物生成谱与真实训练分布对照

化合物（单位）	风格	生成中位数 [P10, P90]	实测中位数 [P10, P90]
乙醇（v/v）	Kriek	6.276 [5.024, 7.115]	6.360 [5.203, 7.066]
二氧化硫（mg/L）	Kriek	3.729 [1.063, 9.547]	3.550 [1.090, 5.510]
苦度（α-酸）（IBU）	Kriek	4.769 [1.397, 9.606]	4.598 [1.502, 7.288]
色度（EBC）	Kriek	35.52 [23.22, 42.63]	35.58 [26.57, 43.23]
乙酸乙酯（mg/L）	Kriek	31.48 [15.02, 40.83]	28.35 [15.62, 41.96]
芳樟醇（mg/L）	Kriek	0.3567 [0.1200, 0.4811]	0.3628 [0.1201, 0.4770]
乙醇（v/v）	Lambic	6.862 [4.383, 7.307]	6.310 [4.370, 7.350]
二氧化硫（mg/L）	Lambic	1.708 [0.8724, 5.406]	1.700 [0.9000, 4.300]
苦度（α-酸）（IBU）	Lambic	3.545 [0.6941, 8.873]	3.765 [1.951, 8.838]
色度（EBC）	Lambic	13.34 [8.683, 22.93]	13.40 [12.35, 19.40]
乙酸乙酯（mg/L）	Lambic	35.28 [19.84, 48.64]	37.10 [21.81, 49.04]
芳樟醇（mg/L）	Lambic	0.3677 [0.02907, 0.6510]	0.3480 [0.1283, 0.6458]

注：生成侧为条件生成实验的设计谱按风格池化（Tripel/Kriek/Lambic 分别 84/30/33 份）的中位数与 P10–P90 区间；实测侧为 `data/clean/q3.csv` 同风格真实训练行（28/10/11 行）。已校准列按物理单位给出（对数校准 $c = 10^x$，线性校准 $c = x$），未校准列标注“存储尺度”。生成谱是条件生成与约束投影后的设计值，不是测量值，也不能读作对某支真实啤酒的预测；三个冻结重复与九个训练上下文不是新增独立啤酒，表中区间不得读作窄置信区间。

表 **11**　Tripel/Kriek/Lambic 核心感官代理生成值与真实分布对照

感官代理	风格	生成均值 [P10, P90]	实测均值 [P10, P90]
麦芽香总体	Tripel	0.04373 [-0.3986, 0.4166]	-0.05151 [-0.6140, 0.6797]
酒花总体	Tripel	0.1017 [-0.1099, 0.3142]	0.07918 [-0.5348, 0.6270]
酯香总体	Tripel	0.1654 [-0.04453, 0.4294]	0.1091 [-0.5552, 0.8075]
酸度	Tripel	-0.2985 [-0.5665, 0.03785]	-0.3367 [-0.4787, -0.1600]
总体	Tripel	0.1142 [0.02068, 0.2242]	0.1139 [-0.4891, 0.8767]
麦芽香总体	Kriek	-0.3684 [-0.7815, -0.04923]	-0.5012 [-0.6906, -0.1906]
酒花总体	Kriek	-0.5575 [-0.8412, -0.2624]	-0.8284 [-1.336, -0.2912]
酯香总体	Kriek	0.4602 [-0.01849, 0.9354]	0.7624 [0.05199, 1.472]
酸度	Kriek	1.762 [0.9287, 2.517]	1.856 [0.2019, 2.880]
总体	Kriek	-0.1352 [-0.2485, -0.01688]	-0.1784 [-0.6242, 0.2228]
麦芽香总体	Lambic	-0.1997 [-0.5127, 0.2740]	-0.2549 [-0.7628, 0.5093]
酒花总体	Lambic	-0.2915 [-0.5605, -0.08942]	-0.2620 [-0.9282, 0.1271]
酯香总体	Lambic	0.07176 [-0.1944, 0.3821]	-0.1079 [-0.3944, 0.01731]
酸度	Lambic	1.704 [1.213, 2.307]	1.791 [0.3946, 2.847]
总体	Lambic	-0.04597 [-0.1897, 0.07886]	0.1492 [-0.5009, 0.4747]

注：数值为 50 维岭回归标签代理对设计谱的预测均值与 P10–P90 区间（标准化 Z 分数，无物理单位），不是消费者感官测量；实测侧为同风格真实训练标签。两栏样本量不同，区间宽度不可直接比较；三个冻结重复与九个训练上下文不是新增独立啤酒，不得据此制造窄置信区间。

单一极端坐标的敏感性诊断（不改变主指标）。上述 Kriek 离群可追溯到单个样本的单个坐标。`data/clean/q3.csv` 中 Kriek 的 `beer156` 在 `chem__X2.methyltetrahydrofuran.3.one` 上取值为 12596.4329，而该列全表第二大值仅 76.8328（相差约 164 倍）。该列在清洗表中按**未校准的存储尺度**保存（其单位未在附件校准表中声明）；本文先核对该值与原始附件一致，并**保留这一真实观测**，不以“极端”为由删点或猜测小数点。当该酒被留出时，它在 3 个外折上的标准化坐标约为 1900，使这 3 个 Kriek 坏折的归一化 V 达 $22.61 \sim 23.93$，而其余 6 折仅 $0.58 \sim 0.77$。固定已生成的样本不变，仅在距离评价时从生成侧、参照侧与归一化分母中同步移除该单一坐标，得到诊断值：全风格归一化 V 由 0.8727 降至 0.5975，Kriek 由 7.4079 降至 0.7186，Tripel/Kriek/Lambic 子集由 1.8079 降至 0.4427。这说明 Kriek 的离群主要来自一个量纲异常的输入坐标，而不是生成机制本身；本文保留原主指标不变，把该诊断并列报告，供读者判断该结论的稳健程度。

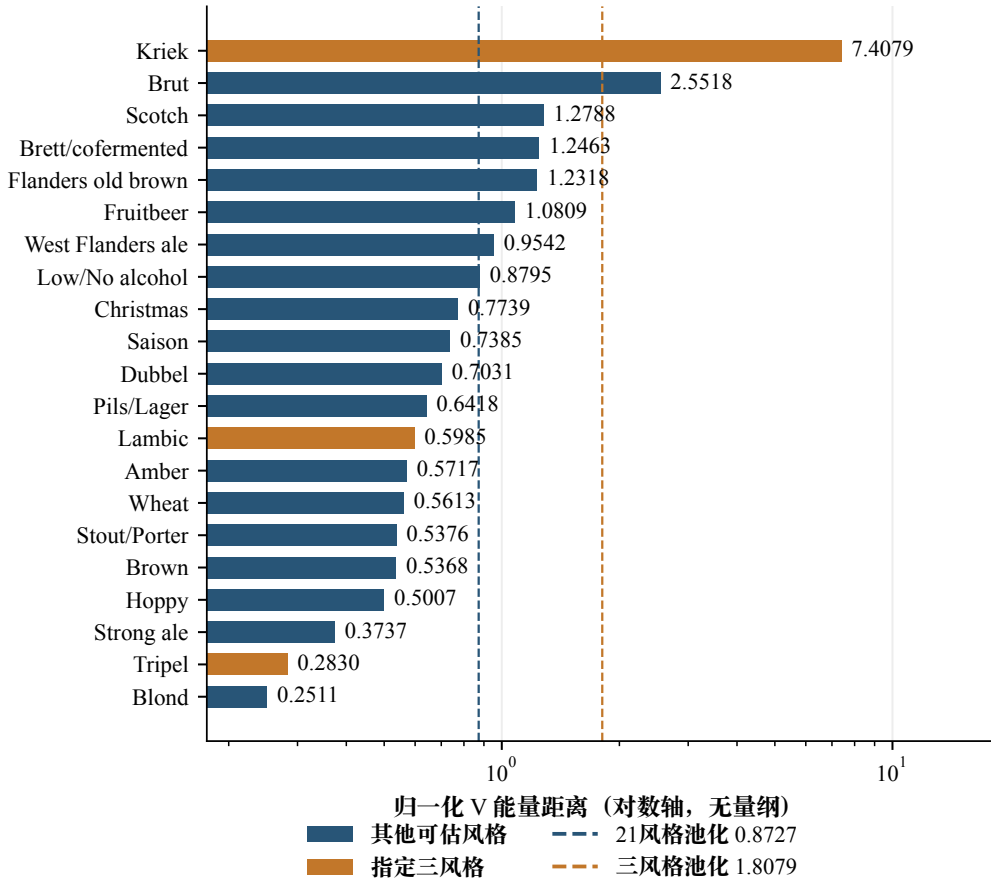


图 **8**　约束设计谱的逐风格归一化 V 能量距离，按生成样本数加权；数值越小越好。橙色为 Tripel、Kriek、Lambic，21 风格池化与三风格池化分别为 0.8727 与 1.8079。Faro 因无可估单元未画入。

约束核验（逐条明细见表 20）共 19670 通过、8211 条件成立、619 不可辨识、0 不可行，可核验比例 0.6902。分层各有来源：

- **不可识别（619）**：4VG 上界 494 条（源值缺测或地板编码）与线性非负 125 条；不计通过、不计失败。
- **条件成立（8211）**：双乙酰 750、反-2-壬烯醛 750、老化酒花代理 750、线性非负 5961；成立依赖训练折派生界、填补假设或代理变量。
- **已验证（19670）**：DMS 上界 750 与线性非负 18664 等，可在写出的样本文件上直接复核。

把不可识别与条件成立两类并入“通过”，会把“没测到”和“依赖假设”说成“已验证”；本文不这样做。

迁移：变量空间、目标与约束

迁移只在声明的凸二次规划下成立，因此必须把“模型”写全，并把它实际用到的数据范围写清——这里有一处容易含糊的地方，本文分开写：

数据范围分开写：每个外层折的迁移上下文由该折训练行中排除被迁移源风格后的行组成，目标均值、目标主成分/相关矩阵与感官代理都在其上估计；但分位数锚点的源侧边际由该折训练行中的源风格样本统一拟合，再经目标边际逆变换，因此“所有拟合都排除了源风格”不成立。需要如实说明自参与范围：99 次主策略查询中有 66 次的源样本本身落在该折训练集内、因而参与了自己的源分位数 CDF 估计（无监督的源 CDF 自参与，不涉及目标标签或感官代理，不构成标签泄漏），其余 33 次源样本处于留出侧。源侧拟合结果不进入目标参照与感官代理；被迁移源样本的原始值只进入盒约束的源值上限与 z_{source}。

变量取折叠标准化的存储空间 $z \in \mathbb{R}^{231}$，$D = \mathrm{diag}(s_d)$ 为目标风格主成分得分的标准差。声明的二次规划为

$$\min_{z,t} (z - z_{\text{ref}})^{\top} Q_t (z - z_{\text{ref}}) + \lambda \|z - z_{\text{anchor}}\|_2^2 + \eta \sum_k t_k \quad \text{s.t.} \quad -t \le z - z_{\text{source}} \le t,\ t \ge 0,\ l \le z \le u,\ Az \le b,$$

其中秩 10 的目标二次型写成与实现一致的形式

$$Q_t = P^\top D^{-1} C^{-1} D^{-1} P, \qquad C = 0.9\widehat{R}_T + 0.1I,$$

P（10×231）为主成分载荷，$\widehat{R}_T$ 为得分相关矩阵，C^{-1} 为其收缩逆。**尺度只除一次**：得分 $s = D^{-1}P(z - \bar{z}_T)$，代回 $s^\top C^{-1} s$ 即为上式（早前两侧各平方一次、写成 D^{-2} 的写法与实现不符，已更正）；中心化用拟合时的 $\bar{z}_T$，故“目标均值到自身”距离为零。z_{anchor} 为分位数保序锚点；$\eta > 0$ 时最优解满足 $t = |z - z_{\text{source}}|$，$t$ 在最优处消去。l, u 承载声明支撑与四条逐坐标化学上限（DMS、双乙酰、4VG、反式-2-壬烯醛均不高于源样本），$Az \le b$ 是唯一一条 `stale_hops` 线性代理行——**代理，不是实测化学约束**。

代理半空间的右端按截距修正。代理为 $f(z) = a^\top z + c$，要求 $f(z_{\text{new}}) \le f(z_{\text{source}}) + \epsilon$，整理后应为

$$a^\top z_{\text{new}} \le b, \qquad b = f(z_{\text{source}}) - c + \epsilon = a^\top z_{\text{source}} + \epsilon,$$

$\epsilon = 10^{-9}$。构造、预检、均值投影、QP、问题哈希、残差与证书都用同一个 b；九折截距 c 全为负（-0.1086 至 -0.0137），漏掉 $-c$ 是把可行域收紧而非放宽。恒等式误差最大 8.27×10^{-17}，半空间残差最大 0。

求解用 ADMM 原-对偶：盒约束与单条半空间的投影用二分法，秩 10 二次型用 Woodbury 求逆；每个求解写入证书（问题哈希、状态、目标值、原/对偶残差、迭代数、可行性、KKT 平稳性与半空间字段 a, c, b、残差、增量），目标梯度另与中心差分核对，唯一允许偏差是 $|z - z_{\text{source}}| = 0$ 处的 L_1 折点。若某次未达容差，交付点回退到源样本并记为“迁移未完成”、**不计入成功**；本轮计划内 198 次全部最优、无回退（fallback 份额 0）。注意该次数是**重复求解、不是等量独立啤酒**：设计为 11 源 × 9 训练上下文（3 重复 × 3 外折）× 2 策略。

为把迁移算子落到可核对的单样本上，表12 给出一个代表源的迁移前后对照；表15 汇总五类受控缺陷的尺度、阈值来源、源状态与逐类核验构成。

表 12　Pils/Lager→Tripel 单源迁移示例（beer 28，repeat 0 / fold 0）

化合物（单位；源状态）	源值	分位数锚点	目标参考值	迁移后设计值	Δ（迁移后 − 源值）
乙醇（v/v）	6.790	9.7848	8.875	9.588	2.798
pH（pH）	4.190	4.1125	4.067	4.111	-0.07930
苦度（α-酸）（IBU）	24.59	29.0006	20.11	28.94	4.345
色度（EBC）	5.400	8.3769	12.99	7.828	2.428
甘油（g/L）	2.120	3.4937	2.998	3.498	1.378
乙酸乙酯（mg/L）	15.61	57.0199	39.95	54.82	39.21
异戊醇（mg/L）	26.67	49.2950	50.81	49.45	22.78
芳樟醇（mg/L）	0.1440	18.4615	4.284	18.31	18.16
二甲基硫（DMS）（ug/L；实测）	27.36	113.9679	44.62	27.36	0.000
双乙酰（mg/L；缺失 · 填补）	0.2473	0.2911	0.3764	1.310e-07	-0.2473
4VG（mg/L；地板码）	0.000	0.0097	0.01519	0.000	0.000
反-2-壬烯醛（mg/L；实测）	0.01153	0.0356	0.6262	0.01153	0.000
老化酒花代理（代理分数；代理）	0.03872	0.0601	—	0.03872	1.000e-09
感官代理 · t2n（代理分数）	-0.04814	—	—	-0.06224	-0.01410

注：代表源按固定规则选取——repeat=0（冻结种子 20261025）、外层 fold=0、在 11 支 Pils/Lager 源啤酒中取最小 id，即 beer 28（transfer_id R0-F0-B028-qp-quantile-anchor，status=optimal、feasible=True、约束违反 0）。源值为 `transfers.csv` 的 `src__chem__`（与 `data/clean/q3.csv` 对应训练行逐格一致），迁移后为 `opt__chem__`；分位数锚点为该折实际使用的分位数保序锚点；“目标参考值”是目标风格参考均值，**不是锚点**，单列以便区分。注意：原始分位数锚点本身并不是声明可行域内的点（99/99 个锚点都超出四条化学 cap 的盒界，需要在 QP 中被投影），表中给出的是锚点值，不是可行解；五项缺陷的“不增”是 QP 构造性质（上界取源值本身），不是感官合格证明；四条化学缺陷源状态为实测/缺失填补/地板码时已在标签中标明。老化酒花与 t2n 只有训练折内的岭回归代理，不能读作实测；核心 5 维感官代理在迁移产物中没有前后对照，故不列入。

求解证书与独立复算

表 13 二次规划求解证书摘要

指标	数值	口径与说明
QP 求解次数	198	两种锚定策略分别 99 / 99 次
求解设计：源 × 训练上下文 × 策略	11 × 9 × 2	同一源在 9 个训练上下文里被重复求解：这是重复求解，不是独立样本数
状态 optimal	198	solver 返回最优
状态 infeasible	0	声明可行域内未出现不可行问题
状态未收敛	0	未出现未收敛
唯一问题哈希数	198	逐问题哈希互不相同（不同问题实例），但同一源在 9 个训练上下文中被重复求解
最大原始残差	9.995e-09	ADMM 一致性分裂的 max |z − w|
最大对偶残差	2.105e-10	缩放对偶残差 rho · max|w_k − w_{k−1}|
最大约束违反	0.0000	box 界与半空间约束的违反量
最大 KKT 平稳性残差	4.069e-09	平滑梯度 + L1 次梯度 + 紧约束乘子的平稳性
stale 半空间右端修正：截距 c 范围	[-0.1086, -0.0137]	每折感官岭回归代理的截距；198 个证书记录的截距均为负
stale 半空间最大残差	0.0000	用修正后的右端复核 a'z − b
stale 半空间恒等式误差最大值	8.274e-17	预测值与代理预测通道的恒等式误差
源状态分层：verified / conditional / not_identifiable / failed	3465 / 198 / 99 / 0	共 3762 条约束核验记录（3267 条 33 列线性非负下界 + 396 条四项化学源值上限 + 99 条 stale 代理上界），可核验分母 3663；不可识别记录不计入分母，也不计为失败
目标梯度有限差分最大绝对差	2.358e-09	198 个检查点的最大值
目标梯度有限差分最大相对差	4.447e-05	同一批检查点
策略分布	99 / 99	QP · 源锚定 / QP · 分位数锚定
最大迭代次数	49	ADMM 迭代上限内收敛
内部不一致记录	0	证书字段之间的自洽检查

注：本表是 198 个逐问题凸二次规划证书的汇总，不是原生优化验证，也不保证全局最优之外的任何性质。198 次求解是 11 个源 × 9 个训练上下文 × 2 个策略的重复求解，不能当作独立样本数。残差是求解器收敛读数，不是迁移准确率；约束违反为 0 说明解在声明的可行域内，可行域本身（box 界、一条 stale 半空间、秩 10 二次目标与两处正则项）由模型声明固定。stale 半空间右端已修正为 b = a'z_source + 1e-9，截距 c 全为负、半空间残差与恒等式误差在浮点误差内；这只说明约束实现一致，不说明 stale 代理是实测老化指标——它是训练折内的岭回归代理。目标梯度有限差分只核对实现中的梯度与目标函数一致。

计划内 198 次求解全部最优（198；infeasible / 未收敛 0 / 0），最大原残差 9.99×10^{-9}、最大对偶残差 2.11×10^{-10}、最大约束违反 0，均低于声明容差 10^{-8}；目标梯度与中心差分最大绝对偏差 2.36×10^{-9}。其中，容差边界需与 "198 个 optimal" 并列说明：

- **落在容差边界**：198 个解中 **131 个恰落在 1e-9 容差边界**（qp-quantile 67 + qp-source 64；495 行全策略合计 189/495），代理约束在这些点活跃，"不增" 只在声明容差内成立，不是严格不等。

不增率的饱和性、完整可行域基线与代理越界

表 14　五种迁移策略的阶段指标

策略	转移能量 V	归一化 V	四条化学列不增率	老化酒花代理不增率	综合不增率（含代理）	fallback 份额	目标均值自距
QP · 分位数锚定（交付策略）	2.0799	0.1134	1.0000	1.0000	1.0000	0.0000	—
QP · 源锚定	7.0906	0.3900	1.0000	1.0000	1.0000	0.0000	—
源固定（平凡对照）	7.1963	0.3958	1.0000	1.0000	1.0000	0.0000	—
目标均值投影（盒约束诊断对照）	6.9415	0.3794	1.0000	0.4141	0.4141	0.0000	4.749e-16
目标均值投影 · 完整可行域（参照基线）	6.3792	0.3485	1.0000	1.0000	1.0000	0.0000	—

注：转移能量按同一批目标参考行计算，越小表示迁移后的谱越接近目标风格。四条化学列不增率只统计二甲基硫、双乙酰、4-乙烯基愈创木酚、反-2-壬烯醛四条 cap；综合不增率另要求老化酒花代理不增。盒约束均值投影（不含半空间）的综合不增率为 0.4141，其偏低完全由代理越界驱动，四条化学 cap 仍全部满足，因此该值不等于“四条 cap 有相应比例不满足”。完整可行域均值投影（box ∩ 同一条 stale 半空间）的综合不增率为 1.0000，它是本轮新增的参照基线：与交付 QP 策略有相同的可行域和非增性质，但转移能量更高，因此“非增率 = 1”本身不能区分策略优劣。源固定策略是不做任何迁移的平凡对照，其四条化学列与综合不增率同样为 1，因此不能把 1.000 读成迁移效果；老化酒花代理是训练折内的岭回归代理，不是实测老化指标。该复合指标在计划内两个执行块与补充种子块中均为 1.000（有界饱和），min-value 规则下“min 1 ≥ 1”不构成改善或强稳健证据。盒约束均值投影是诊断对照，其代理越界记录只属该参照控制，不参与交付判定。fallback 份额为 0 表示没有迁移结果被退回源谱。目标均值自距是“目标均值到自身”的距离，只对盒约束均值投影策略有意义；早期中心化口径下的同名值偏大，已被修复，不再引用。

先分清两个量（早期版本称 “逐列不增率”，本文统一称 “综合不增率”）。**四条化学列**（DMS、双乙酰、4VG、反-2-壬烯醛）的不增率为 1（逐列 1 / 1 / 1 / 1），五种策略下均未被违反。**综合不增率**另要求 `stale_hops` 代理不增：交付 1、源锚定 1、源固定 1；盒约束诊断对照 0.4141 的低值完全由代理越界驱动。

同约束对照与同锚点消融分开解释。均值投影与交付 QP 具有相同可行域，其转移能量分别为 6.3792 与 2.0799；该比较同时包含分位数锚点和后续优化的作用。进一步固定同一分位数锚点，直接投影到完整可行域时，转移能量为 2.0903；交付 QP 为 2.0799，增量约 0.50%（表 17）。因此，接近目标分布的主要收益来自分位数映射，QP 在声明的改动代价与约束下作进一步细化，不把总体变化全部归因于 QP。

迁移侧的两个缺陷命题分开读。真实化学列反-2-壬烯醛不增率为 1（化学命题成立）；**感官 t2n 代理**在同一批迁移行上仍有 0.0606 的比例增加，说明代理口径下 “老化风味不增” 未成立，本文未把感官 t2n 设成硬约束。

源状态分层（四类判定）。交付策略的 3762 条约束核验记录（3267 条线性非负下界 + 396 条四项化学源值上限 + 99 条 stale 代理上界）中，可核验 3465、条件 198、不可识别 99、失败 0（可核验分母 3663）；逐类构成见表 20。代理越界只出现在盒约束诊断对照（99 行中 58 行），与 198 个 QP 的 `max_constraint_violation=0` 不是同一对象。

表 15　五类受控缺陷：尺度/单位、阈值来源、源状态与核验记录构成

缺陷项	尺度/单位	阈值 Mean / Median	源状态实测/缺失/地板	约束记录数
二甲基硫（DMS）	μg/L	17.25 / 19	99 / 0 / 0	99
双乙酰（2,3-丁二酮）	mg/L	0.2993 / 0.09	81 / 18 / 0	99
4-乙烯基愈创木酚（4VG）	mg/L	0.2063 / 0.2	18 / 0 / 81	99
反-2-壬烯醛（trans-2-nonenal）	mg/L	—	99 / 0 / 0	99
老化酒花代理（岭回归）	ridge-proxy score（标准化感官标签单位）	—	0 / 0 / 0	99

缺陷项（约束类型）	约束类型	已验证	条件成立	不可识别	失败	可核验分母
二甲基硫（DMS）	源值上限	99	0	0	0	99
双乙酰（2,3-丁二酮）	源值上限	81	0	18	0	81
4-乙烯基愈创木酚（4VG）	源值上限	18	0	81	0	18
反-2-壬烯醛（trans-2-nonenal）	源值上限	0	99	0	0	99
老化酒花代理（岭回归）	源值上限 _proxy（半空间 a’z<=b）	0	99	0	0	99
四类合计	—	3465	198	99	0	3663

注：阈值取自 `data/clean/q3_thresholds.json`（Mean / Median），单位按该文件的声明；“实际约束”一律是源值上限（`transfer_constraints.csv` 的 bound_type=upper_source_value，老化酒花代理为半空间 $a^\top z \le b$），因此 QP 只保证“不劣化源样本”，不保证低于感官阈值。源状态三列分别为实测 / 缺失（训练折中位数填补）/ 地板码（非检测），每类分母 99 = 3 重复 × 3 折 × 11 支源啤酒。注：五类上界记录合计 495 条（每类 99 条）；表 13 按全部约束统计的 3762 条（= 3267 条 33 列线性非负下界 + 396 条四项化学源值上限 + 99 条 stale 代理上界）复核结果见正文的求解证书汇总表。“不可识别”全部来自源格缺失或地板码，“条件成立”来自代理或存储尺度约定；条件成立 198 条与已验证 3465 条合计 3663 条具有可判定记录；不可识别 99 条不并入该合计。条件成立仍不等于实测已验证，两类均不因该状态而计为失败。

位移、约束残差与 z 空间位移是三组不同的量

表 16　关键列的三组量：约束残差、存储空间位移与 z 空间位移

对象	①约束残差 max	②存储空间位移 max	②存储空间位移 mean	③ z 空间位移 max	③ z 空间位移 mean
反-2-壬烯醛	3.556e-17	0.0010	2.079e-05	—	—
双乙酰	2.22e-16	0.5239	0.0580	—	—
4-乙烯基愈创木酚	0.0000	0.3010	0.0109	—	—
z 空间 · 逐行 231 维绝对位移均值	—	—	—	0.6367	0.4878
z 空间 · 逐行 231 维 L1 位移	—	—	—	147.0727	112.6728

注：三组量不是同一对象，不能相互替代或相减。①约束残差是上限约束下 clamp(新值 − 源值, 0) 的逐行最大值，量级在浮点误差附近，只说明约束未被违反；②存储空间位移是 | 新值 − 源值 | 的逐列最大/均值，描述配方改动了多少；③ z 空间位移是 231 维中心化标准化后的逐行位移，逐列分解未随产物保存，故只给出行级读数。z 空间的独立 centre/scale 重建产物属于上一轮运行（冻结代码/数据 sha 与当前运行不同（[’src/q3_lib.py’, ’src/q3_transfer_fix.py’]）；复算覆盖 [’qp-quantile-anchor’, ’qp-source-anchor’, ’source-fixed’, ’target-mean-projection’] / 396 行，本轮为 [’qp-quantile-anchor’, ’qp-source-anchor’, ’source-fixed’, ’target-mean-projection’, ’target-mean-projection-full’] / 495 行），本轮不引用该过期重建；本轮只做行内恒等式核对 l1_change_z / 231 = mean_abs_change_z（最大差 1.110e-16）。三组都只针对交付的 QP 分位数锚定策略。

三组量分别回答三个问题。第一组**约束残差** $\mathrm{clamp}(x^{\mathrm{new}} - x^{\mathrm{source}}, 0)$ 的量级就是数值零，回答“是否越界”，不回答“改了多少”。第二组**存储空间位移**是 `qp-quantile-anchor` 的逐列最值/均值：三列量级差异很大（双乙酰最大、4VG 次之、反-2-壬烯醛接近噪声），不能用单一“平均改动”概括；这三列**不代表** 231 列的整体位移（整体位移由痕量化合物主导，逐列分布见图 9 的 (a)、(b) 面板），后者也不能反过来判断缺陷约束——那只看残差。

第三组 **z 空间位移**只用于描述改动量级：上一轮的独立 centre/scale 重建对应不同代码/数据哈希，属过期产物，本文不引用；本轮只做行内恒等式核对（$L_1/231$ 与平均绝对位移一致，最大差 1.1×10^{-16}），不作为可复现性证据。

表 17　分位数锚点直接投影消融与两条增强基线对照

策略（同一 99 个源 × 训练上下文问题）	转移能量 V	归一化 V	约束违反点	平均 L1 改动（z 空间）	距离细化（相对锚点直接投影）
源谱不变（不做迁移）	7.1963	0.3958	—	—	—
盒约束均值投影（诊断对照）	6.9415	0.3794	—	—	—
完整可行域均值投影（同一 box∩ 半空间）	6.3792	0.3485	—	—	—
分位数锚点直接投影（消融对照）	2.0903	0.1140	0	113.3848	基准
分位数锚定 QP（交付策略）	2.0799	0.1134	—	112.6728	0.50%

注：五行使用同一组 99 个（源啤酒，训练上下文）问题、同一盒约束与同一条修正后 stale 半空间，差别只在“把源谱变成目标风格谱”的算子。源谱不变与两种均值投影不改变源谱或只把目标均值投到可行域上，因此与目标分布的距离很大；分位数锚点直接投影保留主方案的分位数保序锚点、直接做盒-半空间投影（舍去秩 10 二次项与 L_1 项），其 99 个点的约束违反为 0（数值上限 2.2×10^{-16}）。交付 QP 的 V 为 2.0799，比锚点直接投影低 0.4990%（即约 0.50%）；同时平均 L_1 位移 112.6728 小于直接投影的 113.3848（小 0.6280%）。也就是说，向目标风格靠近的绝大部分收益来自分位数保序映射，QP 在声明目标下再提供距离细化与显式的改动幅度控制。

收益来自分位数保序映射，QP 只做距离细化与改动幅度控制。为分离两部分的贡献，本文在同一组 99 个（源，训练上下文）问题上加做一个消融对照：保留主方案的分位数锚点 z_{anchor}，直接调用与 QP 完全相同的盒界与修正后 stale 半空间做投影（舍去秩 10 二次项与 L_1 项），得到“分位数锚点直接投影”策略。该策略的 99 个点约束违反均为 0，对目标训练分布的转移能量 $V = 2.0903$；而源谱不变时为 7.1963、盒约束均值投影 6.9415、完整可行域均值投影 6.3792、分位数锚定 QP 2.0799。可见，$7.1963 \to 2.0903$ 的绝大部分收益来自分位数保序映射本身；QP 在声明的目标函数下再提供约 0.50% 的距离细化，并以 L_1 项给出显式的改动幅度控制。也就是说，“QP 证书”证明的是声明目标下的最优性，而不是收益的全部来源。本对照是定稿阶段的事后消融，不属于预注册实验，也不改变主实验（分位数锚定 QP）的身份。

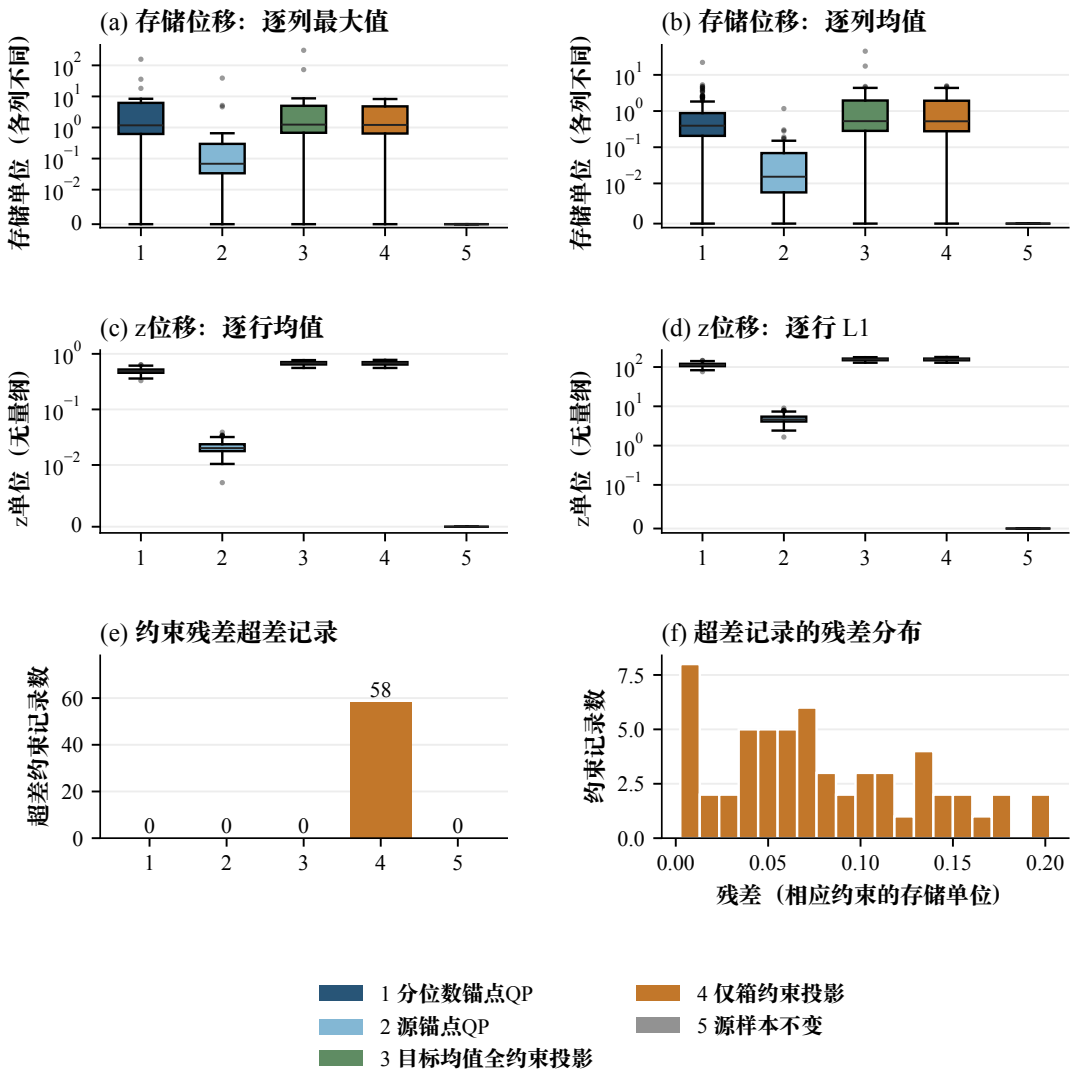


图 **9**　五种迁移策略的存储位移、标准化 z 位移和约束残差，编号见公共图例。每策略 99 条迁移、3762 条约束记录；图 (e) 按残差大于 10^{-8} 计数，图 (f) 只展示这些超限记录。位移与残差分别描述改动大小和约束违反；存储单位随变量而异，z 空间无量纲。

目标参照自距 4.75×10^{-16}（数值零；旧口径 0.6655 是未减目标均值的缺陷，已修正）。迁移前后分布变化：迁移谱 2.0799（归一化 0.1134）、源点固定 7.1963、盒约束诊断 6.9415、完整可行域基线 6.3792。

问题三的实际答案

两个对象分开给答案。

1. **生成侧**：设计谱归一化能量距离 0.8727；约束分层可核验 19670 / 条件 8211 / 不可识别 619 / 不可行 0。边界：未超过同风格重采样参照，三个对照配置 V 也略优，Kriek 逐风格读数离群。
2. **迁移侧**：计划内 198 次求解全部最优；交付策略 $V = 2.0799$（归一化 0.1134）、四条化学列不增率 1；同一完整可行域的均值投影基线 $V = 6.3792$、不增率 1；同锚点直接投影为 $V = 2.0903$，QP 相对其改善约 0.50%。
3. **证据归属**：生成侧与迁移侧各有主结论与稳健性结论，分别对应问题三生成、问题三迁移两支预登记稳健性计划（各含参考块、扰动块与登记后补跑的一块）。计划范围与执行读数见表18及“稳健性与限制”一节。

限制：不是真实工艺可实现性的证据；不增只在 10^{-9} 声明容差内成立（131/198 个 QP 解落在边界）；化学不增与感官代理不增是两个命题；迁移无迁移后真实标签；原生优化检查未通过。

问题三的作用边界

第一，迁移是“声明模型下的最优”，不是真实迁移：结论只在 231 维 z 空间、声明盒界与单条代理半空间、以及“秩 10 二次目标 + λ 锚点项 + L_1 项”下成立；198 个证书不改变这一点。

第二，没有迁移后的真实标签。只报告目标距离与约束状态，不报告准确率；任何“更接近真实 Tripel 风味”的表述都超出证据。

第三，生成不等于优于参照。设计谱劣于源样本重采样参照，另三个对照配置的 V 也略优；Kriek 的离群主要来自单个极端输入坐标，其敏感性诊断已并列报告。

第四，化学不增与感官代理不增是两个命题。真实反-2-壬烯醛增加 0 条，感官 t2n 代理仍增加 366 条（0.488）；迁移侧同理（化学列不增率 1，感官 t2n 代理增加比例 0.0606）。

第五，“不增只在容差内成立”：131 个（共 198 个）QP 解恰好落在声明的 10^{-9} 边界上（qp-quantile 67、qp-source 64），读成严格不等会高估它。

第六，`source-fixed` 平凡对照与 QP 结果必须分开呈现；能区分策略的是同一完整可行域上的转移能量，而不是饱和的不增率。

稳健性与限制

检验设计

稳健性检验在主方案确定后、比较之前预注册：先登记指标、扰动清单与判定规则，再执行扰动。判定量为相对极差

$$R = \frac{\max m - \min m}{\max(|m_{\text{ref}}|, 10^{-12})},$$

其中 m_{ref} 为参考运行（扰动前对照）指标，四个对象各有独立计划与上限、不合并判定。表中绝对极差与真实相对极差分列：前者为计划内取值的最大值减最小值，后者以参考运行值为分母；判定一律按真实相对极差对照预注册上限。

分列的四组检验

表 18　四组稳健性检验的实际范围

对象	计划执行值	补充块	绝对极差	相对极差	规则与判定
问题一	0.3542 / 0.3624 / 0.3576	—	0.0082	0.0232	相对极差 ≤ 0.0500；通过
问题二	0.2103 / 0.2073	0.2097	0.0030	0.0143	相对极差 ≤ 0.1000；通过
问题三生成	0.8639 / 0.8703	0.8839	0.0064	0.0074	相对极差 ≤ 0.0500；通过
问题三迁移	1.0000 / 1.0000	1.0000	0.0000	0.0000	每个取值须 ≥ 1；通过

注：每行是一次预注册的稳健性检验。“计划执行值”只含计划内已执行的块（问题一 3 块；问题二/三各 2 块，方法记为冻结种子扰动，被扰动对象是冻结种子块），“补充种子块”是计划登记之后才执行的第三个种子块，单独列出、不并入计划记录，因此不能把本表读成 n=3 的多种子结果。绝对极差为计划内取值的最大值减最小值；真实相对极差为绝对极差除以参考值——真实相对极差以参考运行值为分母，本表按真实分母重算并分列。判定规则与上限在比较之前登记；问题三迁移用 min-value 规则，因指标有界饱和，1.0 是构造上界。所有判定只描述本组扰动，不构成模型全面稳定的证据。

逐项指标、扰动与限制：

问题一：问题一候选模型的 A/F30 等权 MAE；多种子 + 参数扰动。只覆盖三个冻结种子中的两个，且参数扰动只改内层正则网格的 λ 倍数。

问题二：问题二 Top-3 平均两两 Jaccard；冻结种子扰动。计划内只有两个执行块（种子块 0 参考 + 种子块 1）；种子块 2 的补充块在登记后执行、未并入计划记录；扰动对象是冻结种子，不是多种子重复；Jaccard 是集合一致率不是准确率。

问题三生成：问题三生成谱的归一化能量距离；冻结种子扰动。计划内只有两个执行块；种子块 2 的补充块在登记后执行、未并入计划记录；constraint-satisfaction 恒为 1 是构造性质，不参与判定。

问题三迁移：问题三迁移的综合不增率（四条化学列 + 老化酒花代理合计口径；旧称逐列不增率）；冻结种子扰动。指标有界且饱和（计划内两值均为 1），只能说明没有变差，不能读成强稳健；198 个 QP 是重复求解；种子块 2 的补充块未并入计划记录。

扰动方法的统一说明。四组检验主要比较**冻结种子块**（协议常量 `SEEDS`/repeat）；问题一另含一次内层 λ 网格扰动，其余计划设置保持不变：问题一执行 3 块（参考、种子块 20261025→20261026、内层 λ 网格整体乘 0.5），问题二与问题三各执行 2 块（参考 + 种子块 1），因而都只覆盖 3 个冻结

种子中的 2 个；bootstrap 次数在各块都是 500，没有预算扰动。登记之后补跑的第三块单列于下文、不并入计划记录。

问题一。参考运行与两个扰动的 Aroma/Flavour 30 维等权 MAE 分别为 0.3542、0.3624、0.3576；其中一次扰动把内层 λ 网格整体乘以 0.5。绝对极差与相对极差见上表，预注册上限为 0.05。

问题二：计划内两块 Jaccard 0.2103 与 0.2073（上限 0.10），补充种子块 0.2097 只作补充证据。**问题三·生成：**0.8639 与 0.8703，补充块 0.8839（上限 0.05），约束满足率的构造量不参与判定。**问题三·迁移：**1 与 1，补充块 1，规则为每次保持 1.0。该指标有界饱和，包括补充块在内的极差 0 不构成改善或强稳健证据——饱和极差 0 只是没有信息。

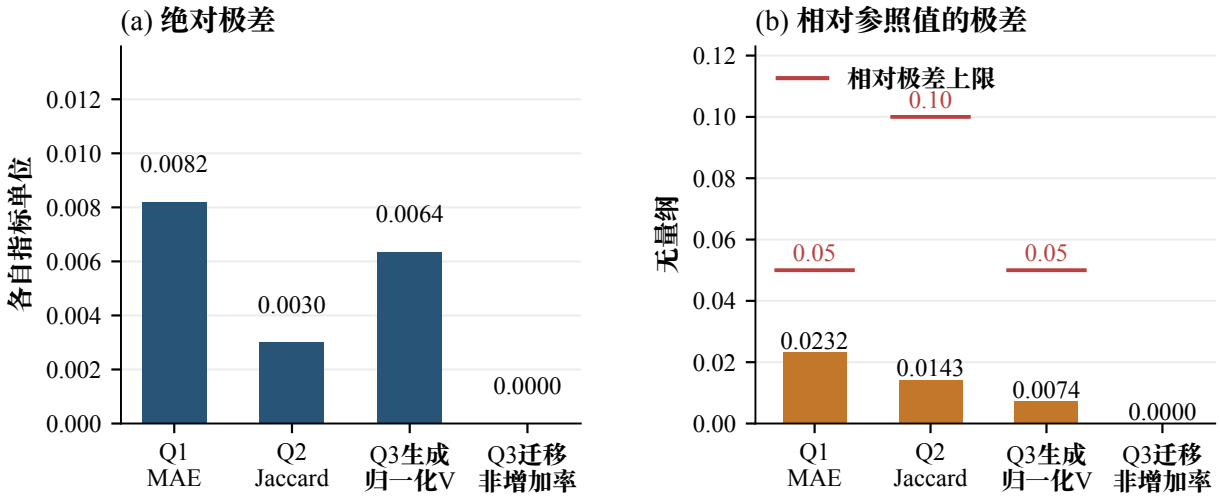


图 **10**　四组预定扰动的绝对极差与相对极差分别展示。Q1 计划含 3 组，Q2 及 Q3 生成、迁移计划各含 2 组，补充块未并入；红线为各自相对极差上限。迁移不增率两组均为 1，零极差来自指标饱和，不能解释为性能提升。

这些检验没有覆盖什么

第一，问题一的计划含 3 组，问题二及问题三生成、迁移的计划各含 2 组冻结种子块。问题一另含一次内层 λ 网格乘 0.5 的扰动。这些检验只覆盖有限种子块和一次正则参数扰动，没有对参数空间作系统扫描，不能据此推断普遍的参数不敏感性。

第二，固定种子的独立复跑与外层实验逐位一致，这只证明**确定性重现**，不是独立实现，也不增加独立样本量。

第三，四组检验的对象、指标与上限都不同，不能合并成“模型全面稳定”的单一结论；问题三的生成侧与迁移侧尤其不可合并——两者的指标一个衡量分布差异、一个衡量约束状态，连量纲都不同。

组件作用与负结果汇总

组件对照分散在各表中，此处只整合，不重复数值。

预测结构（表 1）。图 Ridge 与普通 Ridge 几乎相同，仅加图正则不改变结果；图低秩相对普通 Ridge 的改善按低秩收缩归因，量级小于与中位数基线的差距。未做“无图同等低秩”消融，故不保留“图结构带来增益”的主张。

关联识别（表 19）。方法间一致性低于跨重复一致性，候选名单主要受方法口径影响；方向一致率虽高（0.9817），150 条严格共识仍有 95 条 $\pi < 0.5$——“方向可复现”与“名次稳定”是两个问题，Top-3 应读作排序假设。

生成对照（表 9）。投影的分布代价很小，但生成设计谱劣于源样本重采样参照，三个对照配置 V 也略低；三类 Copula 差异小于它们与参照的差异，依赖结构估计不是主导误差来源，而边际误差仍可观、Kriek 逐风格读数尤其差。

迁移对照（表 14）。平凡对照 `source-fixed` 不增率同为 1，该指标无法区分“有改善”与“未移动”；盒约束诊断的低值完全由 `stale_hops` 代理越界造成（四条化学 cap 仍全为 1）；能区分策略的是同一完整可行域上的转移能量比较。

结论、建议与局限

主要结论

本文以可核实的尺度字典连接预测、关联与生成，并在每问内先固定协议、后比较候选。问题一：图正则低秩的 30 维等权 MAE 0.3593、中位数基线 0.3982、全 50 维 0.3505（基线 0.3919），全部为折外预测，0–6 为训练分布内相对等级；图与低秩单独作用有限。问题二：方法间平均两两 Jaccard 0.2091、π 均值 0.4205，方向一致率 0.9817（旧口径 0.7226 偏低源于定义），该量是集合一致性而非准确率。问题三：生成侧归一化能量距离 0.8727，高于源样本重采样参照、三个对照配置 V 也略优；三风格单独池化显著高于池化值，Kriek 离群；化学反-2-壬烯醛不增（0 条），感官 t2n 代理仍增加 366 条（0.488）；迁移侧 198 次求解全部最优、四条化学列不增率 1，转移能量低于同一完整可行域均值投影基线（2.0799 对 6.3792）；同锚点直接投影为 2.0903，QP 相对其增量约 0.50%。接近目标分布的主要收益来自分位数映射，“不增率 =1” 本身不构成改善证据。

实践建议

配方筛选先用 Top-3 与共表达网络定位候选，再用小规模酿造或加标实验检验机制——候选排序不构成因果证据，不能直接指导添加量。风格设计可参考真实目标风格的酯类、高级醇与酸类范围，同时保留 DMS、双乙酰、4VG 与反式-2-壬烯醛的逐样本阈值核验；Kriek、Lambic 等小样本风格的生成质量明显弱于整体，应优先补样本与人工品评而非直接采用生成均值。预测交付中 0–6 等级只能作同数据内相对排序；要与人工评分可比，需取得原始评分与标定参数并重做一致性检验。

适用边界

第一，证据层级不齐。问题一没有独立外部测试集；问题二的核心量是集合一致性而非准确率；问题三的迁移只是声明模型下的最优，且没有迁移后的真实标签。

第二，0–6 等级未恢复人工评分。附件评分工作簿为空白模板，级别由训练折经验分布标定，预测向中部收缩，整级一致率与二次加权 κ 因此受限。

第三，关联与迁移都不涉及因果。问题二为同期关联，问题三为声明模型下的优化结果；两者都不支持 “机制已被证实” 的表述。

第四，稳健性覆盖不全。问题一计划含 3 组，问题二及问题三生成、迁移的计划各含 2 组冻结种子块；另有一次问题一内层正则参数扰动，并非参数空间扫描。固定种子复跑仅证确定性重现；迁移侧指标饱和使极差为零，不能解释为普遍稳健。

第五，附件之间同源，不能互为独立验证；校准附件与主输入在 33 个线性校准列上有 80 个单元格冲突（79 个实质性），成因未知、未修补，这意味着这些列的物理量换算存在不可消除的输入不确定性。

参考文献

[1] 2026 年 MathorCup 数学应用挑战赛——大数据竞赛. 赛道 A：啤酒酿造化合物分析与啤酒风味预测：初赛题面及附件 1–8. 2026.

[2] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. *Nature Communications*, 2024, 15: 2368. doi:10.1038/s41467-024-46346-0.

[3] Gibson B, Vidgren V, Peddinti G, Krogerus K. Diacetyl control during brewery fermentation via adaptive laboratory engineering of the lager yeast *Saccharomyces pastorianus*. *Journal of Industrial Microbiology & Biotechnology*, 2018, 45: 1103–1112. doi:10.1007/s10295-018-2087-4.

[4] Baldus M, Klie R, Biermann M, et al. On the Behaviour of Dimethyl Sulfoxide in the Brewing Process and its Role as Dimethyl Sulfide Precursor in Beer. *BrewingScience*, 2018, 71(1/2): 1–11. doi:10.23763/BrSc18-01baldus.

[5] DeepSeek Harness（deepseek-flash 模型）. AI 辅助编程与文档排版工具，2026. 使用范围、关键交互与人工核验见《AI 工具使用详情说明》。

[6] OpenAI. Codex（本次论文修订会话）. AI 辅助复核、文字修正与图表排版工具，2026-10-28。使用范围及本轮检查记录见随附《本次修订 AI 辅助记录》。

AI 工具使用声明

本文使用 AI 辅助工具。原稿所列 DeepSeek Harness 使用记录见《AI 工具使用详情说明》（参考文献 [5]）；本轮新增使用 OpenAI Codex（参考文献 [6]）完成论文复核、表述修正与图表排版，已知范围说明如下。

本轮使用范围。 AI 参与了指标口径、评价范围与结论措辞的复核和纠错建议，修改 LaTeX 文字、表格与交叉引用，基于已冻结的折外预测和实验输出重绘 7 张图，并辅助数值一致性检查、编译及逐页排版检查。AI 辅助范围包括指标和结论表述复核，最终取舍与责任由参赛队承担。

本轮核验范围。未重新训练预测模型、生成器或运行迁移求解。检查了 20 张表的原有结果数值保留、重绘图与冻结输入的一致性，以及附录 5 段代码正文在修订前后的逐字一致性；这些检查不等于对全部支撑材料作独立复现，也不替代队员的人工审阅。

记录与责任。本次修订记录列明实际完成的 AI 辅助工作。原稿涉及的其他工具名称、版本、完整使用过程及人工核验情况，须由参赛队依据实际记录核对并补齐；不把尚未确认的人工逐段审阅写作已完成事实。参赛队须核对赛题解释、评价协议、指标阈值、实验结论与最终提交内容，并对真实性、准确性及原创性负责。

附录 A　支撑材料长表索引与两张概率表

A.1 支撑材料长表索引

以下 4 张长表随支撑材料交付（同一数据另有 `.csv`），正文只引用其汇总结论；输入/输出 SHA-256 与表 → 实验映射见 支撑材料`/tables/INDEX.json`，全部进入交付包 `SHA256SUMS`。

表	文件	内容
q1-targets	支撑材料`/tables/q1-targets.md` / `.csv`	Aroma/Flavour 30 维逐维折外 MAE、中位数基线 MAE、差值与 Skill（按 Skill 升序，含 6 个恶化维度），每维 750 折外行
q1-styles	支撑材料`/tables/q1-styles.md` / `.csv`	22 个风格的折外 MAE、啤酒数与折外行数，小样本风格（Faro 3、Brut 4、Flanders old brown 5、Scotch 5）单列
q2-full-top3	支撑材料`/tables/q2-full-top3.md` / `.csv`	50 维 × 每维 3 项的完整严格共识榜（150 条），含 π、S、方向、类型与同模块替代候选
q2-extended	支撑材料`/tables/q2-extended.md` / `.csv`	15 个非单体扩展指标在 50 维上的 Top-3，与严格榜分表、不合并计数

这四张表的数据行一行未删；正文不内联只为控制版式，不表示证据缺失。

A.2 概率与约束明细表

以下两张表内容与正文结论直接相关，放在附录以保持正文篇幅；编号接正文连续。

表 19　入选频率、方法间集合一致性与同源对照

指标	数值	口径与说明
Top-3 平均两两 Jaccard（入选频率优先集合，主指标）	0.2091	50 维 × 4 方法 × 6 个方法对，三次重复池化；集合为最终交付集合
Top-3 平均两两 Jaccard（点估计集合）	0.1977	同一统计量，集合改为点估计 Top-3；与主指标的差异来自集合规则而不是数据
第一次重复的 Jaccard	0.2103	该次重复自身的池化值
第二次重复的 Jaccard	0.2073	该次重复自身的池化值
第三次重复的 Jaccard	0.2097	该次重复自身的池化值
跨重复集合一致率（Jaccard）	0.8748	交付集合在两次重复之间的重合程度
入选频率 π 均值	0.4205	交付严格榜条目：四种方法 × 三次重复池化（每次重复 600 条）
π < 0.5 的条目数 / 分母	1263 / 1800	占比 70.17%；分母为该口径下的条目总数
方向一致率均值：首轮 150 条严格共识（分母 150）	0.9995	与入选频率同表；首轮重复的严格共识条目按各自点方向计一致率后取均值
方向一致率均值：单轮 600 条四方法（分母 600）	0.9820	每（维度，方法，名次）格算一次，再对 600 格取均值
方向一致率均值：三次重复池化 1800 条（分母 1800）	0.9817	一般引用口径：3 次重复 × 600 格池化后取均值
符号稳定性（已弃用口径 P(ρ>0)，不得再称为 sign stability）	0.7226	旧口径把 503 条点估计为负方向的条目计为“不稳定”；该值偏低源于定义而不是方向不稳，仅作历史对照
与附件 6.csv 的同源秩相关（双侧名称对齐）	0.9210	44 行名称两侧对齐；同源对照，不是外部验证
与附件 6.csv 的同源秩相关（已废弃：列位置口径）	0.4451	该值由按列位置对齐加索引混用产生，属错位伪影，已弃用，不得作为佐证
bootstrap 预算（每次重复）	500	风格内分层重采样次数，各执行块相同
稳健性执行值（计划内 2 块：种子块 0 参考 / 种子块 1）	0.2103 / 0.2073	同一指标在两个已执行冻结种子块下的取值；扰动对象是冻结种子，不是多种子重复，也没有预算扰动

续表 19　入选频率、方法间集合一致性与同源对照

指标	数值	口径与说明
补充种子块（种子块 2，登记后执行，未并入计划记录）	0.2097	该块在计划登记后才执行，工具拒绝覆盖已执行计划；不得并入上面的执行值、极差与判定，也不得写成 n=3 的计划结果
绝对极差	0.0030	计划内两值的最大值 − 最小值（不含补充块）
真实相对极差	0.0143	绝对极差 / 参考值；判定上限 0.1000
稳健性判定	通过	预注册规则 relative-range；只描述本组扰动

注：Jaccard 衡量的是不同方法/重复之间 Top-3 集合的重合程度，不是预测准确率；π 是候选在 bootstrap 复制中的入选频率，低于 0.5 表示名次不稳；三个方向一致率分母不同（150 / 600 / 1800），引用时必须写明分母；旧口径 P(ρ>0) 把负方向条目一律计为不一致，其数值天然偏低，只作历史对照。6.csv 与本研究同源，只作排序一致性对照，不能称为外部独立验证；bootstrap 预算固定为每次重复 500 次，各执行块之间没有预算差异。稳健性执行值只覆盖计划内已链接的冻结种子块（方法记为冻结种子扰动），计划参考块是登记后的重跑，因此其值不必等于主实验的池化值；种子块 2 的补充块在登记后执行、单独列出且未并入计划记录；这些判定都不构成模型全面稳定的证据。

表 20　声明约束的核验构成

核验状态	记录数	占比
已验证（verified）	19670	0.6902
条件成立（conditional）	8211	0.2881
不可识别（not identifiable）	619	0.0217
不可行（infeasible）	0	0.0000
合计	28500	1.0000

约束类	记录数	已验证	条件成立	不可识别	不可行
缺陷硬阈值 · 二甲基硫上界	750	750	0	0	0
缺陷硬阈值 · 4-乙烯基愈创木酚上界	750	256	0	494	0
缺陷硬阈值 · 双乙酰上界	750	0	750	0	0
反-2-壬烯醛化学不增（训练 q90 上界）	750	0	750	0	0
老化酒花代理上界（岭回归代理）	750	0	750	0	0
线性非负（33 列）	24750	18664	5961	125	0
合计	28500	19670	8211	619	0

注：上表为逐条约束记录的四类核验状态，下表按约束类拆分。“已验证”指约束在写出的样本文件中被直接复核；“条件成立”指约束依赖训练折内的数据派生界或代理变量；“不可识别”指该格的上界无法从数据确定（如缺陷列缺测）；不可行记录为零。设计谱的约束满足率为 1.0000（由约束投影保证），未约束原始生成仅 0.1267；核验记录合计 28500 条，其中已验证占比 0.6902。线性非负类覆盖 33 个化学列；老化酒花只有训练折内的代理约束，不能读作实测老化指标。

附录 B　复现要点

B.1 计算与复核顺序

环境为项目绑定的 Python 3.12（NumPy、pandas、SciPy、scikit-learn）。复核顺序：

1. 按 `beer` 对齐附件，核对校准字典与缺失计数（含第 2 章的 80 格冲突清单）。
2. 只在训练折拟合预处理、图与超参数；计算折外预测与相对等级，并把评价侧（750 行折外）与展示侧（250 行重拟合）分开落盘。
3. 按名称对齐交叉相关表，计算四种方法的 Top-3 与 bootstrap 频率，同时输出方向的正 / 负 / 零 / 无效计数。
4. 逐折用未填补的有效观测估计边际，生成原始抽样，再投影得到约束设计谱。
5. 在声明 QP（含修正后的代理右端 b）下求解并写证书。
6. 按四组独立计划执行冻结种子块的扰动，并单列登记后补跑的种子块。

B.2 脚本与产物索引

全部数值取自可复算的实验产物与由其生成的表格，不在正文手抄；正文与图表题注只使用中文编号（图 N、表 N），不出现内部实验编号。每张表的输入路径、输入/输出 SHA-256 与表 → 实验映射记录在支撑材料的 `tables/INDEX.json`；逐问复现脚本见附录 C 与支撑材料 `src/`。输入缺失或指标键不存在时，生成脚本具名失败，不填零、不回退旧实验。

附录 C　核心源程序代码

本附录收录三问核心建模、求解与评价算法源码；完整可复现工程源码（含命令行封装、依赖模块与自检断言）见支撑材料 `src/` 目录。为控制篇幅，附录只略去模块与函数文档字符串、契约垫片、命令行参数解析与自检脚手架，其余函数体与出厂脚本逐行一致。代码块按页宽折行（折行仅为排版，不代表源代码换行）；为便于阅读并避免被误读为未解析占位符，同一行内相邻的连续花括号显示为“{ {”与“} }”。

软件与运行命令。全部代码运行于 Python 3.12（NumPy 2.3.5、pandas 2.2.3、SciPy 1.18.1、scikit-learn 1.9.1、Matplotlib 3.11.2）；论文由 pandoc 3.x 与 XeTeX（TeX Live 2025）编译。主实验与后处理的运行入口：`python src/q1_fix.py`、`python src/q2_fix.py`、`python src/q3_fix.py`、`python src/q3_transfer_fix.py`，以及 `python src/q2_v62_regen.py`、`python src/q3_v62_diagnostics.py`、`python src/q3_v62_answer_tables.py`、`bash FINAL_SUBMISSION_v6.2/build/build_all.sh`。完整脚本与依赖模块见支撑材料 `src/`。

C.1　问题一：训练折内稳健表示、图正则低秩求解与评价（`src/q1_fix.py`）

```
# AI-ASSISTANCE NOTICE: the framework of this module
    was drafted with DeepSeek Harness assistance;
# the core algorithm logic, constraint corrections
    and experimental parameters were reviewed, amended
# and independently verified by the team members, who
    take responsibility for the contents.

_SRC_DIR = Path(__file__).resolve().parent
CHEM_COLS = ["chem__" + name for name in
    TC.FEATURE_COLS]
MISSING_COLS = ["chem__" + name for name in
    TC.MISSING_COLUMNS]
N_CHEM = len(CHEM_COLS)
N_IND = len(MISSING_COLS)
TAU = 0.5                          # frozen
    correlation threshold
Z_CLIP = 8.0                       # frozen in
    q_common.apply_standardizer, kept for comparability
LAMBDAS = (0.1, 1.0, 10.0)
GRAPH_GAMMAS = (0.1, 1.0)          # strictly
    positive: this is what makes "graph ridge" distinct
LOWRANK_GAMMAS = (0.0, 0.1, 1.0)  # gamma >= 0 as
    required
RANKS = (3, 6, 10)
REPS = ("zscore", "rankgauss")
MODELS = ("median", "ridge", "graph-ridge",
    "graph-lowrank")
PRIMARY = "graph-lowrank"          # the Q1 candidate
    this experiment is about
METRIC_SUFFIX = {"median": "median", "ridge":
    "ridge0",
                 "graph-ridge": "graph_ridge",
    "graph-lowrank": "graph_lowrank"}
class Preprocessor:

    def __init__(self, train_frame: pd.DataFrame):
        raw = _as_matrix(train_frame, CHEM_COLS)
        self.n_train = int(raw.shape[0])
        finite = np.isfinite(raw)
        medians = np.zeros(N_CHEM, dtype=float)
        for j in range(N_CHEM):
            column = raw[finite[:, j], j]
            medians[j] = float(np.median(column)) if
    column.size else 0.0
        self.medians = medians
        self.n_missing_cells = int((~finite).sum())
        imputed = np.where(finite, raw, medians[None,
    :])
        self.z_centre = imputed.mean(axis=0)
        spread = imputed.std(axis=0)
        self.z_scale = np.where(np.isfinite(spread) &
    (spread > 1e-12), spread, 1.0)
        self.rg_grids = [np.sort(imputed[:, j]) for j
    in range(N_CHEM)]
        self.rg_eps = 1.0 / (2.0 * max(self.n_train,
    1))

    def imputed(self, frame: pd.DataFrame) ->
    np.ndarray:
        raw = _as_matrix(frame, CHEM_COLS)
        return np.where(np.isfinite(raw), raw,
    self.medians[None, :])

    def indicators(self, frame: pd.DataFrame) ->
    np.ndarray:
        raw = _as_matrix(frame, MISSING_COLS)
        return np.where(np.isfinite(raw), 0.0, 1.0)

    def design(self, frame: pd.DataFrame, rep: str)
    -> np.ndarray:
        imputed = self.imputed(frame)
        if rep == "zscore":
            transformed = np.clip((imputed -
    self.z_centre) / self.z_scale, -Z_CLIP, Z_CLIP)
        elif rep == "rankgauss":
            transformed = np.empty_like(imputed)
            for j in range(N_CHEM):
                grid = self.rg_grids[j]
                column = imputed[:, j]
                below = np.searchsorted(grid, column,
    side="left")
                equal = np.searchsorted(grid, column,
    side="right") - below
                u = (below + 0.5 * equal) /
    float(grid.size)
                u = np.clip(u, self.rg_eps, 1.0 -
    self.rg_eps)
                transformed[:, j] = ndtri(u)
        else:
            raise ValueError("unknown representation
    %r" % (rep,))
        return np.concatenate([transformed,
    self.indicators(frame)], axis=1)
def graph_laplacian(imputed_train: np.ndarray) ->
    tuple:
    ranks = rankdata(imputed_train[:, :N_CHEM],
    axis=0)          # mid-ranks, ties averaged
    ranks = ranks - ranks.mean(axis=0, keepdims=True)
    norm = np.linalg.norm(ranks, axis=0,
    keepdims=True)
    norm = np.where(norm > 1e-12, norm, 1.0)
    ranks = ranks / norm
    rho = ranks.T @ ranks
    rho = 0.5 * (rho + rho.T)
    non_finite = int((~np.isfinite(rho)).sum())
    rho = np.nan_to_num(rho, nan=0.0, posinf=0.0,
    neginf=0.0)
    np.clip(rho, -1.0, 1.0, out=rho)
    weight = np.where(np.abs(rho) >= TAU, rho, 0.0)
    np.fill_diagonal(weight, 0.0)
    degree = np.abs(weight).sum(axis=1)
    lap231 = np.diag(degree) - weight
    lap231 = 0.5 * (lap231 + lap231.T)
    laplacian = np.zeros((N_CHEM + N_IND, N_CHEM +
    N_IND), dtype=float)
```

```
    laplacian[:N_CHEM, :N_CHEM] = lap231
    info = {
        "tau": float(TAU),
        "nodes": int(N_CHEM),
        "edges": int(np.count_nonzero(np.triu(weight,
    1))),
        "isolatedNodes": int(np.count_nonzero(degree
    == 0.0)),
        "minEigenvalue231": float(np.linalg.eigvalsh(
    lap231).min()),
        "nonFiniteRho": non_finite,
        "indicatorBlock": "zero (missing indicators
    are not connected on the chemistry graph)",
    }
    return laplacian, info
def moments(z_train: np.ndarray, y_train: np.ndarray)
    -> dict:
    n = z_train.shape[0]
    z_bar = z_train.mean(axis=0)
    y_bar = y_train.mean(axis=0)
    centred = z_train - z_bar
    return {
        "n": int(n),
        "z_bar": z_bar,
        "y_bar": y_bar,
        "gram": centred.T @ centred / float(n),
        "cross": centred.T @ (y_train - y_bar) /
    float(n),
        "z_centred": centred,
        "y_centred": y_train - y_bar,
    }
class NormalSystem:

    def __init__(self, stats: dict, laplacian:
    np.ndarray, lam: float, gamma: float):
        d = stats["gram"].shape[0]
        self.lam = float(lam)
        self.gamma = float(gamma)
        self.cross = stats["cross"]
        matrix = stats["gram"] + self.lam * np.eye(d)
    + self.gamma * laplacian
        matrix = 0.5 * (matrix + matrix.T)
        self.factor = cho_factor(matrix, lower=True,
    check_finite=False)   # raises if not SPD
        self.solution = cho_solve(self.factor,
    self.cross, check_finite=False)  # A^{-1} C

    def coefficients(self, rank):
        if rank is None:
            return self.solution
        moment = self.cross.T @ self.solution
    # C^T A^{-1} C, q x q PSD
        moment = 0.5 * (moment + moment.T)
        values, vectors = np.linalg.eigh(moment)
        order = np.argsort(values,
    kind="mergesort")[::-1][:int(rank)]
        leading = vectors[:, order]
        return self.solution @ (leading @ leading.T)
    # A^{-1} C Vr Vr^T
def fit_and_predict(z_train: np.ndarray, y_train:
    np.ndarray, z_eval: np.ndarray,
                    laplacian: np.ndarray, lam:
    float, gamma: float, rank):
    stats = moments(z_train, y_train)
    system = NormalSystem(stats, laplacian, lam,
    gamma)
    beta = system.coefficients(rank)
    prediction = stats["y_bar"] + (z_eval -
    stats["z_bar"]) @ beta
    info = {"n_train": int(stats["n"]), "n_columns":
    int(z_train.shape[1]),
            "lambda": float(lam), "gamma":
    float(gamma),
            "rank": None if rank is None else
    int(rank),
            "interceptRule": "b = ybar - zbar @ B",
            "coefficientNorm": float(np.linalg.norm(
    beta))}
    return prediction, beta, info
def _rows(frame: pd.DataFrame, beer_position: dict,
beers) -> pd.DataFrame:
return frame.iloc[[beer_position[int(b)] for b in
beers]]
def inner_select(frame: pd.DataFrame, beer_position:
dict, repeat: int, outer_fold: int,
                 outer_train_beers, target_af_idx:
np.ndarray, target_all_idx: np.ndarray) -> dict:
folds = list(v6.iter_inner(repeat, outer_fold,
outer_train_beers))
cache = []
for inner_fold, inner_train_beers,
inner_valid_beers in folds:
    train_frame = _rows(frame, beer_position,
inner_train_beers)
    valid_frame = _rows(frame, beer_position,
inner_valid_beers)
    prep = Preprocessor(train_frame)
    laplacian, graph_info =
graph_laplacian(prep.imputed(train_frame))
    designs = {rep: (prep.design(train_frame,
rep), prep.design(valid_frame, rep)) for rep in REPS}
    counts = {}
    for style in valid_frame[v6.STYLE_COL].astype(
str).to_numpy():
        counts[style] = counts.get(style, 0) + 1
    cache.append({
        "inner_fold": int(inner_fold),
        "designs": designs,
        "laplacian": laplacian,
        "graph": graph_info,
        "y_train": _as_matrix(train_frame,
v6.TARGETS_50),
        "y_valid": _as_matrix(valid_frame,
v6.TARGETS_50),
        "n_train": int(len(inner_train_beers)),
        "n_valid": int(len(inner_valid_beers)),
        "missingCells": int(prep.n_missing_cells),
        "styleCounts": {k: int(v) for k, v in
sorted(counts.items())},
    })

cells = {}   # (rep, lambda, gamma, rank) -> list
of validation predictions, one per inner fold
for entry in cache:
    stats_by_rep = {rep: moments(
entry["designs"][rep][0], entry["y_train"]) for rep
in REPS}
    for rep in REPS:
        stats = stats_by_rep[rep]
        _, z_valid = entry["designs"][rep]
        eval_centred = z_valid - stats["z_bar"]
        for lam in LAMBDAS:
            for gamma in LOWRANK_GAMMAS:
                system = NormalSystem(stats,
entry["laplacian"], lam, gamma)
                base = stats["y_bar"] +
eval_centred @ system.solution
                cells.setdefault((rep, lam,
gamma, None), []).append(base)
                for rank in RANKS:
                    beta = system.coefficients(
rank)
                    cells.setdefault((rep, lam,
gamma, rank), []).append(
                        stats["y_bar"] +
eval_centred @ beta)

truth = np.concatenate([entry["y_valid"] for
entry in cache], axis=0)
scored = {}
for key, predictions in cells.items():
    pooled = np.concatenate(predictions, axis=0)
    if pooled.shape != truth.shape:
        raise RuntimeError("inner pooled
prediction %s != truth %s" % (pooled.shape,
truth.shape))
    scored[key] = {
        "pooled_valid_mae_af30":
```

```
float(np.abs(truth[:, target_af_idx] - pooled[:,
target_af_idx]).mean()),
        "pooled_valid_mae_all50":
float(np.abs(truth[:, target_all_idx] - pooled[:,
target_all_idx]).mean()),
        "n_pooled_valid_rows":
int(pooled.shape[0]),
    }

def best(keys):
    return min(keys, key=lambda k:
(scored[k]["pooled_valid_mae_af30"],

scored[k]["pooled_valid_mae_all50"],

REPS.index(k[0]), k[1], k[2],
                                    -1 if k[3] is
None else k[3]))

full_rank_keys = [k for k in scored if k[3] is
None]
ridge_keys = [k for k in full_rank_keys if k[2]
== 0.0]
graph_keys = [k for k in full_rank_keys if k[2] !=
0.0]
lowrank_keys = [k for k in scored if k[3] is not
None]
if not ridge_keys or not graph_keys or not
lowrank_keys:
    raise RuntimeError("the inner grid lost a
model family")
selection = {"ridge": best(ridge_keys),
"graph-ridge": best(graph_keys),
             "graph-lowrank": best(lowrank_keys)}

def family_diagnostics(family, keys, chosen):
    value = scored[chosen]["pooled_valid_mae_af30"
]
    others = [scored[key]["pooled_valid_mae_af30"]
for key in keys if key != chosen]
    lam, gamma, rank = chosen[1], chosen[2],
chosen[3]
    gammas = LOWRANK_GAMMAS if family ==
"graph-lowrank" else GRAPH_GAMMAS
    return {"nCells": int(len(keys)),
            "selectedInnerMaeAf30": float(value),
            "runnerUpInnerMaeAf30":
float(min(others)) if others else None,
            "marginToRunnerUp": float(min(others)
- value) if others else None,
            "lambdaAtGridEdge": bool(lam in
(LAMBDAS[0], LAMBDAS[-1])),
            "gammaAtGridEdge": bool(gamma in
(gammas[0], gammas[-1])),
            "rankAtGridEdge": None if rank is
None else bool(rank in (RANKS[0], RANKS[-1]))}

diagnostics = {
    "ridge": family_diagnostics("ridge",
ridge_keys, selection["ridge"]),
    "graph-ridge": family_diagnostics(
"graph-ridge", graph_keys, selection["graph-ridge"]),
    "graph-lowrank": family_diagnostics(
"graph-lowrank", lowrank_keys, selection["graph-lowra
k"]),
}

grid = []
for key in sorted(scored, key=lambda k:
(REPS.index(k[0]), k[1], k[2], -1 if k[3] is None
else k[3])):
    rep, lam, gamma, rank = key
    cell = {"representation": rep, "lambda":
float(lam), "gamma": float(gamma),
            "rank": None if rank is None else
int(rank)}
    cell.update(scored[key])
    grid.append(cell)
geometry = {
    "n_folds": int(len(cache)),
    "n_train_rows": [entry["n_train"] for entry
in cache],
    "n_valid_rows": [entry["n_valid"] for entry
in cache],
    "missingCellsInInnerTrain":
[entry["missingCells"] for entry in cache],
    "styleValidCounts": {str(
entry["inner_fold"]): entry["styleCounts"] for entry
in cache},
    "styleCountsBelowTwo": sorted({style for
entry in cache
                                   for style,
count in entry["styleCounts"].items() if count < 2}),
    "graph": [entry["graph"] for entry in cache],
    "n_pooled_valid_rows": int(sum(
entry["n_valid"] for entry in cache)),
    "n_cells": int(len(scored)),
}
return {"cells": scored, "selection": selection,
"grid": grid, "geometry": geometry,
        "diagnostics": diagnostics}
def _regression(truth, prediction, targets,
styles=None, baseline_mae=None) -> dict:
return v6.regression_block(truth, prediction,
list(targets), styles=styles, baseline_mae=baseline_ma
e)
def _grade_arrays(prediction: np.ndarray, truth:
np.ndarray, reference: np.ndarray) -> tuple:
g_pred = np.empty(prediction.shape, dtype=int)
g_true = np.empty(truth.shape, dtype=int)
for j in range(prediction.shape[1]):
    cut = reference[:, j]
    g_pred[:, j] = v6.to_grade(prediction[:, j],
cut)
    g_true[:, j] = v6.to_grade(truth[:, j], cut)
return g_pred, g_true
def _grade_block(g_pred: np.ndarray, g_true:
np.ndarray) -> dict:
mask = (g_pred >= 0) & (g_true >= 0)
n = int(mask.sum())
if n == 0:
    return {"n_grade_cells": 0}
qwk = v6.quadratic_weighted_kappa(g_true[mask],
g_pred[mask])
return {
    "n_grade_cells": n,
    "qwk": float(qwk) if math.isfinite(qwk) else
None,
    "grade_mae": float(np.abs(g_true[mask] -
g_pred[mask]).mean()),
    "exact_agreement": float((g_true[mask] ==
g_pred[mask]).mean()),
    "within_one_agreement":
float((np.abs(g_true[mask] - g_pred[mask]) <=
1).mean()),
    "grade_counts_predicted":
np.bincount(g_pred[mask].ravel(),
minlength=7).tolist(),
    "grade_counts_reference":
np.bincount(g_true[mask].ravel(),
minlength=7).tolist(),
"note": "relative grades of a prediction, not
an accuracy claim about the panel's original 0-6
score",
}
```

C.2 问题二：候选榜构建、风格分层 bootstrap、稳定性与同模块替代（src/q2_fix.py）

```
# AI-ASSISTANCE NOTICE: the framework of this module
    was drafted with DeepSeek Harness assistance;
# the core algorithm logic, constraint corrections
    and experimental parameters were reviewed, amended
# and independently verified by the team members, who
```

```
    take responsibility for the contents.
SRC = Path(__file__).resolve().parent
PROJECT = SRC.parent
CONTRACT = install_contract_shim(PROJECT)
TRAIN_CSV = PROJECT / "data/splits/SPLIT-0001/train.cs
    v"          # 200 registered train rows (Q2's only
    rows)
CLEAN_CSV = PROJECT / "data/clean/q1_mapped.csv"
CHEM_META = PROJECT / "data/clean/q2_chem_meta.json"
EXTERNAL_CSV = PROJECT / "data/raw/imported/a55004fb03
    90299a-6.csv"
ALIGN_CSV = PROJECT / "MathorCup_A_ 独立审查与重跑方案/sensory
    ame_alignment.csv"
METHODS = ("total_spearman", "within_style_spearman",
    "graph_lowrank", "consensus")
BASE_METHODS = METHODS[:3]
K = 3
TAU = 0.5
GRAPH_SETTINGS = {"rank": 6, "lambda": 1.0, "gamma":
    0.1, "nIter": 25}
N_BOOTSTRAP = 500
N_DIMENSIONS = len(C.TARGETS_50)
N_FEATURES = len(FEATURE_COLS)
STYLE_COL = C.STYLE_COL
PI_STABLE = 0.5
SIGN_STABLE = 0.9
ZERO_TOL = 1e-12
BUCKET_POSITIVE = "positive"
BUCKET_NEGATIVE = "negative"
BUCKET_ZERO = "zero"
BUCKET_INVALID = "invalid"
BUCKET_KEYS = (BUCKET_POSITIVE, BUCKET_NEGATIVE,
    BUCKET_ZERO, BUCKET_INVALID)
DIRECTION_POSITIVE = "positive"
DIRECTION_NEGATIVE = "negative"
DIRECTION_UNDETERMINED = "direction-undetermined"
NO_ALTERNATIVE = "none"
FILE_SETS = "sets.json"
FILE_STRICT = "strict_top3.csv"
FILE_STRICT_METHODS = "strict_top3_by_method.csv"
FILE_EXTENDED = "extended_top3.csv"
FILE_EXTENDED_METHODS = "extended_top3_by_method.csv"
FILE_CANDIDATES = "candidate_scores.csv"
FILE_BOOT = "bootstrap_pi.csv"
FILE_BOOT_EXT = "bootstrap_pi_extended.csv"
FILE_SIGN_COUNTS = "sign_counts.csv"
FILE_LOWPI = "strict_lowpi_sign_status.csv"
FILE_SIGN_SUMMARY = "sign_stability_summary.json"
FILE_METRICS = "metrics.json"
FILE_OUTPUTS = "outputs.json"
TOP3_HEADER_METHOD = TOP3_HEADER + ["method"]
BOOT_HEADER = ["bootstrap_id", "seed", "dimension",
    "method", "rank", "compound"]
CANDIDATE_HEADER = ["dimension", "compound", "kind",
    "rho_total", "rho_within", "beta_graphlowrank",
                    "score", "pi"]
SIGN_COUNTS_HEADER = ["dimension", "compound",
    "kind", "rho_point_signed", "point_direction",
                      "n_positive", "n_negative",
    "n_zero", "n_invalid", "valid", "p_positive",
                      "p_negative",
    "p_positive_legacy", "sign_stability",
    "direction_status"]
def sign_buckets(replicate_signed, zero_tol: float =
    ZERO_TOL) -> dict:
    values = np.asarray(replicate_signed, dtype=float)
    finite = np.isfinite(values)
    return {
        BUCKET_POSITIVE: finite & (values > zero_tol),
        BUCKET_NEGATIVE: finite & (values <
    -zero_tol),
        BUCKET_ZERO: finite & (np.abs(values) <=
    zero_tol),
        BUCKET_INVALID: ~finite,
        # superseded rule: (rho > 0); NaN and exact
    zeros were both counted as "not positive"
        "legacyPositive": values > 0,
    }
def point_direction_of(value: float, zero_tol: float
    = ZERO_TOL) -> str:
    try:
        number = float(value)
    except (TypeError, ValueError):
        return DIRECTION_UNDETERMINED
    if not math.isfinite(number) or abs(number) <=
    zero_tol:
        return DIRECTION_UNDETERMINED
    return DIRECTION_POSITIVE if number > 0.0 else
    DIRECTION_NEGATIVE
def sign_fields(n_positive: int, n_negative: int,
                n_zero: int, n_invalid: int, legacy_positive: int,
                reps: int, point_value: float,
    zero_tol: float = ZERO_TOL) -> dict:
    counts = {BUCKET_POSITIVE: int(n_positive),
    BUCKET_NEGATIVE: int(n_negative),
              BUCKET_ZERO: int(n_zero),
    BUCKET_INVALID: int(n_invalid)}
    reps = int(reps)
    valid = reps - counts[BUCKET_INVALID]
    if valid < 0 or sum(counts.values()) != reps:
        raise RuntimeError(f"sign counts do not
    partition the {reps} replicates: {counts}")
    direction = point_direction_of(point_value,
    zero_tol)
    if direction == DIRECTION_POSITIVE:
        aligned = counts[BUCKET_POSITIVE]
    elif direction == DIRECTION_NEGATIVE:
        aligned = counts[BUCKET_NEGATIVE]
    else:
        aligned = 0
    def ratio(value: int, denominator: int) -> float:
        return float(value) / float(denominator) if
    denominator > 0 else 0.0
    return {
        "n_positive": counts[BUCKET_POSITIVE],
    "n_negative": counts[BUCKET_NEGATIVE],
        "n_zero": counts[BUCKET_ZERO], "n_invalid":
    counts[BUCKET_INVALID],
        "valid": int(valid), "reps": reps, "aligned":
    int(aligned),
        "p_positive": ratio(counts[BUCKET_POSITIVE],
    valid),
        "p_negative": ratio(counts[BUCKET_NEGATIVE],
    valid),
        "p_positive_legacy": ratio(int(
    legacy_positive), reps),
        "sign_stability": ratio(aligned, valid),
        "point_direction": direction,
        "direction_status": direction,
        "direction_undetermined": direction ==
    DIRECTION_UNDETERMINED,
    }
def sign_summary(point_value: float,
    replicate_signed, zero_tol: float = ZERO_TOL) -> dict:
    values = np.asarray(replicate_signed,
    dtype=float).ravel()
    buckets = sign_buckets(values, zero_tol)
    return sign_fields(int(buckets[BUCKET_POSITIVE].su
    m()), int(buckets[BUCKET_NEGATIVE].sum()),
                       int(buckets[BUCKET_ZERO].sum()),
    int(buckets[BUCKET_INVALID].sum()),
                       int(buckets["legacyPositive"].su
    m()), int(values.size), point_value, zero_tol)
def load_train() -> pd.DataFrame:
    frame = C.load_frame(TRAIN_CSV)
    columns = C.chem_columns(frame)
    # asserts 231 chemical features
    targets = C.all_targets(frame)
    # asserts the 50 frozen targets
    C.style_of(frame)
    # asserts the style column exists
    if len(frame) != 200:
        raise RuntimeError(f"expected the 200
    SPLIT-0001 train rows, got {len(frame)}")
    if list(targets) != list(C.TARGETS_50):
        raise RuntimeError("v6_common.TARGETS_50
    order disagrees with the table")
```

```
    if list(TARGET_COLS) != list(C.TARGETS_50):
        raise RuntimeError("src/target_columns.py
    TARGET_COLS order differs from v6_common.TARGETS_50; "
                           "the two frozen orders
    must be identical before any matrix is built")
    if [column[len(C.CHEM_PREFIX):] for column in
    columns] != list(FEATURE_COLS):
        raise RuntimeError("table chem columns
    disagree with target_columns.FEATURE_COLS order")
    return frame.reset_index(drop=True)
def build_categories(meta: dict) -> dict:
    names = [record["name"] for record in
    meta["features"]]
    if names != list(FEATURE_COLS):
        raise RuntimeError("q2_chem_meta.json feature
    order differs from target_columns.FEATURE_COLS")
    kind = {record["name"]: record["kind"] for record
    in meta["features"]}
    unknown = sorted(set(kind.values()) -
    {"single_compound", "physical_parameter",
    "derived_sum"})
    if unknown:
        raise RuntimeError(f"unexpected meta kinds:
    {unknown}")

    element = list(C.NON_SINGLE_COMPOUND["element_or_i
    ndex"])
    physical = list(C.NON_SINGLE_COMPOUND["physical_pa
    rameter"])
    derived = list(C.NON_SINGLE_COMPOUND["derived_sum"
    ])
    for group, expected in (("element_or_index",
    element), ("physical_parameter", physical),
                            ("derived_sum", derived)):
        if len(expected) != len(set(expected)):
            raise RuntimeError(f"{group} list has
    duplicates")
        for name in expected:
            if name not in kind:
                raise RuntimeError(f"{group} member
    {name!r} is not a chemistry feature")
            if group == "physical_parameter" and
    kind[name] != "physical_parameter":
                raise RuntimeError(f"{name!r} is
    listed as a physical parameter but meta says
    {kind[name]!r}")
            if group == "derived_sum" and kind[name]
    != "derived_sum":
                raise RuntimeError(f"{name!r} is
    listed as a derived sum but meta says {kind[name]!r}")
            if group == "element_or_index" and
    kind[name] != "single_compound":
                raise RuntimeError(f"{name!r} is an
    element/index feature but meta says {kind[name]!r}")
    reclassified = [name for name in element if
    kind[name] == "single_compound"]

    single = [name for name in FEATURE_COLS if
    kind[name] == "single_compound" and name not in
    element]
    expected_single = 216
    meta_single = sum(1 for name in FEATURE_COLS if
    kind[name] == "single_compound")
    if len(single) != expected_single:
        reason = (f"meta tags {meta_single}
    single_compound features; removing the {len(element)}
    "
                  f"element_or_index members leaves
    {len(single)} (expected {expected_single})")
    else:
        reason = (f"meta tags {meta_single}
    single_compound features; the {len(element)}
    element_or_index "
                  f"members (iron, protein.g.L.,
    betaglucan.mg.L.) are reclassified out of the strict "
                  f"universe, {meta_single} -
    {len(element)} = {len(single)}")
    categories = {"single_compound": single,
    "element_or_index": element,
                  "physical_parameter": physical,
    "derived_sum": derived}
    universe = [name for group in categories.values()
    for name in group]
    if sorted(universe) != sorted(FEATURE_COLS) or
    len(universe) != N_FEATURES:
        raise RuntimeError("the category lists do not
    partition the 231 chemistry features exactly once")
    return {
        "groups": categories,
        "strictNames": single,
        "extendedNames": [name for name in
    FEATURE_COLS if name not in set(single)],
        "kindOf": {name: group for group, members in
    categories.items() for name in members},
        "counts": {group: len(members) for group,
    members in categories.items()},
        "strictSize": len(single),
        "strictSizeExpected": expected_single,
        "strictSizeMatchesExpected": bool(len(single)
    == expected_single),
        "metaSingleCompoundCount": meta_single,
        "metaReclassified": reclassified,
        "reason": reason,
    }
def build_chemistry_mapping(meta: dict, external:
pd.DataFrame) -> dict:
    features = meta["features"]
    names = [record["name"] for record in features]
    label_map: dict = {}
    for record in features:
        label_map.setdefault(QL.normalise_label(
    record["corrLabel"]), []).append(record["name"])
    name_map = {QL.normalise_label(name): [name] for
    name in names}
    ambiguous_labels = {key: value for key, value in
    label_map.items() if len(value) > 1}

    labels = [str(value) for value in
    external.iloc[:, 0].tolist()]
    normalised = [QL.normalise_label(label) for label
    in labels]
    if len(set(normalised)) != len(normalised):
        raise RuntimeError("6.csv has duplicate
    compound rows after normalisation")

    table, unmatched, ambiguous = [], [], []
    for label, key in zip(labels, normalised):
        candidates = sorted(set(label_map.get(key,
    [])) | set(name_map.get(key, [])))
        if len(candidates) == 1:
            position = len(table)
            name = candidates[0]
            table.append({"row": position, "label":
    label, "feature": name,
                          "kind": None, "corrIndex":
    None, "position": FEATURE_COLS.index(name)})
        elif len(candidates) > 1:
            ambiguous.append({"label": label,
    "candidates": candidates})
        else:
            unmatched.append({"label": label,
    "reason": "no corrLabel and no feature name matches"})
    if unmatched or ambiguous:
        raise RuntimeError(
            f"6.csv chemistry rows are not
    one-to-one: unmatched={unmatched}
    ambiguous={ambiguous}; "
            f"a partial row set would silently change
    the comparison denominator")
    matched_names = [record["feature"] for record in
    table]
    if len(set(matched_names)) != len(matched_names):
        raise RuntimeError("two 6.csv rows map to the
    same chemistry feature")

    kind = {record["name"]: record["kind"] for record
    in features}
    corr = {record["name"]: record["corrIndex"] for
```

```
    record in features}
    for record in table:
        record["corrIndex"] = int(
    corr[record["feature"]])
        record["kind"] = kind[record["feature"]]
    if any(record["corrIndex"] != record["position"]
    for record in table):
        raise RuntimeError("meta corrIndex disagrees
    with the FEATURE_COLS position; name and index
    disagree")

    return {
        "source": "6.csv column 1 (display name)
    matched to data/clean/q2_chem_meta.json corrLabel,
    then "
                  "name, after q2_lib.normalise_label
    (lower case, alphanumerics only)",
        "n": len(labels), "matched": len(table),
    "nUnmatched": len(unmatched), "nAmbiguous":
    len(ambiguous),
        "unmatched": unmatched, "ambiguous":
    ambiguous, "oneToOne": True,
        "duplicateNormalisedLabels": [{"label": key,
    "features": value}
                                      for key, value
    in sorted(ambiguous_labels.items())],
        "duplicateNormalisedLabelNote":
            "E-alpha-ionone and E-beta-ionone both
    normalise to 'eionone' (Greek letters are stripped).
            "No 6.csv row uses that label, so 44/44
    rows resolve; if one ever did, the run aborts instead
    of "
            "picking a feature by position.",
        "table": table,
    }
def build_sensory_mapping(external: pd.DataFrame,
    alignment_path: Path) -> dict:
    alignment = pd.read_csv(alignment_path)
    if list(alignment.columns) != ["external_header",
    "target"]:
        raise RuntimeError(f"{alignment_path} must
    have columns ['external_header', 'target']")
    if len(alignment) != N_DIMENSIONS:
        raise RuntimeError(f"alignment table has
    {len(alignment)} rows, expected {N_DIMENSIONS}")
    align_map = {QL.normalise_label(header):
    str(target)
                 for header, target in
    zip(alignment["external_header"],
    alignment["target"])}
    if len(align_map) != len(alignment):
        raise RuntimeError("alignment table has
    duplicate headers after normalisation")
    if len(set(align_map.values())) != len(align_map):
        raise RuntimeError("alignment table maps two
    headers to one target")
    if set(align_map.values()) != set(C.TARGETS_50):
        raise RuntimeError("alignment table targets
    differ from v6_common.TARGETS_50")
    if align_map != SENSORY_HEADER_TO_TARGET:
        differing = {key: (align_map.get(key),
    SENSORY_HEADER_TO_TARGET.get(key))
                     for key in set(align_map) |
    set(SENSORY_HEADER_TO_TARGET)
                     if align_map.get(key) !=
    SENSORY_HEADER_TO_TARGET.get(key)}
        raise RuntimeError(f"the literal dictionary
    and {alignment_path} disagree: {differing}")

    headers = [str(value) for value in
    external.columns[1:]]
    if len(headers) != N_DIMENSIONS:
        raise RuntimeError(f"6.csv carries
    {len(headers)} sensory columns, expected
    {N_DIMENSIONS}")
    norms = [QL.normalise_label(header) for header in
    headers]
    if len(set(norms)) != len(norms):
        raise RuntimeError("6.csv has duplicate
    sensory headers after normalisation")
    missing = [header for header, key in zip(headers,
    norms) if key not in SENSORY_HEADER_TO_TARGET]
    unused = sorted(set(SENSORY_HEADER_TO_TARGET) -
    set(norms))
    if missing or unused:
        raise RuntimeError(f"6.csv headers do not
    cover the dictionary: missing={missing}
    unused={unused}")
    targets = [SENSORY_HEADER_TO_TARGET[key] for key
    in norms]
    if len(set(targets)) != len(targets) or
    set(targets) != set(C.TARGETS_50):
        raise RuntimeError("6.csv sensory columns are
    not one-to-one onto the 50 targets")

    column_for_target = {target: index for index,
    target in enumerate(targets)}
    positional_mismatch = [{"column": index + 1,
    "header": headers[index],
                            "targetIfPositional":
    C.TARGETS_50[index],
                            "targetIfNamed":
    targets[index]}
                           for index in
    range(len(targets)) if targets[index] !=
    C.TARGETS_50[index]]
    return {
        "source": "explicit literal 50-entry
    dictionary, cross-checked against "
                  "MathorCup_A_ 独立审查与重跑方案/sensory_name_
    alignment.csv and the 6.csv header set",
        "n": len(headers), "matched": len(headers),
    "oneToOne": True, "unmatched": [], "ambiguous": [],
        "positionalMismatch": positional_mismatch,
        "positionalMismatchCount":
    len(positional_mismatch),
        "columnsForTargets": [column_for_target[target
    ] for target in C.TARGETS_50],
        "table": [{"externalHeader": header,
    "normalised": key, "target": target,
                   "externalColumnIndex": index + 1,
    "targetPosition": C.TARGETS_50.index(target)}
                  for index, (header, key, target) in
    enumerate(zip(headers, norms, targets))],
    }
def build_mapping(meta: dict) -> dict:
    external = pd.read_csv(EXTERNAL_CSV)
    chemistry = build_chemistry_mapping(meta,
    external)
    sensory = build_sensory_mapping(external,
    ALIGN_CSV)
    assertions = [
        {"id": "M1", "description": "every 6.csv
    chemistry row maps to exactly one feature by name",
         "passed": chemistry["oneToOne"] and
    chemistry["nUnmatched"] == 0 and
    chemistry["nAmbiguous"] == 0,
         "detail": f"{chemistry['matched']}/{chemistry
    ['n']} matched"},
        {"id": "M2", "description": "no two 6.csv
    rows map to the same feature",
         "passed": True, "detail":
    f"{chemistry['matched']} distinct features"},
        {"id": "M3", "description": "meta corrIndex
    equals the FEATURE_COLS position for every matched
    row",
         "passed": True, "detail": "name and index
    spaces agree"},
        {"id": "M4", "description": "every 6.csv
    sensory column maps to exactly one of the 50 targets",
         "passed": sensory["oneToOne"], "detail":
    f"{sensory['matched']}/{sensory['n']} matched"},
        {"id": "M5", "description": "literal
    dictionary == alignment table == 6.csv header set",
         "passed": True, "detail": "three independent
    sources agree on 50/50"},
        {"id": "M6", "description": "positional
    reading of the sensory columns is wrong (evidence,
```

```
    not a guess)",                                          for "
     "passed": sensory["positionalMismatchCount"]                       "most picks. moduleSize is
    > 0,                                                    reported next to every alternative so this is
     "detail": f"{sensory['positionalMismatchCount          visible, "
    ']} columns differ: " +                                             "not hidden."}
                ", ".join(row["header"] for row in      def attach_modules(board: dict, modules: dict) ->
    sensory["positionalMismatch"])},                        None:
    ]                                                       groups: dict = {}
    failed = [record for record in assertions if not        for name in board["names"]:
    record["passed"]]                                           groups.setdefault(modules["perCompound"][name]
    if failed:                                              ["moduleId"], []).append(name)
        raise RuntimeError(f"mapping assertions             board["byModule"] = groups
    failed: {failed}")                                  def pointscores(frame: pd.DataFrame, tau: float =
    return {"chemistry": chemistry, "sensory":              TAU) -> dict:
    sensory, "assertions": assertions,                      imputed = L.imputed_blocks(frame, {"train":
            "files": {str(EXTERNAL_CSV.relative_to(         frame})["train"]
    PROJECT)): C.sha256_file(EXTERNAL_CSV),                 features = imputed[:, :L.N_FEATURES]
                    str(ALIGN_CSV.relative_to(              targets = L.raw_target_matrix(frame)
    PROJECT)): C.sha256_file(ALIGN_CSV),                    signed_total = _guard_matrix(QC.spearman_matrix(
                    str(CHEM_META.relative_to(              features, targets), "signed_total")
    PROJECT)): C.sha256_file(CHEM_META),                    basis = QL.style_basis(frame[STYLE_COL].to_numpy(
                    str(TRAIN_CSV.relative_to(              ))
    PROJECT)): C.sha256_file(TRAIN_CSV),                    within_signed = _guard_matrix(
                    str(CLEAN_CSV.relative_to(                  QL.column_correlation(QL.residualise(
    PROJECT)): C.sha256_file(CLEAN_CSV)} }                  QL.midranks(features), basis),
def make_board(names: list, label: str) -> dict:                                    QL.residualise(
    rows = np.asarray([FEATURE_COLS.index(name) for         QL.midranks(targets), basis)), "within_signed")
    name in names], dtype=int)                              fitted = L.FittedModel(frame, "zscore",
    if rows.size != len(names) or                           GRAPH_SETTINGS["rank"], GRAPH_SETTINGS["lambda"],
    len(set(rows.tolist())) != len(names):                                  GRAPH_SETTINGS["gamma"],
        raise RuntimeError(f"{label} board:                 tau=tau)
    name->position mapping is not injective")               beta = _guard_matrix(np.asarray(
    if rows.size == 0 or int(rows.max()) >=                 fitted.model["beta"])[:L.N_FEATURES, :], "graph_beta")
    N_FEATURES:                                             return {"signedTotal": signed_total,
        raise RuntimeError(f"{label} board: position        "total_spearman": np.abs(signed_total),
    outside the 231-feature space")                                "within_signed": within_signed,
    name_rank = {name: rank for rank, name in               "within_style_spearman": np.abs(within_signed),
    enumerate(sorted(names))}                                      "graph_lowrank": np.abs(beta),
    return {"label": label, "names": list(names),           "graph_beta_signed": beta, "graph": fitted.graph}
    "rows": rows,                                       def board_ranked(scores: dict, board: dict) -> dict:
            "nameRank": np.asarray([name_rank[name]         rows = board["rows"]
    for name in names], dtype=int),                         ranked = {method: QC.percentile_rank(
            "index": {name: local for local, name in        scores[method][rows, :]) for method in BASE_METHODS}
    enumerate(names)}, "size": len(names)}                  ranked["consensus"] = (ranked["total_spearman"] +
def module_lookup(graph: dict) -> dict:                     ranked["within_style_spearman"]
    table = QL.module_table(graph)                                      + ranked["graph_lowrank"])
    labels = np.asarray(table["labels"], dtype=int)         / 3.0
    if labels.size != N_FEATURES:                           for method in METHODS:
        raise RuntimeError(f"module labels cover                if ranked[method].shape != (board["size"],
    {labels.size} features, expected {N_FEATURES}")         N_DIMENSIONS):
    sizes = np.bincount(labels)                                     raise RuntimeError(f"{board['label']}/{met
    per = {}                                                hod}: percentile block has the wrong shape")
    for name in FEATURE_COLS:                               return ranked
        position = FEATURE_COLS.index(name)             def order_by_score(ranked: dict, board: dict) -> dict:
        per[name] = {"moduleId":                            out = {}
    int(labels[position]), "moduleSize":                    for method in METHODS:
    int(sizes[labels[position]])}                               matrix = ranked[method]
    return {"count": int(table["count"]),                       out[method] = [np.lexsort((board["nameRank"],
    "edgeCount": int(table["edgeCount"]),                   -matrix[:, dimension]))[:K].tolist()
            "moduleSizes": [int(value) for value in                         for dimension in
    table["moduleSizes"]], "perCompound": per,              range(N_DIMENSIONS)]
            "largestModuleSize": int(max(int(value)         return out
    for value in table["moduleSizes"])),                def order_by_pi(ranked: dict, pi: dict, board: dict)
            "largestModuleShare":                           -> dict:
    float(max(int(value) for value in                       out = {}
    table["moduleSizes"]) / N_FEATURES),                    rows = board["rows"]
            "partitionBalanced": bool(max(int(value)        for method in METHODS:
    for value in table["moduleSizes"]) <= N_FEATURES //         matrix = ranked[method]
    3),                                                         out[method] = [np.lexsort((board["nameRank"],
            "definition": "connected components of          -matrix[:, dimension],
    the point-estimate training graph |W| > 0 at tau =                                  -pi[method][rows,
    0.5 over "                                              dimension]))[:K].tolist()
                          "the 231 chemistry features                   for dimension in
    (200 SPLIT-0001 train rows)",                           range(N_DIMENSIONS)]
            "readingNote": "the partition is                return out
    UNBALANCED: one component absorbs most features; see    def sets_as_names(chosen: dict, board: dict) -> dict:
    the "                                                   return {method: [[board["names"][int(local)]] for
                           "non-delivered same-module       local in chosen[method][dimension]]
    substitute pool is nearly the global next-best list                 for dimension in
```

```
    range(N_DIMENSIONS)] for method in METHODS}
def bootstrap(frame: pd.DataFrame, seed: int, reps:
    int, boards: dict, paths: dict) -> dict:
    styles = frame[STYLE_COL].to_numpy()
    style_order = sorted(set(styles.tolist()))
    groups = [np.nonzero(styles == style)[0] for
    style in style_order]
    if sum(group.size for group in groups) !=
    len(frame):
        raise RuntimeError("style strata do not cover
    the training rows")
    rng = np.random.default_rng(int(seed) + 1000003 *
    21)

    counts = {method: np.zeros((N_FEATURES,
    N_DIMENSIONS), dtype=np.int32) for method in METHODS}
    sign = {key: np.zeros((N_FEATURES, N_DIMENSIONS),
    dtype=np.int32)
            for key in BUCKET_KEYS +
    ("legacyPositive",)}
    handles = {label: open(path, "w",
    encoding="utf-8", newline="") for label, path in
    paths.items()}
    writers = {label: csv.writer(handle,
    lineterminator="\n") for label, handle in
    handles.items()}
    for writer in writers.values():
        writer.writerow(BOOT_HEADER)
    rows_written = {label: 0 for label in paths}
    try:
        for replicate in range(int(reps)):
            indices = np.concatenate(
    [group[rng.integers(0, group.size, size=group.size)]
                                      for group in
    groups])
            sample = frame.iloc[indices].reset_index(
    drop=True)
            scores = pointscores(sample)
            for key, mask in sign_buckets(
    scores["signedTotal"], ZERO_TOL).items():
                sign[key] += mask.astype(np.int32)
            for label, board in boards.items():
                chosen = order_by_score(board_ranked(
    scores, board), board)
                writer = writers[label]
                for method in METHODS:
                    for dimension in
    range(N_DIMENSIONS):
                        for rank, local in
    enumerate(chosen[method][dimension]):
                            counts[method][board["rows
    "][int(local)], dimension] += 1
                            writer.writerow(
    [replicate, int(seed), C.TARGETS_50[dimension],
    method,
                                             rank +
    1, board["names"][int(local)]])
                            rows_written[label] += 1
    finally:
        for handle in handles.values():
            handle.close()
    expected = int(reps) * N_DIMENSIONS *
    len(METHODS) * K
    for label, written in rows_written.items():
        if written != expected:
            raise RuntimeError(f"{label}: wrote
    {written} bootstrap rows, expected {expected}")
    # every cell must be partitioned exactly once per
    replicate; a miss would silently move a denominator
    totals = sum(sign[key] for key in BUCKET_KEYS)
    if not np.array_equal(totals,
    np.full((N_FEATURES, N_DIMENSIONS), int(reps),
    dtype=np.int32)):
        raise RuntimeError("sign buckets do not cover
    every replicate for every (compound, dimension) cell")
    return {"counts": counts, "sign": sign,
    "zeroTol": float(ZERO_TOL), "reps": int(reps),
            "rowsWritten": rows_written,
            "strata": {style: int(group.size) for
    style, group in zip(style_order, groups)} }
def frequencies(counts: dict, reps: int) -> dict:
    return {method: counts[method].astype(float) /
    float(reps) for method in METHODS}
def module_alternatives(method: str, dimension: int,
    module_id: int, board: dict, ranked: dict,
                        delivered: set) -> tuple:
    members = board["byModule"].get(module_id, [])
    if not members:
        return NO_ALTERNATIVE, []
    score = ranked[method]
    ordered = sorted(members, key=lambda name:
    (-float(score[board["index"][name], dimension]),
    name))
    candidates = [name for name in ordered if name
    not in delivered]
    alternative = candidates[0] if candidates else
    NO_ALTERNATIVE
    return alternative, [name for name in
    candidates[:10] if name not in delivered]
def delivered_table(board: dict, ranked: dict, pi:
    dict, sign: dict, scores: dict,
                    modules: dict, kind_of: dict,
    reps: int, zero_tol: float = ZERO_TOL) -> tuple:
    chosen = order_by_pi(ranked, pi, board)
    rows, sets = [], {method: [] for method in
    METHODS}
    for method in METHODS:
        for dimension, target in
    enumerate(C.TARGETS_50):
            picked = [board["names"][int(local)] for
    local in chosen[method][dimension]]
            if len(set(picked)) != K:
                raise RuntimeError(
    f"{board['label']}/{method}/{target}: duplicate names
    in a Top-3 set")
            sets[method].append(picked)
            delivered = set(picked)
            for rank, local in enumerate(
    chosen[method][dimension]):
                name = board["names"][int(local)]
                grow = int(board["rows"][int(local)])
                value = float(pi[method][grow,
    dimension])
                fields = sign_fields(
                    int(sign[BUCKET_POSITIVE][grow,
    dimension]), int(sign[BUCKET_NEGATIVE][grow,
    dimension]),
                    int(sign[BUCKET_ZERO][grow,
    dimension]), int(sign[BUCKET_INVALID][grow,
    dimension]),
                    int(sign["legacyPositive"][grow,
    dimension]), int(reps),
                    float(scores["signedTotal"][grow,
    dimension]), zero_tol)
                alternative, _pool =
    module_alternatives(
                    method, dimension,
    modules["perCompound"][name]["moduleId"], board,
    ranked, delivered)
                if fields["direction_undetermined"]:
                    status = DIRECTION_UNDETERMINED
                elif value >= PI_STABLE and
    fields["sign_stability"] >= SIGN_STABLE:
                    status = "stable"
                elif value >= PI_STABLE:
                    status = "sign-mixed"
                elif fields["sign_stability"] >=
    SIGN_STABLE:
                    status = "low-pi"
                else:
                    status = "low-pi+sign-mixed"
                rows.append({
                    "dimension": target, "rank": rank
    + 1, "compound": name, "kind": kind_of[name],
                    "method": method,
                    "score": float(ranked[method][int(
    local), dimension]),
                    "pi": value, "pi_mc_se":
```

```
    math.sqrt(max(value * (1.0 - value), 0.0) /
    float(reps)),
                    "sign_stability":
    fields["sign_stability"],
                    # `direction` is kept as a
    TRI-STATE alias of point_direction so the existing
    paper /
                    # figure scripts that read this
    column keep working; the superseded binary rule that
                    # wrote "negative" for a zero or
    missing value is gone.
                    "direction":
    fields["point_direction"],
                    "p_positive":
    fields["p_positive"], "p_negative":
    fields["p_negative"],
                    "p_positive_legacy":
    fields["p_positive_legacy"],
                    "n_positive":
    fields["n_positive"], "n_negative":
    fields["n_negative"],
                    "n_zero": fields["n_zero"],
    "n_invalid": fields["n_invalid"],
                    "point_direction":
    fields["point_direction"],
                    "direction_status":
    fields["direction_status"],
                    "module_id":
    int(modules["perCompound"][name]["moduleId"]),
                    "alternative_rank4": alternative,
    "evidence_status": status,
                })
    return rows, sets
def module_alternative_report(table: list, board:
    dict, ranked: dict) -> list:
    delivered_by_cell = {}
    for row in table:
        delivered_by_cell.setdefault((row["method"],
    row["dimension"]), set()).add(row["compound"])
    out = []
    for row in table:
        method, dimension = row["method"],
    C.TARGETS_50.index(row["dimension"])
        members = board["byModule"].get(
    row["module_id"], [])
        score = ranked[method][:, dimension]
        ordered = sorted(members, key=lambda name:
    (-float(score[board["index"][name]]), name))
        delivered = delivered_by_cell[(method,
    row["dimension"])]
        candidates = [name for name in ordered if
    name not in delivered]
        # board-wide fallback: the best candidate
    that is NOT delivered, so a pick whose module is too
    small
        # for a same-module substitute still exposes
    a substitute (clearly labelled board-wide, not
    same-module)
        global_order = sorted(board["names"],
    key=lambda name: (-float(score[board["index"][name]]),
    name))
        fallback = next((name for name in
    global_order if name not in delivered),
    NO_ALTERNATIVE)
        out.append({"dimension": row["dimension"],
    "method": method, "rank": row["rank"],
                    "compound": row["compound"],
    "pi": row["pi"], "moduleId": row["module_id"],
                    "moduleSize": len(members),
    "moduleOrder": ordered[:10],
                    "moduleRank4To10":
    candidates[:10],
                    "sameModuleAlternativeExists":
    bool(candidates),
                    "globalFallback": fallback})
    return out
def sign_counts_rows(sign: dict, scores: dict,
    kind_of: dict, reps: int,
                     zero_tol: float = ZERO_TOL) ->
    list:
    rows = []
    signed_total = _guard_matrix(
    scores["signedTotal"], "signed_total")
    for name in FEATURE_COLS:
        grow = FEATURE_COLS.index(name)
        for dimension, target in
    enumerate(C.TARGETS_50):
            fields = sign_fields(
                int(sign[BUCKET_POSITIVE][grow,
    dimension]), int(sign[BUCKET_NEGATIVE][grow,
    dimension]),
                int(sign[BUCKET_ZERO][grow,
    dimension]), int(sign[BUCKET_INVALID][grow,
    dimension]),
                int(sign["legacyPositive"][grow,
    dimension]), int(reps),
                float(signed_total[grow, dimension]),
    zero_tol)
            rows.append([target, name, kind_of[name],
    fmt(signed_total[grow, dimension]),
                         fields["point_direction"],
    fields["n_positive"], fields["n_negative"],
                         fields["n_zero"],
    fields["n_invalid"], fields["valid"],
                         fmt(fields["p_positive"]),
    fmt(fields["p_negative"]),
                         fmt(fields["p_positive_legacy
    "]), fmt(fields["sign_stability"]),
                         fields["direction_status"]])
    return rows
def low_pi_rows(table: list, module_report: list,
    pi_threshold: float = PI_STABLE) -> list:
    alternatives = {(record["dimension"],
    record["method"], record["rank"]): record
                    for record in module_report}
    out = []
    for row in table:
        if row["method"] != "consensus" or row["pi"]
    >= pi_threshold:
            continue
        record = alternatives.get((row["dimension"],
    row["method"], row["rank"]), {})
        pool = record.get("moduleRank4To10", [])
        out.append({
            "dimension": row["dimension"], "rank":
    row["rank"], "compound": row["compound"],
            "kind": row["kind"], "pi": row["pi"],
    "pi_mc_se": row["pi_mc_se"],
            "point_direction": row["point_direction"],
    "p_positive": row["p_positive"],
            "p_negative": row["p_negative"],
    "p_positive_legacy": row["p_positive_legacy"],
            "n_positive": row["n_positive"],
    "n_negative": row["n_negative"],
            "n_zero": row["n_zero"], "n_invalid":
    row["n_invalid"],
            "sign_stability": row["sign_stability"],
    "direction_status": row["direction_status"],
            "evidence_status": row["evidence_status"],
    "module_id": row["module_id"],
            "module_size": int(record.get(
    "moduleSize", 0)),
            "alternative_rank4":
    row["alternative_rank4"],
            "module_rank4_to_10": "; ".join(pool) if
    pool else NO_ALTERNATIVE,
            "same_module_alternative_exists":
    bool(record.get("sameModuleAlternativeExists",
    False)),
            "global_fallback": record.get(
    "globalFallback", NO_ALTERNATIVE),
        })
    return out
def spearman_between(left: np.ndarray, right:
    np.ndarray) -> float:
    left = np.asarray(left, dtype=float)
    right = np.asarray(right, dtype=float)
    if left.size < 3 or right.size != left.size:
```

```
        raise RuntimeError("spearman_between needs
    two equal vectors of at least 3 entries")
    ranked_left = QL.midranks(left.reshape(-1, 1))[:,
    0]
    ranked_right = QL.midranks(right.reshape(-1,
    1))[:, 0]
    ranked_left = ranked_left - ranked_left.mean()
    ranked_right = ranked_right - ranked_right.mean()
    denominator = float(np.linalg.norm(ranked_left) *
    np.linalg.norm(ranked_right))
    if denominator <= 1e-12:
        raise RuntimeError("constant vector: rank
    correlation is undefined")
    return float(ranked_left @ ranked_right /
    denominator)
EXPORT_FILES = (FILE_STRICT, FILE_STRICT_METHODS,
    FILE_EXTENDED, FILE_EXTENDED_METHODS,
                FILE_SIGN_COUNTS, FILE_LOWPI,
    FILE_SIGN_SUMMARY)
REPORT_HEADER = ["dimension", "rank", "compound",
    "pi", "pi_mc_se", "sign_stability", "p_positive",
                 "p_negative", "p_positive_legacy",
    "n_positive", "n_negative", "n_zero", "n_invalid",
                 "point_direction", "evidence_status"]
```

C.3 问题三：两层收缩边际、地板质量点、Copula、能量距离与迁移问题（src/q3_lib.py）

```
# AI-ASSISTANCE NOTICE: the framework of this module
    was drafted with DeepSeek Harness assistance;
# the core algorithm logic, constraint corrections
    and experimental parameters were reviewed, amended
# and independently verified by the team members, who
    take responsibility for the contents.

CHEM_PREFIX = "chem__"
CHEM_COLS = [CHEM_PREFIX + name for name in
    FEATURE_COLS]
LABEL_COLS = list(TARGET_COLS)
LOG10_SET = set(LOG10_CALIBRATED)
LINEAR_SET = set(LINEAR_CALIBRATED)
FLOOR_SET = set(FLOOR_COLUMNS)
THRESHOLD_MEAN = {"DMS": 17.25, "X2.3.butanedione":
    0.29925, "X4VG": 0.206333333}
THRESHOLD_MEDIAN = {"DMS": 19.0, "X2.3.butanedione":
    0.09, "X4VG": 0.2}
DEFECT_KEYS = ("DMS", "X2.3.butanedione", "X4VG")
T2N = "t2n"
STALE = "stale_hops"
PROXY_TARGETS = (T2N, STALE)
FLOOR_MIN_REPEATS = 20
PROXY_MARGIN = 1e-6
MIN_STYLE_OBS = 3
def prepare(frame: pd.DataFrame) -> dict:
    X = frame[CHEM_COLS].to_numpy(dtype=float)
    return {
        "X": X,
        "styles": frame[STYLE_COLUMN].astype(
    str).to_numpy(),
        "rows": frame["row"].to_numpy(dtype=int),
        "Y": frame[LABEL_COLS].to_numpy(dtype=float),
        "n": X.shape[0],
    }
def column_medians(X: np.ndarray) -> np.ndarray:
    med = np.nanmedian(X, axis=0)
    return np.where(np.isfinite(med), med, 0.0)
def apply_median(X: np.ndarray, med: np.ndarray) ->
    np.ndarray:
    out = np.array(X, dtype=float, copy=True)
    bad = ~np.isfinite(out)
    if bad.any():
        rows, cols = np.nonzero(bad)
        out[rows, cols] = med[cols]
    return out
def fit_standardizer(X: np.ndarray) -> tuple:
    centre = X.mean(axis=0)
    scale = X.std(axis=0)
    scale = np.where(np.isfinite(scale) & (scale >
    1e-12), scale, 1.0)
    return centre, scale
def standardize(X: np.ndarray, centre: np.ndarray,
    scale: np.ndarray) -> np.ndarray:
    return (np.asarray(X, dtype=float) - centre) /
    scale
def floor_code_map(X: np.ndarray, min_repeats: int =
    FLOOR_MIN_REPEATS) -> dict:
    X = np.asarray(X, dtype=float)
    codes = {}
    for j, name in enumerate(FEATURE_COLS):
        values = X[:, j]
        values = values[np.isfinite(values)]
        if values.size == 0:
            continue
        if name in FLOOR_SET:
            codes[name] = float(FLOOR_VALUE)
            continue
        if name in LOG10_SET:
            low = float(values.min())
            repeats = int(np.sum(np.isclose(values,
    low, rtol=1e-6, atol=0.0)))
            if repeats >= int(min_repeats):
                codes[name] = low
    return codes
def floor_mask(X: np.ndarray, codes: dict) ->
    np.ndarray:
    mask = np.zeros(np.shape(X), dtype=bool)
    for j, name in enumerate(FEATURE_COLS):
        if name in codes:
            mask[:, j] = np.isclose(X[:, j],
    codes[name], rtol=1e-6, atol=0.0)
    return mask
def physical_conc(column: str, storage, codes: dict =
    None):
    values = np.asarray(storage, dtype=float)
    if column in LOG10_SET:
        out = np.power(10.0, values)
    elif column in LINEAR_SET:
        out = values.copy()
    else:
        out = values.copy()
    if codes and column in codes:
        out = np.where(np.isclose(values,
    codes[column], rtol=1e-6, atol=0.0), 0.0, out)
    return out
def storage_bound(column: str, physical_bound):
    if column in LOG10_SET:
        return np.log10(np.maximum(np.asarray(
    physical_bound, dtype=float), 1e-300))
    return np.asarray(physical_bound, dtype=float)
def constraint_arrays(X: np.ndarray, t2n_bound,
    proxy, codes: dict, thresholds: dict,
                      stale_bound=None) -> dict:
    X = np.asarray(X, dtype=float)
    out = {}
    for name in DEFECT_KEYS:
        j = FEATURE_COLS.index(name)
        conc = physical_conc(name, X[:, j], codes)
        bound = float(thresholds[name])
        out["excess__" + name] = np.maximum(conc /
    bound - 1.0, 0.0)
    pred = proxy.predict_storage(X, T2N)
    out["t2n_margin"] = np.asarray(t2n_bound,
    dtype=float) - pred
    pred_stale = proxy.predict_storage(X, STALE)
    if stale_bound is None:
        stale = np.full(len(pred_stale),
    np.sqrt(np.finfo(float).max), dtype=float)
    else:
        stale = np.asarray(stale_bound, dtype=float)
    out["stale_hops_margin"] = stale - pred_stale
    return out
def constraint_satisfied(parts: dict) -> np.ndarray:
    ok = np.ones(np.asarray(parts["t2n_margin"]).shape
    [0], dtype=bool)
    for name in DEFECT_KEYS:
```

```
        ok &= np.asarray(parts["excess__" + name]) <=
    0.0
    ok &= np.asarray(parts["t2n_margin"]) >= 0.0
    if "stale_hops_margin" in parts:
        ok &= np.asarray(parts["stale_hops_margin"])
    >= 0.0
    return ok
def defect_exceeded(parts: dict) -> np.ndarray:
    bad = np.zeros(np.asarray(parts["t2n_margin"]).sha
    pe[0], dtype=bool)
    for name in DEFECT_KEYS:
        bad |= np.asarray(parts["excess__" + name]) >
    0.0
    return bad
def project_thresholds(X: np.ndarray, physical_caps:
    dict, codes: dict) -> np.ndarray:
    out = np.array(X, dtype=float, copy=True)
    for name, cap in physical_caps.items():
        j = FEATURE_COLS.index(name)
        out[:, j] = np.minimum(out[:, j],
    float(np.asarray(storage_bound(name, cap))))
    return out
def project_proxy_bound(X: np.ndarray, proxy, target:
    str, bounds, centre, scale,
                        excluded_columns=(),
    max_rounds: int = 4) -> np.ndarray:
    X = np.array(X, dtype=float, copy=True)
    bounds = np.broadcast_to(np.asarray(bounds,
    dtype=float), (X.shape[0],)).copy() - PROXY_MARGIN
    weight = np.array(proxy.coef_for(target),
    dtype=float)
    intercept = float(proxy.intercept_for(target))
    excl = np.zeros(len(FEATURE_COLS), dtype=bool)
    for name in excluded_columns:
        excl[FEATURE_COLS.index(name)] = True
    direction = np.where(excl, 0.0, weight)
    norm_sq = float(np.sum(direction * direction))
    if norm_sq <= 1e-18:
        return X
    for _ in range(int(max_rounds)):
        Z = standardize(X, centre, scale)
        pred = Z @ weight + intercept
        gap = pred - bounds
        active = gap > 1e-12
        if not active.any():
            break
        delta = gap[active] / norm_sq
        X[active, :] -= delta[:, None] * (scale *
    direction)[None, :]
    return X
def project_transfer_feasible(problem, v: np.ndarray,
    t2n_caps: np.ndarray, stale_caps: np.ndarray,
                              rounds: int = 6) ->
    np.ndarray:
    out = np.clip(np.array(v, dtype=float,
    copy=True), 0.0, 1.0)
    for _ in range(int(rounds)):
        out = np.clip(out, 0.0, problem.v_upper)
        out = problem.shift_along_proxy(out, STALE,
    stale_caps)
        out = problem.shift_along_proxy(out, T2N,
    t2n_caps)
        out = np.clip(out, 0.0, problem.v_upper)
        if problem.proxy_ok(out, t2n_caps,
    stale_caps):
            break
    return out
class LabelProxy:

    def __init__(self, centre, scale, alphas=(0.1,
    1.0, 10.0, 100.0, 1000.0, 10000.0), folds: int = 5):
        self.centre = np.asarray(centre, dtype=float)
        self.scale = np.asarray(scale, dtype=float)
        self.alphas = tuple(float(a) for a in alphas)
        self.folds = int(folds)
        self.alpha = {}
        self.coef = None
        self.intercept = None
        self.cv_r2 = {}

    # -- internals
    def _design(self, X):
        return standardize(np.asarray(X,
    dtype=float), self.centre, self.scale)

    @staticmethod
    def _ridge_fit(Z, y, alpha):
        zm = Z.mean(axis=0)
        ym = float(np.mean(y))
        Zc = Z - zm
        gram = Zc.T @ Zc
        rhs = Zc.T @ (y - ym)
        coef = np.linalg.solve(gram + float(alpha) *
    np.eye(gram.shape[0]), rhs)
        intercept = ym - float(zm @ coef)
        return coef, intercept

    def fit(self, X, Y):
        Z = self._design(X)
        Y = np.asarray(Y, dtype=float)
        n = Z.shape[0]
        order = np.arange(n)
        blocks = np.array_split(order, self.folds)
        # per-fold gram matrices are shared by every
    alpha and every target
        grams, rhss, means = [], [], []
        for block in blocks:
            keep = np.ones(n, dtype=bool)
            keep[block] = False
            Zt = Z[keep]
            Yt = Y[keep]
            zm = Zt.mean(axis=0)
            ym = Yt.mean(axis=0)
            Zc = Zt - zm
            grams.append(Zc.T @ Zc)
            rhss.append(Zc.T @ (Yt - ym))
            means.append((zm, ym))
        identity = np.eye(Z.shape[1])
        best = np.full(Y.shape[1], -np.inf)
        chosen = np.ones(Y.shape[1], dtype=float) *
    self.alphas[0]
        for alpha in self.alphas:
            pred = np.zeros_like(Y)
            for k, block in enumerate(blocks):
                zm, ym = means[k]
                coefs = np.linalg.solve(grams[k] +
    alpha * identity, rhss[k])  # (231, 50)
                pred[block] = (Z[block] - zm) @ coefs
    + ym
            resid = np.sum((Y - pred) ** 2, axis=0)
            total = np.sum((Y - Y.mean(axis=0)) ** 2,
    axis=0)
            with np.errstate(divide="ignore",
    invalid="ignore"):
                r2 = 1.0 - resid / np.where(total >
    1e-12, total, np.inf)
            better = r2 > best
            best = np.where(better, r2, best)
            chosen = np.where(better, alpha, chosen)
        self.alpha = {name: float(chosen[j]) for j,
    name in enumerate(LABEL_COLS)}
        self.cv_r2 = {name: float(best[j]) for j,
    name in enumerate(LABEL_COLS)}
        coef = np.zeros((Y.shape[1], Z.shape[1]),
    dtype=float)
        intercept = np.zeros(Y.shape[1], dtype=float)
        for alpha in sorted(set(float(a) for a in
    chosen)):
            targets = [j for j in range(Y.shape[1])
    if float(chosen[j]) == alpha]
            zm = Z.mean(axis=0)
            Zc = Z - zm
            ym = Y[:, targets].mean(axis=0)
            gram = Zc.T @ Zc
            rhs = Zc.T @ (Y[:, targets] - ym)
            block = np.linalg.solve(gram + alpha *
    identity, rhs)
            coef[targets, :] = block.T
```

```
            intercept[targets] = ym - zm @ block
        self.coef = coef
        self.intercept = intercept
        return self

    # -- prediction
    def predict_z(self, Z):
        return np.asarray(Z, dtype=float) @
    self.coef.T + self.intercept

    def predict_storage(self, X, target: str = None):
        pred = self.predict_z(self._design(X))
        if target is None:
            return pred
        return pred[:, LABEL_COLS.index(target)]

    def coef_for(self, target: str):
        return self.coef[LABEL_COLS.index(target)]

    def intercept_for(self, target: str):
        return float(self.intercept[LABEL_COLS.index(
    target)])

    def refit_with_alphas(self, X, Y):
        Z = self._design(X)
        Y = np.asarray(Y, dtype=float)
        identity = np.eye(Z.shape[1])
        coef = np.zeros((Y.shape[1], Z.shape[1]),
    dtype=float)
        intercept = np.zeros(Y.shape[1], dtype=float)
        zm = Z.mean(axis=0)
        Zc = Z - zm
        gram = Zc.T @ Zc
        for alpha in sorted(set(self.alpha.values())):
            targets = [j for j in range(Y.shape[1])
    if float(self.alpha[LABEL_COLS[j]]) == alpha]
            ym = Y[:, targets].mean(axis=0)
            rhs = Zc.T @ (Y[:, targets] - ym)
            block = np.linalg.solve(gram + alpha *
    identity, rhs)
            coef[targets, :] = block.T
            intercept[targets] = ym - zm @ block
        clone = LabelProxy(self.centre, self.scale,
    alphas=self.alphas, folds=self.folds)
        clone.alpha = dict(self.alpha)
        clone.cv_r2 = dict(self.cv_r2)
        clone.coef = coef
        clone.intercept = intercept
        return clone
class Marginal:

    __slots__ = ("grid_x", "grid_u", "pi",
    "floor_code", "has_floor")

    def __init__(self, grid_x, grid_u, pi,
    floor_code):
        self.grid_x = grid_x
        self.grid_u = grid_u
        self.pi = float(pi)
        self.floor_code = None if floor_code is None
    else float(floor_code)
        self.has_floor = self.floor_code is not None

    def forward(self, x):
        x = np.asarray(x, dtype=float)
        u = np.interp(x, self.grid_x, self.grid_u,
    left=self.grid_u[0], right=self.grid_u[-1])
        if self.has_floor:
            hit = np.isclose(x, self.floor_code,
    rtol=1e-6, atol=0.0)
            if hit.any():
                u = np.where(hit, 0.5 * self.pi, u)
        return u

    def inverse(self, u):
        u = np.asarray(u, dtype=float)
        t = np.clip((u - self.pi) / max(1.0 -
    self.pi, 1e-12), 0.0, 1.0)
        base = (self.grid_u - self.pi) / max(1.0 -
    self.pi, 1e-12)
        x = np.interp(t, base, self.grid_x,
    left=self.grid_x[0], right=self.grid_x[-1])
        if self.has_floor:
            x = np.where(u <= self.pi,
    self.floor_code, x)
        return x
def _midrank_cdf(sorted_values, grid):
    below = np.searchsorted(sorted_values, grid,
    side="left")
    equal = np.searchsorted(sorted_values, grid,
    side="right") - below
    return (below + 0.5 * equal) /
    float(sorted_values.size)
def build_marginal(cont_style, cont_pool, pi_style,
    pi_pool, omega, floor_code, eps=1e-9):
    cont_style = np.sort(np.asarray(cont_style,
    dtype=float))
    cont_pool = np.sort(np.asarray(cont_pool,
    dtype=float))
    if float(omega) <= 0.0:                      #
    declared pooled fallback: the style grid must not
    enter at all
        cont_style = np.zeros(0, dtype=float)
    if cont_style.size == 0:
        cont_style = cont_pool
    if cont_pool.size == 0:
        cont_pool = cont_style
    if cont_style.size == 0:
        cont_style = np.array([0.0])
        cont_pool = np.array([0.0])
    grid = np.union1d(cont_style, cont_pool)
    u_style = _midrank_cdf(cont_style, grid)
    u_pool = _midrank_cdf(cont_pool, grid)
    u = float(omega) * u_style + (1.0 - float(omega))
    * u_pool
    pi = float(np.clip(float(omega) * float(pi_style)
    + (1.0 - float(omega)) * float(pi_pool), 0.0, 1.0 -
    1e-6))
    u = pi + (1.0 - pi) * u
    # strictly increasing: cumulative max + a
    negligible rank ramp to break exact ties
    u = np.maximum.accumulate(u)
    ramp = np.arange(grid.size, dtype=float) * 1e-12
    u = np.minimum(u + ramp, 1.0 - eps)
    u = np.maximum.accumulate(u)
    u = np.clip(u, pi + 1e-12, 1.0 - eps)
    return Marginal(grid, u, pi, floor_code)
class StyleMarginals:

    def __init__(self, kappa: float, codes: dict,
    eps: float = 1e-9, drop_floor_atoms: bool = False,
                 min_style_obs: int = MIN_STYLE_OBS):
        self.kappa = float(kappa)
        self.codes = dict(codes)
        self.eps = float(eps)
        self.drop_floor_atoms = bool(drop_floor_atoms)
        self.min_style_obs = int(min_style_obs)
        self.styles = []
        self.marg = {}
        self.pool = {}
        self.state = []
        self.state_index = {}
        self.pool_state = {}
        self.summary = {}
        self.valid = None

    # -- fitting
    def fit(self, X: np.ndarray, styles, valid=None)
    -> "StyleMarginals":
        X = np.asarray(X, dtype=float)
        styles = np.asarray(styles)
        if valid is None:
            valid = np.isfinite(X)
        else:
            valid = np.asarray(valid, dtype=bool) &
    np.isfinite(X)
        self.valid = valid
        self.styles = sorted(set(str(s) for s in
```

```
styles))
    mask = floor_mask(X, self.codes) & valid
    # ---- pooled (training) pieces, one per
column
    pool_cont = {}
    pool_pi = {}
    pool_state = {}
    for j, name in enumerate(FEATURE_COLS):
        keep = valid[:, j] & ~mask[:, j]
        values = np.sort(X[keep, j])
        n_valid = int(valid[:, j].sum())
        n_cont = int(keep.sum())
        n_floor = 0 if self.drop_floor_atoms else
int(mask[:, j].sum())
        pi_pool = float(n_floor / max(n_valid, 1))
        code = None if self.drop_floor_atoms else
self.codes.get(name, None)
        pool_cont[name] = values
        pool_pi[name] = pi_pool
        self.pool[name] = build_marginal(values,
values, pi_pool, pi_pool, 1.0, code, self.eps)
        pool_state[name] = {"n_valid": n_valid,
"n_continuous": n_cont, "n_floor": n_floor,
                            "pi_floor": pi_pool,
"floor_code": code,

"continuous_denominator": n_cont,
"floor_mass_denominator": n_valid,
                            "status":
"training_pooled" if n_valid else "empty_column"}
    self.pool_state = pool_state
    # ---- per style pieces
    self.marg = {}
    self.state = []
    self.state_index = {}
    for style in self.styles:
        rows = np.nonzero(styles == style)[0]
        n_rows = int(rows.size)
        style_margs = []
        for j, name in enumerate(FEATURE_COLS):
            ok = valid[rows, j]
            hit = mask[rows, j] & ok
            n_valid = int(ok.sum())
            n_cont = int((ok & ~hit).sum())
            n_floor = 0 if self.drop_floor_atoms
else int(hit.sum())
            cont = np.sort(X[rows][ok & ~hit, j])
            pi_style = float(n_floor /
max(n_valid, 1))
            code = None if self.drop_floor_atoms
else self.codes.get(name, None)
            if n_valid <= 0:
                omega = 0.0
                status = "pooled_fallback:no_valid
_observation"
            elif n_valid < self.min_style_obs:
                omega = 0.0
                status = "pooled_fallback:n_valid_
lt_%d" % self.min_style_obs
            elif n_valid + self.kappa > 0:
                omega = float(n_valid / (n_valid
+ self.kappa))
                status = "style_empirical_shrunk"
            else:
                omega = 1.0
                status = "style_empirical"
            if n_cont == 0:
                status = status +
"+continuous_from_pool"
            style_cont = cont if omega > 0.0 else
np.zeros(0, dtype=float)
            style_margs.append(build_marginal(
style_cont, pool_cont[name], pi_style, pool_pi[name],

omega, code, self.eps))
            record = {"style": style, "column":
name, "n_style_rows": n_rows, "n_valid": n_valid,
                      "n_continuous": n_cont,
"n_floor": n_floor, "omega": float(omega),
                                "kappa": float(self.kappa),
"pi_style_floor": pi_style, "pi_pool_floor":
pool_pi[name],
                                "continuous_denominator":
n_cont, "floor_mass_denominator": n_valid,
                                "status": status}
            self.state.append(record)
            self.state_index[(style, name)] =
record
        self.marg[style] = style_margs
    self.summary = self._summarise()
    return self

def _summarise(self) -> dict:
    counts = {}
    for record in self.state:
        counts[record["status"]] =
counts.get(record["status"], 0) + 1
    fallback = sum(count for status, count in
counts.items() if status.startswith(
"pooled_fallback"))
    return {"nStyles": int(len(self.styles)),
"nColumns": int(len(FEATURE_COLS)),
            "nCells": int(len(self.state)),
"statusCounts": {k: int(v) for k, v in
sorted(counts.items())},
            "nPooledFallbackCells":
int(fallback), "nZeroContinuousCells":
                int(sum(1 for record in
self.state if record["n_continuous"] == 0)),
            "kappa": float(self.kappa),
"minStyleObs": int(self.min_style_obs),
            "empiricalSample": "valid (finite,
non-imputed) cells only; imputed values never enter
the marginal",
            "continuousDenominator":
"n_continuous = valid & not floor-coded cells of the
(style, column)",
            "floorMassDenominator": "n_valid =
all valid cells of the (style, column)"}

def state_table(self) -> list:
    return list(self.state)

# -- mapping
def _marginals(self, style):
    return self.marg.get(str(style), None)

def forward(self, style: str, X: np.ndarray) ->
np.ndarray:
    X = np.asarray(X, dtype=float)
    margs = self._marginals(style)
    U = np.empty_like(X)
    for j, name in enumerate(FEATURE_COLS):
        marginal = margs[j] if margs is not None
else self.pool[name]
        U[:, j] = marginal.forward(X[:, j])
    return U

def inverse(self, style: str, U: np.ndarray) ->
np.ndarray:
    U = np.asarray(U, dtype=float)
    margs = self._marginals(style)
    X = np.empty_like(U)
    for j, name in enumerate(FEATURE_COLS):
        marginal = margs[j] if margs is not None
else self.pool[name]
        X[:, j] = marginal.inverse(U[:, j])
    return X

class GaussianCopula:

def __init__(self, kappa_R: float, shrink: float
= 0.05, eig_floor: float = 1e-6):
    self.kappa_R = float(kappa_R)
    self.shrink = float(shrink)
    self.eig_floor = float(eig_floor)
    self.shared = None
    self.factor = {}
    self.corr = {}
```

```
@staticmethod
def _correlation(Z):
    Z = np.asarray(Z, dtype=float)
    n = Z.shape[0]
    if n < 3:
        return np.eye(Z.shape[1])
    centre = Z - Z.mean(axis=0, keepdims=True)
    scale = centre.std(axis=0)
    scale = np.where(scale > 1e-12, scale, 1.0)
    W = centre / scale
    return (W.T @ W) / float(n - 1)

def _spd(self, R):
    R = np.asarray(R, dtype=float)
    R = 0.5 * (R + R.T)
    values, vectors = np.linalg.eigh(R)
    values = np.maximum(values, self.eig_floor)
    R = (vectors * values) @ vectors.T
    diag = np.sqrt(np.clip(np.diag(R), 1e-18,
None))
    R = R / np.outer(diag, diag)
    R = (1.0 - self.shrink) * R + self.shrink *
np.eye(R.shape[0])
    values, vectors = np.linalg.eigh(R)
    values = np.maximum(values, self.eig_floor)
    R = (vectors * values) @ vectors.T
    diag = np.sqrt(np.clip(np.diag(R), 1e-18,
None))
    return R / np.outer(diag, diag)

@staticmethod
def _cholesky(R):
    try:
        return np.linalg.cholesky(R)
    except np.linalg.LinAlgError:
        for jitter in (1e-10, 1e-8, 1e-6, 1e-4,
1e-2):
            try:
                return np.linalg.cholesky(R +
jitter * np.eye(R.shape[0]))
            except np.linalg.LinAlgError:
                continue
    raise RuntimeError("copula correlation matrix
is not positive definite even after jitter")

def fit(self, Z: np.ndarray, styles) ->
"GaussianCopula":
    Z = np.asarray(Z, dtype=float)
    styles = np.asarray(styles)
    shared = self._spd(self._correlation(Z))
    self.shared = shared
    self.factor = {}
    for style in sorted(set(str(s) for s in
styles)):
        rows = np.nonzero(styles == style)[0]
        n_s = rows.size
        eta = n_s / (n_s + self.kappa_R) if n_s +
self.kappa_R > 0 else 1.0
        empirical = self._correlation(Z[rows]) if
n_s >= 4 else shared
        R = eta * empirical + (1.0 - eta) * shared
        R = self._spd(R)
        self.corr[str(style)] = R
        self.factor[str(style)] =
self._cholesky(R)
    return self

def uniforms_from_normal(self, style: str, z:
np.ndarray) -> np.ndarray:
    factor = self.factor.get(str(style),
self._cholesky(self.shared))
    return ndtr(np.asarray(z, dtype=float) @
factor.T)

def uniforms(self, style: str, n: int, rng) ->
np.ndarray:
    z = rng.standard_normal((int(n),
self.shared.shape[0]))
    return self.uniforms_from_normal(style, z)
def sample_from_uniforms(marginals: StyleMarginals,
    style: str, U: np.ndarray) -> np.ndarray:
    return marginals.inverse(style, U)
def mean_pairwise_distance(A: np.ndarray) -> float:
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    if n < 2:
        return 0.0
    D = cdist(A, A, metric="euclidean")
    return float(D.sum() / (n * (n - 1)))
def energy_distance_sq(gen: np.ndarray, real:
    np.ndarray, within_fallback: float = None) -> float:
    gen = np.asarray(gen, dtype=float)
    real = np.asarray(real, dtype=float)
    if gen.shape[0] == 0 or real.shape[0] == 0:
        return 0.0
    cross = float(cdist(gen, real,
    metric="euclidean").mean())
    within_gen = mean_pairwise_distance(gen)
    within_real = mean_pairwise_distance(real)
    if within_fallback is not None:
        if gen.shape[0] < 2:
            within_gen = float(within_fallback)
        if real.shape[0] < 2:
            within_real = float(within_fallback)
    return float(2.0 * cross - within_gen -
    within_real)
def energy_distance_terms(gen: np.ndarray, real:
    np.ndarray) -> dict:
    gen = np.asarray(gen, dtype=float)
    real = np.asarray(real, dtype=float)
    m, n = int(gen.shape[0]), int(real.shape[0])
    if m == 0 or n == 0:
        return {"V": None, "U": None, "cross": None,
    "within_gen": None, "within_real": None,
                "sum_gen": None, "sum_real": None,
    "m": m, "n": n}
    cross = float(cdist(gen, real,
    metric="euclidean").mean())
    aa = float(cdist(gen, gen, metric="euclidean").sum
    ())
    bb = float(cdist(real, real,
    metric="euclidean").sum())
    V = 2.0 * cross - aa / (m * m) - bb / (n * n)
    within_gen = aa / (m * (m - 1)) if m > 1 else None
    within_real = bb / (n * (n - 1)) if n > 1 else
    None
    U = None if (within_gen is None or within_real is
    None) else 2.0 * cross - within_gen - within_real
    return {"V": float(V), "U": None if U is None
    else float(U), "cross": cross,
            "within_gen": within_gen, "within_real":
    within_real, "sum_gen": aa, "sum_real": bb, "m": m,
    "n": n}
def style_dtrain(Z: np.ndarray, styles, style) ->
    float:
    Z = np.asarray(Z, dtype=float)
    styles = np.asarray(styles)
    rows = np.nonzero(styles == str(style))[0]
    if rows.size >= 2:
        block = Z[rows]
        return float(cdist(block, block,
    metric="euclidean").sum() / (rows.size * (rows.size -
    1)))
    n = Z.shape[0]
    return float(cdist(Z, Z, metric="euclidean").sum(
    ) / (n * (n - 1))) if n >= 2 else 1.0
def normalized_energy_distance(gen, real, dref) ->
    tuple:
    e2 = energy_distance_sq(gen, real,
    within_fallback=dref)
    dref = float(dref)
    if not np.isfinite(dref) or dref <= 1e-12:
        return 0.0, e2
    return float(np.sqrt(max(e2, 0.0) / dref)), e2
class StyleModel:

    def __init__(self, centre, scale, medians,
```

```
    marginals, copula, proxy, codes, t2n_bounds,
    global_t2n,
                 stale_bounds=None,
    global_stale=None):
        self.centre = centre
        self.scale = scale
        self.medians = medians
        self.marginals = marginals
        self.copula = copula
        self.proxy = proxy
        self.codes = codes
        self.t2n_bounds = dict(t2n_bounds)
        self.global_t2n = float(global_t2n)
        self.stale_bounds = dict(stale_bounds) if
    stale_bounds else None
        self.global_stale = None if global_stale is
    None else float(global_stale)

    def t2n_bound(self, styles):
        styles = np.asarray(styles)
        return np.array([self.t2n_bounds.get(str(s),
    self.global_t2n) for s in styles], dtype=float)

    def stale_bound(self, styles):
        styles = np.asarray(styles)
        if self.stale_bounds is None or
    self.global_stale is None:
            return np.full(len(styles),
    np.sqrt(np.finfo(float).max), dtype=float)
        return np.array([self.stale_bounds.get(str(s),
    self.global_stale) for s in styles], dtype=float)

    def generate(self, style: str, n: int, rng, mode:
    str = "copula", marginals=None, copula=None,
                 normal: np.ndarray = None):
        marginals = marginals if marginals is not
    None else self.marginals
        copula = copula if copula is not None else
    self.copula
        if mode == "independent":
            U = rng.random((int(n),
    len(FEATURE_COLS)))
        else:
            if normal is None:
                U = copula.uniforms(style, int(n),
    rng)
            else:
                U = copula.uniforms_from_normal(style,
    normal)
        return sample_from_uniforms(marginals, style,
    U)
def fit_style_model(X_train, styles_train, Y_train,
    kappa, kappa_R, codes, t2n_bounds, global_t2n,
                    centre=None, scale=None,
    proxy=False, floor_aware=True, valid=None):
    medians = column_medians(X_train)
    X = apply_median(X_train, medians)
    if centre is None or scale is None:
        centre, scale = fit_standardizer(X)
    used_codes = dict(codes)
    marginals = StyleMarginals(kappa, used_codes,
    drop_floor_atoms=not floor_aware).fit(
        X, styles_train, valid=valid)
    Z = np.empty_like(X)
    for style in sorted(set(str(s) for s in
    styles_train)):
        rows = np.nonzero(styles_train == style)[0]
        Z[rows] = marginals.forward(style, X[rows])
    Z = np.clip(ndtri(np.clip(Z, 1e-9, 1 - 1e-9)),
    -8.0, 8.0)
    copula = GaussianCopula(kappa_R).fit(Z,
    styles_train)
    label_proxy = None
    stale_bounds = None
    global_stale = None
    if proxy:
        label_proxy = LabelProxy(centre,
    scale).fit(X, Y_train)
        predicted_stale = label_proxy.predict_storage(
    X, STALE)
        stale_bounds = {}
        for style in sorted(set(str(s) for s in
    styles_train)):
            rows = np.nonzero(styles_train ==
    style)[0]
            stale_bounds[style] =
    float(np.quantile(predicted_stale[rows], 0.90))
        global_stale = float(np.quantile(
    predicted_stale, 0.90))
    model = StyleModel(centre, scale, medians,
    marginals, copula, label_proxy, used_codes,
                       t2n_bounds, global_t2n,
    stale_bounds, global_stale)
    return model, label_proxy
def select_hyperparameters(X, styles, kappas=(0.0,
    5.0, 15.0), kappa_Rs=(20.0, 50.0, 100.0),
                           codes=None, rng_seed=0,
    max_cells=6):
    styles = np.asarray(styles)
    half = _half_split(styles)
    rng = np.random.default_rng(int(rng_seed))
    scores = {(float(k), float(r)): [] for k in
    kappas for r in kappa_Rs}
    for side in (0, 1):
        train = half != side
        hold = ~train
        Xtr = apply_median(X[train],
    column_medians(X[train]))
        Xho = apply_median(X[hold],
    column_medians(X[train]))
        centre, scale = fit_standardizer(Xtr)
        Ztr = standardize(Xtr, centre, scale)
        Zho = standardize(Xho, centre, scale)
        s_tr, s_ho = styles[train], styles[hold]
        used_codes = codes if codes is not None else
    floor_code_map(Xtr)
        for kappa in kappas:
            marginals = StyleMarginals(kappa,
    used_codes).fit(Xtr, s_tr)
            # the copula only depends on kappa
    through z, so one z-matrix serves every kappa_R
            Zc = np.empty_like(Ztr)
            for style in sorted(set(str(s) for s in
    s_tr)):
                rows = np.nonzero(s_tr == style)[0]
                U = marginals.forward(style,
    Xtr[rows])
                Zc[rows] = np.clip(ndtri(np.clip(U,
    1e-9, 1 - 1e-9)), -8.0, 8.0)
            for kappa_R in kappa_Rs:
                copula = GaussianCopula(kappa_R).fit(
    Zc, s_tr)
                num = 0.0
                den = 0
                for style in sorted(set(str(s) for s
    in s_ho)):
                    rows = np.nonzero(s_ho ==
    style)[0]
                    n = rows.size
                    U = copula.uniforms(style, n, rng)
                    gen = marginals.inverse(style, U)
                    gen_z = standardize(gen, centre,
    scale)
                    real_z = Zho[rows]
                    tr_rows = np.nonzero(s_tr ==
    style)[0]
                    dref = reference_scale(
    Ztr[tr_rows], n, rng, repeats=max_cells)
                    value, _ = normalized_energy_dista
    nce(gen_z, real_z, dref)
                    num += value * n
                    den += n
                scores[(float(kappa),
    float(kappa_R))].append(num / max(den, 1))
    table = {key: float(np.mean(values)) for key,
    values in scores.items()}
    # ties break towards the stronger-shrinkage grid
    point (larger kappa, then larger kappa_R)
```

```
    best_key = min(table, key=lambda key:
    (table[key], -key[0], -key[1]))
    return best_key[0], best_key[1], table
class TransferProblem:

    def __init__(self, X_train, styles_train,
    Y_train, source_rows, source_style, target_style,
                 kappa, kappa_R, proxy, codes,
    thresholds, n_components: int = 10):
        X_train = np.asarray(X_train, dtype=float)
        styles_train = np.asarray(styles_train)
        self.codes = dict(codes)
        self.target_style = str(target_style)
        self.source_style = str(source_style)
        self.thresholds = dict(thresholds)
        self.medians = column_medians(X_train)
        self.X_train = apply_median(X_train,
    self.medians)
        self.centre, self.scale =
    fit_standardizer(self.X_train)
        lo = np.nanmin(self.X_train, axis=0)
        hi = np.nanmax(self.X_train, axis=0)
        span = hi - lo
        span = np.where(np.isfinite(span) & (span >
    1e-12), span, 1e-12)
        self.lo = np.where(np.isfinite(lo), lo, 0.0)
        self.hi = self.lo + span
        self.span = span
        self.v_upper = np.ones(len(FEATURE_COLS))
        self.marginals = StyleMarginals(kappa,
    self.codes).fit(self.X_train, styles_train)
        source_X = apply_median(np.asarray(
    source_rows, dtype=float), self.medians)
        self.X_source = source_X
        source_styles = np.array([self.source_style]
    * source_X.shape[0])
        self.source_marginals = StyleMarginals(0.0,
    self.codes).fit(source_X, source_styles)
        self.x0 = self.marginals.inverse(
    self.target_style,

    self.source_marginals.forward(self.source_style,
    source_X))
        self.v0 = np.clip((self.x0 - self.lo) /
    self.span, 0.0, 1.0)
        self.proxy = proxy
        self.source_phys = {name: physical_conc(name,
    source_X[:, FEATURE_COLS.index(name)], self.codes)
                            for name in DEFECT_KEYS}
        self.source_pred = {name:
    proxy.predict_storage(source_X, name) for name in
    PROXY_TARGETS}
        label_sd = float(np.nanstd(Y_train[:,
    LABEL_COLS.index(STALE)]))
        self.stale_tolerance = 0.05 * label_sd
        self.label_sd = {name: float(np.nanstd(
    Y_train[:, LABEL_COLS.index(name)])) for name in
    PROXY_TARGETS}
        # target PCA on the standardised target-style
    training rows
        target_rows = np.nonzero(styles_train ==
    self.target_style)[0]
        Zt = standardize(self.X_train[target_rows],
    self.centre, self.scale)
        self.z_mean = Zt.mean(axis=0)
        Zc = Zt - self.z_mean
        _, _, vt = np.linalg.svd(Zc,
    full_matrices=False)
        n_comp = int(min(n_components, Zc.shape[0] -
    1, Zc.shape[1]))
        self.pca = vt[:n_comp]
        scores = Zc @ self.pca.T
        self.score_mean = scores.mean(axis=0)
        score_sd = scores.std(axis=0, ddof=1)
        self.score_sd = np.where(score_sd > 1e-12,
    score_sd, 1.0)
        corr = np.corrcoef((scores - self.score_mean)
    / self.score_sd, rowvar=False)
        corr = np.atleast_2d(np.nan_to_num(corr,
    nan=0.0))
        corr = 0.9 * corr + 0.1 *
    np.eye(corr.shape[0])
        self.target_cov = corr
        self.target_cov_inv = np.linalg.inv(corr)
        # gradients: z = (lo + span*v -
    centre)/scale, so dz/dv = span/scale
        self.dz_dv = self.span / self.scale
        self.d_score_dv = (self.pca *
    self.dz_dv[None, :]) / self.score_sd[:, None]
        # proxy gradients w.r.t. v (used by the exact
    shifts and by the L1 change penalty)
        self.proxy_grad = {}
        for name in PROXY_TARGETS:
            self.proxy_grad[name] =
    self.proxy.coef_for(name) * self.dz_dv
        self.source_defect_profile =
    self.defect_profile(source_X)

    # -- coordinate changes
    def v_to_x(self, v):
        return self.lo + self.span * np.asarray(v,
    dtype=float)

    def x_to_v(self, x):
        return np.clip((np.asarray(x, dtype=float) -
    self.lo) / self.span, 0.0, 1.0)

    def score(self, v):
        v = np.asarray(v, dtype=float)
        Z = (self.v_to_x(v) - self.centre) /
    self.scale
        return ((Z @ self.pca.T) - self.score_mean) /
    self.score_sd

    def target_mahalanobis(self, v):
        s = self.score(v)
        return float(np.sqrt(max(float(s @
    self.target_cov_inv @ s), 0.0)))

    def target_objective(self, v):
        s = self.score(v)
        return float(s @ self.target_cov_inv @ s)

    def predicts(self, v, name):
        x = self.v_to_x(v)
        return float(self.proxy.predict_storage(
    x[None, :], name)[0])

    def defect_profile(self, x):
        x = np.asarray(x, dtype=float)
        columns = []
        for name in DEFECT_KEYS:
            columns.append(physical_conc(name, x[:,
    FEATURE_COLS.index(name)], self.codes)
                           / float(
    self.thresholds[name]))
        for name in PROXY_TARGETS:
            columns.append(self.proxy.predict_storage(
    x, name) / self.label_sd[name])
        return np.stack(columns, axis=1)

    def defect_gradient(self, index: int) ->
    np.ndarray:
        grad = np.zeros((5, len(FEATURE_COLS)))
        for k, name in enumerate(DEFECT_KEYS):
            j = FEATURE_COLS.index(name)
            conc = float(self.source_phys[name][index]
    )
            factor = (np.log(10.0) * conc) if name in
    LOG10_SET else 1.0
            grad[k, j] = factor * self.span[j] /
    float(self.thresholds[name])
        for k, name in enumerate(PROXY_TARGETS):
            grad[len(DEFECT_KEYS) + k, :] =
    self.proxy_grad[name] / self.label_sd[name]
        return grad

    # -- exact linear shifts
```

```
def shift_along_proxy(self, v, name, caps,
excluded=tuple(DEFECT_KEYS)):
    v = np.array(v, dtype=float, copy=True)
    caps = np.broadcast_to(np.asarray(caps,
dtype=float), (1,)).copy()
    grad = np.array(self.proxy_grad[name],
dtype=float)
    mask = np.ones(len(FEATURE_COLS), dtype=bool)
    for column in excluded:
        mask[FEATURE_COLS.index(column)] = False
    grad = np.where(mask, grad, 0.0)
    norm_sq = float(np.sum(grad * grad))
    if norm_sq <= 1e-18:
        return v
    Z = (self.v_to_x(v) - self.centre) /
self.scale
    pred = float(self.proxy.coef_for(name) @ Z +
self.proxy.intercept_for(name))
    gap = pred - (float(caps[0]) - PROXY_MARGIN)
    if gap > 0.0:
        v = v - (gap / norm_sq) * grad
    return np.clip(v, 0.0, 1.0)

def proxy_ok(self, v, t2n_caps, stale_caps, tol:
float = 1e-7):
    pred_t2n = self.predicts(v, T2N)
    pred_stale = self.predicts(v, STALE)
    return (pred_t2n <= float(t2n_caps[0]) + tol)
and (pred_stale <= float(stale_caps[0]) + 1e-7)

# -- optimisation
def solve(self, index: int, lambda_: float, eta:
float, caps: dict, t2n_cap=None, stale_cap=None,
          maxiter: int = 150):
    v0 = self.v0[index]
    if t2n_cap is None:
        t2n_cap = float(self.source_pred[T2N][inde
x])
    if stale_cap is None:
        stale_cap = float(self.source_pred[STALE][
index]) + self.stale_tolerance
    t2n_caps = np.array([t2n_cap], dtype=float)
    stale_caps = np.array([stale_cap],
dtype=float)
    upper = apply_transfer_caps(self, caps)
    self.v_upper = upper
    defect_grad = self.defect_gradient(index)
    source_profile = self.source_defect_profile[in
dex]

    def objective(v):
        v = np.clip(v, 0.0, upper)
        v = project_transfer_feasible(self, v,
t2n_caps, stale_caps)
        v = np.clip(v, 0.0, upper)
        diff = v - v0
        f_quad = float(diff @ diff)
        s = self.score(v)
        grad_target = self.target_cov_inv @ s
        f_target = float(s @ grad_target)
        profile = self.defect_profile(self.v_to_x(
v)[None, :])[0]
        delta = profile - source_profile
        f_l1 = float(np.sum(np.abs(delta)))
        value = f_quad + float(lambda_) *
f_target + float(eta) * f_l1
        grad = 2.0 * diff
        grad = grad + float(lambda_) * (2.0 *
(self.d_score_dv.T @ grad_target))
        if eta:
            signs = np.sign(delta)
            grad = grad + float(eta) * (signs @
defect_grad)
        return value, grad

    start = project_transfer_feasible(self, v0,
t2n_caps, stale_caps)
    start = np.clip(start, 0.0, upper)
    result = minimize(objective, start, jac=True,
method="L-BFGS-B",
                      bounds=list(zip(
np.zeros_like(upper), upper)),
                      options={"maxiter":
int(maxiter), "ftol": 1e-12, "gtol": 1e-8, "maxls":
30})
    v = project_transfer_feasible(self,
np.clip(result.x, 0.0, upper), t2n_caps, stale_caps)
    v = np.clip(v, 0.0, upper)
    x = self.v_to_x(v)
    pred_t2n = self.predicts(v, T2N)
    pred_stale = self.predicts(v, STALE)
    nonworsen = {}
    for name in DEFECT_KEYS:
        out_conc = float(physical_conc(name,
x[FEATURE_COLS.index(name)], self.codes))
        src_conc = float(self.source_phys[name][in
dex])
        nonworsen[name] = bool(out_conc <=
src_conc * (1.0 + 1e-9) + 1e-12)
    nonworsen[T2N] = bool(pred_t2n <=
float(self.source_pred[T2N][index]) + 1e-9)
    nonworsen[STALE] = bool(pred_stale <=
float(self.source_pred[STALE][index]) +
self.stale_tolerance + 1e-9)
    return {
        "v": v,
        "x": x,
        "targetMahalanobis":
self.target_mahalanobis(v),
        "targetObjective": self.target_objective(
v),
        "l1Change": float(np.sum(np.abs(v - v0))),
        "meanAbsChange": float(np.mean(np.abs(v -
v0))),
        "quadratic": float((v - v0) @ (v - v0)),
        "feasible": bool(self.proxy_ok(v,
t2n_caps, stale_caps, tol=1e-6)),
        "nonworsen": nonworsen,
        "nonworsenAll": bool(all(nonworsen.values(
))),
        "iterations": int(getattr(result, "nit",
0)),
        "converged": bool(getattr(result,
"success", False)),
    }
CAP_AT_MAX_SOURCE_THRESHOLD = "cap-at-max-source-thres
NON_INCREASING = "non-increasing"
```

C.4 问题三·生成：折叠上下文、约束投影、质量与评价统计（src/q3_fix.py）

```
# AI-ASSISTANCE NOTICE: the framework of this module
    was drafted with DeepSeek Harness assistance;
# the core algorithm logic, constraint corrections
    and experimental parameters were reviewed, amended
# and independently verified by the team members, who
    take responsibility for the contents.

_HERE = Path(__file__).resolve().parent
_ROOT = _HERE.parent
NONENAL = "X.E..2.nonenal"
KAPPAS = (0.0, 5.0, 15.0)
KAPPA_RS = (20.0, 50.0, 100.0)
THRESHOLDS = dict(L.THRESHOLD_MEAN)          #
    declared mean sensory thresholds (physical units)
REFERENCE_DRAWS = 20
DIVERSITY_DRAWS = 100
PROXY_EXCLUDED = tuple(L.DEFECT_KEYS) + (NONENAL,)  #
    a proxy shift must never undo a chemistry cap
COVERAGE_TOL = 1e-9
STALE_PROXY_MARGIN = 1e-9
LINEAR_MASK = np.array([name in L.LINEAR_SET for name
    in L.FEATURE_COLS])
LOG10_MASK = np.array([name in L.LOG10_SET for name
```

```
    in L.FEATURE_COLS])
_RUN_DIR = run_dir()
_OUTPUTS = {}
def load_data():
    frame = pd.read_csv(V6.DATA_Q3)
    for column in ("beer", "tasting_category_fine",
    "row"):
        if column not in frame.columns:
            raise RuntimeError("data/clean/q3.csv
    lacks the %r column" % column)
    if not frame["beer"].is_unique:
        raise RuntimeError("'beer' is not unique: row
    identity would be a guess")
    chem = [c for c in frame.columns if
    c.startswith(V6.CHEM_PREFIX)]
    if chem != L.CHEM_COLS:
        raise RuntimeError("the chem__* column order
    of data/clean/q3.csv differs from the frozen
    FEATURE_COLS")
    X = frame[L.CHEM_COLS].to_numpy(dtype=float)
    Y = frame[L.LABEL_COLS].to_numpy(dtype=float)
    if np.isnan(Y).any():
        raise RuntimeError("the 50 sensory labels
    contain NaN: the profile proxy would silently
    propagate them")
    styles = frame["tasting_category_fine"].astype(
    str).to_numpy()
    beers = frame["beer"].to_numpy(dtype=int)
    return frame, X, Y, styles, beers
def fit_fold_context(X_raw, Y, styles, train_idx,
    repeat, fold, kappa, kappa_R, with_copulas=True):
    raw_train = np.asarray(X_raw[train_idx],
    dtype=float)
    medians = L.column_medians(raw_train)
    X_train = L.apply_median(raw_train, medians)
    valid_train = np.isfinite(raw_train)          #
    raw-observation mask: imputed cells are NOT
    observations
    centre, scale = L.fit_standardizer(X_train)
    codes = L.floor_code_map(X_train)
    styles_train = styles[train_idx]
    marginals = L.StyleMarginals(kappa,
    codes).fit(X_train, styles_train, valid=valid_train)
    proxy = L.LabelProxy(centre, scale).fit(X_train,
    Y[train_idx])
    # rank-gauss transform of the training rows,
    shared by every copula fit of this fold
    Z = np.empty_like(X_train)
    for style in sorted(set(styles_train.tolist())):
        rows = np.nonzero(styles_train == style)[0]
        U = marginals.forward(style, X_train[rows])
        Z[rows] = np.clip(ndtri(np.clip(U, 1e-9, 1.0
    - 1e-9)), -8.0, 8.0)
    copulas = {}
    if with_copulas:
        for label, value in (("selected",
    float(kappa_R)), ("shared-only", 1e9),
    ("style-empirical", 0.0)):
            copulas[label] = L.GaussianCopula(
    value).fit(Z, styles_train)
    predicted_stale = proxy.predict_storage(X_train,
    L.STALE)
    predicted_t2n = proxy.predict_storage(X_train,
    L.T2N)
    stale_bounds, t2n_bounds, nonenal_bounds = {},
    {}, {}
    nonenal_index = L.FEATURE_COLS.index(NONENAL)
    for style in sorted(set(styles_train.tolist())):
        rows = np.nonzero(styles_train == style)[0]
        stale_bounds[style] = float(np.quantile(
    predicted_stale[rows], 0.90))
        t2n_bounds[style] = float(np.quantile(
    predicted_t2n[rows], 0.90))
        nonenal_bounds[style] =
    float(np.quantile(X_train[rows, nonenal_index], 0.90))
    pooled = {
        "stale": float(np.quantile(predicted_stale,
    0.90)),
        "t2n": float(np.quantile(predicted_t2n,
    0.90)),
        "nonenal": float(np.quantile(X_train[:,
    nonenal_index], 0.90)),
    }
    train_lo = np.nanmin(X_train, axis=0)
    train_hi = np.nanmax(X_train, axis=0)
    for j in np.nonzero(LINEAR_MASK)[0]:
        train_lo[j] = max(train_lo[j], 0.0)
    return {
        "repeat": int(repeat), "fold": int(fold),
    "train_idx": np.asarray(train_idx, dtype=int),
        "medians": medians, "X_train": X_train,
    "valid_train": valid_train,
        "centre": centre, "scale": scale, "codes":
    codes,
        "styles_train": styles_train, "marginals":
    marginals, "marginal_state": marginals.state_table(),
        "marginal_summary": marginals.summary,
    "proxy": proxy, "Z_train": Z,
        "Z_std_train": L.standardize(X_train, centre,
    scale),
        "copulas": copulas, "kappa": float(kappa),
    "kappa_R": float(kappa_R),
        "stale_bounds": stale_bounds, "t2n_bounds":
    t2n_bounds, "nonenal_bounds": nonenal_bounds,
        "global_stale": pooled["stale"],
    "global_t2n": pooled["t2n"], "global_nonenal":
    pooled["nonenal"],
        "train_lo": train_lo, "train_hi": train_hi,
    }
def generation_draws(ctx, style: str, n: int, rng:
    np.random.Generator) -> dict:
    z0 = rng.standard_normal((n, len(L.FEATURE_COLS)))
    out = {}
    for label, copula in (("copula-selected",
    ctx["copulas"]["selected"]),
                          ("copula-shared-only",
    ctx["copulas"]["shared-only"]),
                          ("copula-style-empirical",
    ctx["copulas"]["style-empirical"])):
        uniforms = copula.uniforms_from_normal(style,
    z0)
        out[label] = L.sample_from_uniforms(
    ctx["marginals"], style, uniforms)
    uniforms = rng.random((n, len(L.FEATURE_COLS)))
    # no dependence structure at all
    out["independent-marginals"] =
    L.sample_from_uniforms(ctx["marginals"], style,
    uniforms)
    train_rows = np.nonzero(ctx["styles_train"] ==
    style)[0]
    if train_rows.size:
        pick = rng.choice(train_rows, size=n,
    replace=n > train_rows.size)
        out["source-resample"] =
    np.array(ctx["X_train"][pick], dtype=float, copy=True)
    else:
        pick = rng.choice(ctx["X_train"].shape[0],
    size=n, replace=True)
        out["source-resample"] =
    np.array(ctx["X_train"][pick], dtype=float, copy=True)
    return out
def project_design(raw: np.ndarray, ctx: dict, style:
    str, max_rounds: int = 12) -> dict:
    X = np.array(raw, dtype=float, copy=True)
    defect_caps = []
    for name in L.DEFECT_KEYS:
        j = L.FEATURE_COLS.index(name)
        cap = float(np.asarray(L.storage_bound(name,
    THRESHOLDS[name])))
        defect_caps.append((j, cap))
    nonenal_cap = style_nonenal_bound(ctx, style)
    nonenal_index = L.FEATURE_COLS.index(NONENAL)
    raw_nonenal = np.array(X[:, nonenal_index],
    dtype=float, copy=True)
    stale_cap = style_stale_bound(ctx, style)
    l2 = LINEAR_MASK
    rounds = 0
    for rounds in range(1, int(max_rounds) + 1):
```

```
        for j, cap in defect_caps:
            X[:, j] = np.minimum(X[:, j], cap)
        X[:, nonenal_index] = np.minimum(np.minimum(
    X[:, nonenal_index], nonenal_cap), raw_nonenal)
        if ctx["proxy"] is not None:
            X = L.project_proxy_bound(X,
    ctx["proxy"], L.STALE, np.full(X.shape[0], stale_cap),
                                      ctx["centre"],
    ctx["scale"], excluded_columns=PROXY_EXCLUDED,
                                      max_rounds=1)
        X[:, l2] = np.maximum(X[:, l2], 0.0)
        violations = constraint_violations(X, ctx,
    style)
        if violations["max"] <= COVERAGE_TOL:
            break
    return {"design": X, "rounds": int(rounds),
    "violations": constraint_violations(X, ctx, style)}
def constraint_violations(X: np.ndarray, ctx: dict,
    style: str) -> dict:
    out = {}
    worst = 0.0
    for name in L.DEFECT_KEYS:
        j = L.FEATURE_COLS.index(name)
        cap = float(np.asarray(L.storage_bound(name,
    THRESHOLDS[name])))
        violation = np.maximum(X[:, j] - cap, 0.0)
        out[name] = float(violation.max()) if
    violation.size else 0.0
        worst = max(worst, out[name])
    j = L.FEATURE_COLS.index(NONENAL)
    violation = np.maximum(X[:, j] -
    style_nonenal_bound(ctx, style), 0.0)
    out[NONENAL] = float(violation.max()) if
    violation.size else 0.0
    worst = max(worst, out[NONENAL])
    predicted = ctx["proxy"].predict_storage(X,
    L.STALE)
    violation = np.maximum(predicted -
    style_stale_bound(ctx, style), 0.0)
    out[L.STALE] = float(violation.max()) if
    violation.size else 0.0
    worst = max(worst, out[L.STALE])
    negative = np.maximum(-X[:, LINEAR_MASK], 0.0)
    out["linear-nonneg"] = float(negative.max()) if
    negative.size else 0.0
    worst = max(worst, out["linear-nonneg"])
    out["max"] = float(worst)
    return out
def energy_statistics(gen_z: np.ndarray, real_z:
    np.ndarray, dref: float) -> dict:
    terms = L.energy_distance_terms(gen_z, real_z)
    if terms["V"] is None:
        return {"V": None, "U": None, "dref":
    float(dref), "cross": None}
    gen_pair = terms["within_gen"] if
    terms["within_gen"] is not None else float(dref)
    real_pair = terms["within_real"] if
    terms["within_real"] is not None else float(dref)
    return {"V": float(terms["V"]), "U": float(2.0 *
    terms["cross"] - gen_pair - real_pair),
            "dref": float(dref), "cross":
    float(terms["cross"]),
            "within_gen": float(gen_pair),
    "within_real": float(real_pair),
            "m": int(terms["m"]), "n":
    int(terms["n"])}
def marginal_error(gen_z: np.ndarray, real_z:
    np.ndarray) -> float | None:
    if gen_z.shape[0] == 0 or real_z.shape[0] == 0:
        return None
    grid = np.array([0.1, 0.2, 0.3, 0.4, 0.5, 0.6,
    0.7, 0.8, 0.9])
    qg = np.quantile(gen_z, grid, axis=0)
    qr = np.quantile(real_z, grid, axis=0)
    return float(np.mean(np.abs(qg - qr)))
def dependence_error(gen_z: np.ndarray, real_z:
    np.ndarray) -> float | None:
    if gen_z.shape[0] < 3 or real_z.shape[0] < 3:
        return None
    cg = np.corrcoef(gen_z, rowvar=False)
    cr = np.corrcoef(real_z, rowvar=False)
    mask = ~np.eye(cg.shape[0], dtype=bool)
    value = np.abs(cg - cr)[mask].mean()
    return float(value) if np.isfinite(value) else
    None
def diversity_statistics(gen_z: np.ndarray, rng:
    np.random.Generator) -> dict:
    m = gen_z.shape[0]
    if m < 2:
        return {"diversity": None,
    "nearest_neighbor": None}
    take = np.arange(m) if m <= DIVERSITY_DRAWS else
    np.sort(rng.choice(m, size=DIVERSITY_DRAWS,
    replace=False))
    block = gen_z[take]
    distance = cdist(block, block, metric="euclidean")
    size = block.shape[0]
    diversity = float(distance.sum() / (size * (size
    - 1)))
    np.fill_diagonal(distance, np.inf)
    nearest = float(distance.min(axis=1).mean())
    return {"diversity": diversity,
    "nearest_neighbor": nearest}
def duplicate_share(block: np.ndarray) -> float:
    if block.shape[0] < 2:
        return 0.0
    unique = np.unique(block, axis=0)
    return float(1.0 - unique.shape[0] /
    block.shape[0])
def edge_fraction(block: np.ndarray, ctx: dict) ->
    float:
    hits = np.zeros(block.shape, dtype=bool)
    for j, name in enumerate(L.FEATURE_COLS):
        if name in ctx["codes"]:
            hits[:, j] |= np.isclose(block[:, j],
    ctx["codes"][name], rtol=1e-9, atol=0.0)
        hits[:, j] |= np.isclose(block[:, j],
    ctx["train_lo"][j], rtol=1e-9, atol=1e-12)
        hits[:, j] |= np.isclose(block[:, j],
    ctx["train_hi"][j], rtol=1e-9, atol=1e-12)
        if LINEAR_MASK[j]:
            hits[:, j] |= np.isclose(block[:, j],
    0.0, rtol=0.0, atol=1e-12)
    return float(hits.mean())
def spectrum_mae(gen_z: np.ndarray, real_z:
    np.ndarray, components: int = 20) -> float | None:
    if gen_z.shape[0] < 4 or real_z.shape[0] < 4:
        return None
    cg = np.corrcoef(gen_z, rowvar=False)
    cr = np.corrcoef(real_z, rowvar=False)
    k = int(min(components, cg.shape[0]))
    eg = np.linalg.eigvalsh(cg)[::-1][:k] /
    float(np.trace(cg))
    er = np.linalg.eigvalsh(cr)[::-1][:k] /
    float(np.trace(cr))
    value = float(np.mean(np.abs(eg - er)))
    return value if np.isfinite(value) else None
def _style_dtrain(ctx, style: str) -> float:
    return L.style_dtrain(ctx["Z_std_train"],
    ctx["styles_train"], style)
def select_hyperparameters_inner(X_raw, styles,
    train_idx, beer_index, splits, repeat, fold, seed):
    tables, diagnostics, inner_sizes = [], [], []
    for inner, inner_train, inner_valid in
    V6.iter_inner(repeat, fold, beer_index["train_beers"],
    splits):
        itr = np.array([beer_index["index_of"][int(
    b)] for b in inner_train], dtype=int)
        iva = np.array([beer_index["index_of"][int(
    b)] for b in inner_valid], dtype=int)
        raw_tr = np.asarray(X_raw[itr], dtype=float)
        valid_tr = np.isfinite(raw_tr)        #
    imputed cells must not enter the inner empirical
    marginals
        medians = L.column_medians(raw_tr)
        Xtr = L.apply_median(raw_tr, medians)
        Xva = L.apply_median(raw_va :=
    np.asarray(X_raw[iva], dtype=float), medians)
```

```
    centre, scale = L.fit_standardizer(Xtr)
    Ztr = L.standardize(Xtr, centre, scale)
    Zva = L.standardize(Xva, centre, scale)
    codes = L.floor_code_map(Xtr)
    s_tr, s_va = styles[itr], styles[iva]
    rng = seed_rng(seed, 3, repeat, fold, inner)
    # paired draws: one standard-normal block per
(style, n), reused across the WHOLE grid, so a grid
point is
    # compared with the others on identical
uniforms instead of on fresh noise
    style_order_va = sorted(set(s_va.tolist()))
    normal_by_style = {}
    for style in style_order_va:
        rows = np.nonzero(s_va == style)[0]
        normal_by_style[style] =
rng.standard_normal((int(rows.size),
len(L.FEATURE_COLS)))
    # D_train depends only on the inner-training
rows, not on (kappa, kappa_R): compute it once per
style
    d_train_by_style = {style:
L.style_dtrain(Ztr, s_tr, style) for style in
sorted(set(s_tr.tolist()))}
    marginals = {}
    marginal_summary_by_kappa = {}
    zblocks = {}
    for kappa in KAPPAS:
        marginal = L.StyleMarginals(kappa,
codes).fit(Xtr, s_tr, valid=valid_tr)
        marginals[kappa] = marginal
        marginal_summary_by_kappa[kappa] =
marginal.summary
        Zc = np.empty_like(Xtr)
        for style in sorted(set(s_tr.tolist())):
            rows = np.nonzero(s_tr == style)[0]
            U = marginal.forward(style, Xtr[rows])
            Zc[rows] = np.clip(ndtri(np.clip(U,
1e-9, 1.0 - 1e-9)), -8.0, 8.0)
        zblocks[kappa] = Zc
    scores = {}
    for kappa in KAPPAS:
        marginal = marginals[kappa]
        for kappa_R in KAPPA_RS:
            copula = L.GaussianCopula(
kappa_R).fit(zblocks[kappa], s_tr)
            num_v = num_u = 0.0
            den = 0
            n_cells = n_excluded = 0
            for style in style_order_va:
                rows = np.nonzero(s_va ==
style)[0]
                n = int(rows.size)
                if n < 2:
not estimable: excluded and counted, never filled in
                    n_excluded += 1
                    continue
                U = copula.uniforms_from_normals(
style, normal_by_style[style])
                gen_z = L.standardize(
L.sample_from_uniforms(marginal, style, U), centre,
scale)
                terms = L.energy_distance_terms(
gen_z, Zva[rows])
                d_train = float(
d_train_by_style.get(style, 1.0))
                value = float(terms["V"] /
d_train) if d_train > 1e-12 else float(terms["V"])
                num_v += value * n
                num_u += float(terms["U"]) * n
                den += n
                n_cells += 1
            scores[(float(kappa),
float(kappa_R))] = (num_v / den) if den else None
            diagnostics.append({"inner":
int(inner), "nTrain": int(itr.size), "nValid":
int(iva.size),
                                "kappa":
float(kappa), "kappa_R": float(kappa_R),
                                "V_normalized":
(num_v / den) if den else None,
                                "U_signed":
(num_u / den) if den else None,
                                "nCells":
int(n_cells), "nExcludedCells": int(n_excluded),
                                "nSamples":
int(den),
"nMarginalPooledFallbackCells":
int(marginal_summary_by_kappa[kappa]["nPooledFallbackC
ells"]),
                                "normalisation":
"V / D_train(inner-fold style train rows)"})
    tables.append(scores)
    inner_sizes.append(int(len(iva)))
keys = sorted({key for table in tables for key in
table})
mean_table = {}
for key in keys:
    values = [table[key] for table in tables if
table.get(key) is not None]
    mean_table[key] = float(np.mean(values)) if
values else None
# ties break towards the stronger shrinkage,
exactly like the frozen selection rule; an
unestimable grid point
# sorts last, it is never chosen by pretending
its score is 0
best = min(keys, key=lambda key: (mean_table[key]
if mean_table[key] is not None else np.inf, -key[0],
-key[1]))
return float(best[0]), float(best[1]),
mean_table, tables, inner_sizes, diagnostics
```

C.5 问题三·迁移：目标二次型、盒-半空间投影与带证书 QP（src/q3_transfer_fix.py）

```
# AI-ASSISTANCE NOTICE: the framework of this module
    was drafted with DeepSeek Harness assistance;
# the core algorithm logic, constraint corrections
    and experimental parameters were reviewed, amended
# and independently verified by the team members, who
    take responsibility for the contents.

_HERE = Path(__file__).resolve().parent
_ROOT = _HERE.parent
SOURCE_STYLE = "Pils/Lager"
TARGET_STYLE = "Tripel"
KAPPA = 5.0                 # marginal shrinkage for
the quantile anchor (declared, not tuned)
N_COMPONENTS = 10
LAMBDA = 1.0
ETA = 0.01
ADMM_RHO = 1.0
ADMM_TOL = 1e-8
ADMM_MAX_ITER = 40000
STALE_TOL = 1e-9            # "not worse" tolerance
on the stale_hops PROXY (numerical only)
VERDICTS = ("verified", "conditional",
    "not_identifiable", "failed")
COVERAGE_TOL = 1e-9
NONENAL = "X.E..2.nonenal"
CAPPED_CHEMISTRY = tuple(L.DEFECT_KEYS) + (NONENAL,)
LINEAR_MASK = np.array([name in L.LINEAR_SET for name
    in L.FEATURE_COLS])
LOG10_MASK = np.array([name in L.LOG10_SET for name
    in L.FEATURE_COLS])
THRESHOLDS = dict(L.THRESHOLD_MEAN)
RUN_DIR = run_dir()
_OUTPUTS = {}
_DEFECT_UNITS = {}
def load_data():
    frame = pd.read_csv(V6.DATA_Q3)
    chem = [c for c in frame.columns if
```

```
    c.startswith(V6.CHEM_PREFIX)]
    if chem != L.CHEM_COLS:
        raise RuntimeError("the chem__* column order
    of data/clean/q3.csv differs from the frozen
    FEATURE_COLS")
    X = frame[L.CHEM_COLS].to_numpy(dtype=float)
    Y = frame[L.LABEL_COLS].to_numpy(dtype=float)
    styles = frame["tasting_category_fine"].astype(
    str).to_numpy()
    beers = frame["beer"].to_numpy(dtype=int)
    return frame, X, Y, styles, beers
def fit_transfer_context(X_raw, Y, styles,
    context_idx, repeat, fold):
    raw_context = np.asarray(X_raw[context_idx],
    dtype=float)
    medians = L.column_medians(raw_context)
    X = L.apply_median(raw_context, medians)
    valid_context = np.isfinite(raw_context)     #
    imputed cells are not observations for the empirical
    marginals
    centre, scale = L.fit_standardizer(X)
    codes = L.floor_code_map(X)
    style_context = styles[context_idx]
    marginals = L.StyleMarginals(KAPPA, codes).fit(X,
    style_context, valid=valid_context)
    proxy = L.LabelProxy(centre, scale).fit(X,
    Y[context_idx])
    target_rows = np.nonzero(style_context ==
    TARGET_STYLE)[0]
    if target_rows.size < 3:
        raise RuntimeError("fold %d/%d has %d
    target-style training rows: the reference is not
    identifiable"
                           % (repeat, fold,
    target_rows.size))
    Z_t = L.standardize(X[target_rows], centre, scale)
    z_mean = Z_t.mean(axis=0)
    _, _, vt = np.linalg.svd(Z_t - z_mean,
    full_matrices=False)
    n_comp = int(min(N_COMPONENTS, Z_t.shape[0] - 1,
    Z_t.shape[1]))
    pca = np.array(vt[:n_comp], dtype=float,
    copy=True)
    scores = (Z_t - z_mean) @ pca.T
    score_mean = scores.mean(axis=0)
    score_sd = scores.std(axis=0, ddof=1)
    score_sd = np.where(score_sd > 1e-12, score_sd,
    1.0)
    correlation = np.corrcoef((scores - score_mean) /
    score_sd, rowvar=False)
    correlation = np.atleast_2d(np.nan_to_num(
    correlation, nan=0.0))
    correlation = 0.9 * correlation + 0.1 *
    np.eye(correlation.shape[0])
    correlation_inv = np.linalg.inv(correlation)
    lo = np.nanmin(X, axis=0)
    hi = np.nanmax(X, axis=0)
    lo = np.where(np.isfinite(lo), lo, 0.0)
    hi = np.where(np.isfinite(hi), hi, lo + 1.0)
    lo = np.where(LINEAR_MASK, np.maximum(lo, 0.0),
    lo)   # physical concentration non-negativity
    hi = np.maximum(hi, lo)
    pm = pca / score_sd[:, None]
    # (10, 231)
    return {
        "repeat": int(repeat), "fold": int(fold),
    "context_idx": np.asarray(context_idx, dtype=int),
        "medians": medians, "X": X, "valid":
    valid_context, "styles": style_context,
        "centre": centre, "scale": scale, "codes":
    codes,
        "marginals": marginals, "marginal_state":
    marginals.state_table(),
        "marginal_summary": marginals.summary,
    "proxy": proxy, "target_rows": target_rows,
    "Z_target": Z_t,
        "z_mean": z_mean, "pca": pca, "score_mean":
    score_mean, "score_sd": score_sd,
        "correlation_inv": correlation_inv, "pm": pm,
        "lowrank_U": pm.T, "lowrank_C":
    correlation_inv, "lowrank_C2": 2.0 * correlation_inv,
        "mref": pm @ z_mean, "lo": lo, "hi": hi,
        "low_z": (lo - centre) / scale, "up_z": (hi -
    centre) / scale,
        "d_target": float(cdist(Z_t, Z_t).sum() /
    (Z_t.shape[0] * (Z_t.shape[0] - 1))),
    }
def score(ctx, z):
    z = np.atleast_2d(np.asarray(z, dtype=float))
    s = ((z - ctx["z_mean"]) @ ctx["pca"].T -
    ctx["score_mean"]) / ctx["score_sd"]
    return s
def score_legacy(ctx, z):
    z = np.atleast_2d(np.asarray(z, dtype=float))
    s = (z @ ctx["pca"].T - ctx["score_mean"]) /
    ctx["score_sd"]
    return s
def mahalanobis(ctx, z, legacy: bool = False):
    s = score_legacy(ctx, z) if legacy else
    score(ctx, z)
    value = float((s @ ctx["correlation_inv"] @
    s.T)[0, 0])
    return float(np.sqrt(max(value, 0.0)))
def target_objective(ctx, z):
    s = score(ctx, z)
    return float((s @ ctx["correlation_inv"] @
    s.T)[0, 0])
def _project_box_halfspace(point, low, up, arow, b):
    w = np.clip(point, low, up)
    if arow is None or float(arow @ w) <= b:
        return w
    lo_mu, hi_mu = 0.0, 1.0
    for _ in range(80):
        if float(arow @ np.clip(point - hi_mu * arow,
    low, up)) <= b:
            break
        hi_mu *= 2.0
    else:
        raise RuntimeError("half-space projection
    failed: the feasible set may be empty")
    for _ in range(120):
        mid = 0.5 * (lo_mu + hi_mu)
        if float(arow @ np.clip(point - mid * arow,
    low, up)) > b:
            lo_mu = mid
        else:
            hi_mu = mid
    return np.clip(point - hi_mu * arow, low, up)
def admm_qp(ctx, zref, anchor, z_source, low, up,
    arow, b, lam, eta,
            tol=ADMM_TOL, max_iter=ADMM_MAX_ITER,
    rho=ADMM_RHO):
    U = ctx["lowrank_U"]
    C = ctx["lowrank_C"]
    C2 = ctx["lowrank_C2"]
    mref = ctx["mref"]
    n = zref.size
    C2_inv = np.linalg.inv(C2)

    def refresh(rho_value):
        a0 = rho_value + 2.0 * lam
        S = np.linalg.inv(C2_inv + (1.0 / a0) * (U.T
    @ U))
        return a0, S

    a0, S = refresh(rho)
    rhs_const = 2.0 * (U @ (C @ mref)) + 2.0 * lam *
    anchor

    def apply_minv(rhs):
        return (1.0 / a0) * rhs - (1.0 / (a0 * a0)) *
    (U @ (S @ (U.T @ rhs)))

    def prox(v):
        threshold = eta / rho
        shifted = v - z_source
        point = np.sign(shifted) *
    np.maximum(np.abs(shifted) - threshold, 0.0) +
```

```
    z_source
        w = np.clip(point, low, up)
        if arow is None or float(arow @ w) <= b:
            return w
        # exact constrained prox: bisection on the
    half-space multiplier
        lo_mu, hi_mu = 0.0, 1.0

        def candidate(mu):
            shifted_mu = v - mu * arow - z_source
            point_mu = np.sign(shifted_mu) *
    np.maximum(np.abs(shifted_mu) - threshold, 0.0) +
    z_source
            return np.clip(point_mu, low, up)

        for _ in range(80):
            if float(arow @ candidate(hi_mu)) <= b:
                break
            hi_mu *= 2.0
        for _ in range(120):
            mid = 0.5 * (lo_mu + hi_mu)
            if float(arow @ candidate(mid)) > b:
                lo_mu = mid
            else:
                hi_mu = mid
        return candidate(hi_mu)

    z = np.clip(anchor, low, up)
    w = prox(z)
    u = z - w
    w_prev = np.array(w, dtype=float, copy=True)
    primal = dual = np.inf
    iterations = 0
    for iterations in range(1, int(max_iter) + 1):
        z = apply_minv(rhs_const + rho * (w - u))
        w_prev = w
        w = prox(z + u)
        u = u + z - w
        primal = float(np.max(np.abs(z - w)))
        dual = float(rho * np.max(np.abs(w - w_prev)))
        if primal < tol and dual < tol:
            break
        if iterations % 50 == 0:  # residual
    balancing; the scaled dual variable is rescaled with
    rho
            if primal > 10.0 * dual:
                u = u * 0.5
                rho *= 2.0
            elif dual > 10.0 * primal:
                u = u * 2.0
                rho *= 0.5
            else:
                continue
            a0, S = refresh(rho)
            rhs_const = 2.0 * (U @ (C @ mref)) + 2.0
    * lam * anchor
    return {"w": w, "z": z, "iterations":
    int(iterations), "primal_residual": float(primal),
            "dual_residual": float(dual), "rho":
    float(rho)}
def objective_value(ctx, z, zref, anchor, z_source,
    lam, eta):
    d = np.asarray(z, dtype=float) - zref
    s = ctx["pm"] @ d
    quad = float(s @ ctx["correlation_inv"] @ s)
    return quad + lam * float((z - anchor) @ (z -
    anchor)) + eta * float(np.sum(np.abs(z - z_source)))
def objective_gradient(ctx, z, zref, anchor,
    z_source, lam, eta):
    d = np.asarray(z, dtype=float) - zref
    smooth = 2.0 * (ctx["pm"].T @
    (ctx["correlation_inv"] @ (ctx["pm"] @ d))) + 2.0 *
    lam * (z - anchor)
    return smooth + eta * np.sign(z - z_source)
def finite_difference_check(ctx, z, zref, anchor,
    z_source, lam, eta, coords, step=1e-6):
    base = np.array(z, dtype=float, copy=True)
    analytic = objective_gradient(ctx, base, zref,
    anchor, z_source, lam, eta)
    abs_errors, rel_errors, kinks = [], [], 0
    for j in coords:
        shift = np.zeros_like(base)
        shift[j] = step
        plus = objective_value(ctx, base + shift,
    zref, anchor, z_source, lam, eta)
        minus = objective_value(ctx, base - shift,
    zref, anchor, z_source, lam, eta)
        numeric = (plus - minus) / (2.0 * step)
        error = abs(numeric - analytic[j])
        if abs(base[j] - z_source[j]) > 4.0 * step:
            abs_errors.append(error)
            if abs(analytic[j]) > 1e-6:  # a relative
    error is only meaningful away from gradient
    cancellation
                rel_errors.append(error /
    abs(analytic[j]))
        else:
            kinks += 1  # the L1 kink |z - z_source|
    = 0 lies inside the step: the secant is not the
    subgradient
    return {"n_coords": int(len(coords)),
    "max_abs_error": float(max(abs_errors)) if abs_errors
    else None,
            "max_rel_error": float(max(rel_errors))
    if rel_errors else None,
            "l1_kink_coords": int(kinks)}
def kkt_stationarity_residual(ctx, w, zref, anchor,
    z_source, low, up, arow, b, lam, eta,
                              tol=1e-7, kink_tol=1e-6):
    d = w - zref
    smooth = 2.0 * (ctx["pm"].T @
    (ctx["correlation_inv"] @ (ctx["pm"] @ d))) + 2.0 *
    lam * (w - anchor)
    n = w.size
    rows = [np.eye(n), -np.eye(n)]
    rhs = [np.asarray(up, dtype=float),
    -np.asarray(low, dtype=float)]
    if arow is not None:
        rows.append(arow.reshape(1, -1))
        rhs.append(np.array([b], dtype=float))
    A = np.vstack(rows)
    bvec = np.concatenate(rhs)
    slack = A @ w - bvec
    tight = np.abs(slack) <= tol
    kink = np.abs(w - z_source) <= kink_tol
    off_kink = ~kink
    gradient = smooth + eta * np.where(off_kink,
    np.sign(w - z_source), 0.0)
    blocks, target = [], -gradient
    if tight.any():
        blocks.append(A[tight].T)
    n_kink = int(kink.sum())
    if n_kink:
        selector = np.zeros((n, n_kink))
        selector[np.nonzero(kink)[0],
    np.arange(n_kink)] = 1.0
        blocks.append(2.0 * eta * selector)
        target = target + eta * selector.sum(axis=1)
    if not blocks:
        return {"kkt_residual":
    float(np.linalg.norm(gradient)), "n_tight": 0,
    "n_kink": n_kink}
    design = np.hstack(blocks)
    solution, _ = nnls(design, target)
    if n_kink:
        multipliers = solution[:design.shape[1] -
    n_kink]
        sub = np.clip(solution[design.shape[1] -
    n_kink:], 0.0, 1.0)
        residual_vector = gradient + (A[tight].T @
    multipliers if tight.any() else 0.0) \
            + eta * (selector @ (2.0 * sub - 1.0))
    else:
        residual_vector = gradient + A[tight].T @
    solution
    return {"kkt_residual": float(np.linalg.norm(
    residual_vector)),
            "n_tight": int(tight.sum()), "n_kink":
```

```
    n_kink}
def problem_hash(*arrays, meta: dict) -> str:
    digest = hashlib.sha256()
    for value in arrays:
        if value is None:
            digest.update(b"none")
            continue
        digest.update(np.ascontiguousarray(np.asarray(
    value, dtype=np.float64)).tobytes())
    digest.update(json.dumps(meta,
    sort_keys=True).encode("utf-8"))
    return digest.hexdigest()
def proxy_linear_terms(ctx, x_storage, target: str)
    -> dict:
    x_storage = np.asarray(x_storage,
    dtype=float).reshape(-1)
    z = (x_storage - np.asarray(ctx["centre"],
    dtype=float)) / np.asarray(ctx["scale"], dtype=float)
    a = np.asarray(ctx["proxy"].coef_for(target),
    dtype=float).copy()
    c = float(ctx["proxy"].intercept_for(target))
    a_dot_z = float(a @ z)
    return {"a": a, "c": c, "z": z, "a_dot_z":
    a_dot_z, "prediction": float(a_dot_z + c)}
def chemistry_caps(ctx, x_source, x_raw_source=None):
    caps, states = {}, {}
    x_source = np.asarray(x_source, dtype=float)
    raw = (np.full(x_source.shape, np.nan) if
    x_raw_source is None
           else np.asarray(x_raw_source,
    dtype=float).reshape(-1))
    for name in CAPPED_CHEMISTRY:
        j = L.FEATURE_COLS.index(name)
        stored = float(x_source[j])
        raw_value = float(raw[j]) if
    np.isfinite(raw[j]) else None
        is_na = raw_value is None
        caught = bool(np.isclose(stored,
    ctx["codes"][name], rtol=1e-6, atol=0.0)) if name in
    ctx["codes"] else False
        if name in L.LOG10_SET:
            physical = 0.0 if caught else float(10.0
    ** stored)
            cap = float(ctx["lo"][j]) if physical <=
    0.0 else float(np.log10(physical))
        else:
            physical = 0.0 if caught else max(stored,
    0.0)
            cap = 0.0 if physical <= 0.0 else physical
        caps[name] = float(min(cap,
    float(ctx["hi"][j])) if name != NONENAL else cap)
        if is_na:
            source_state, cap_state =
    "imputed_training_median", "conditional_on_imputation"
        elif caught:
            source_state, cap_state = "floor_coded",
    "conditional_floor_code"
        else:
            source_state, cap_state = "measured",
    "measured_source_value"
        states[name] = {
            "stored_source": stored,
    "raw_source_value": raw_value, "is_na": bool(is_na),
            "is_floor": bool(caught),
    "imputed_value": (stored if is_na else None),
            "imputation_basis":
    ("training_fold_median" if is_na else "not_imputed"),
            "physical_source": physical,
    "cap_stored": float(caps[name]),
            "source_state": source_state,
    "cap_state": cap_state,
            "censored_source": caught,
            "state": ("floor_coded_source (physical
    value not identified)" if caught else
                      ("imputed_from_training_median
    (source cell is missing)" if is_na else
                       "measured_source_value")),
        }
    return caps, states
def source_predictions(ctx, x_source):
    x_source = np.asarray(x_source,
    dtype=float).reshape(1, -1)
    return {"stale_hops": float(
    ctx["proxy"].predict_storage(x_source, L.STALE)[0]),
            "t2n": float(ctx["proxy"].predict_storage(
    x_source, L.T2N)[0])}
def build_problem(ctx, x_source,
    quantile_anchor_storage, x_raw_source=None):
    caps, cap_states = chemistry_caps(ctx, x_source,
    x_raw_source)
    low = np.array(ctx["low_z"], dtype=float,
    copy=True)
    up = np.array(ctx["up_z"], dtype=float, copy=True)
    for name, cap in caps.items():
        j = L.FEATURE_COLS.index(name)
        up[j] = min(up[j], float((cap -
    ctx["centre"][j]) / ctx["scale"][j]))
        low[j] = min(low[j], up[j])   # keep the box
    non-empty: the chemistry cap wins over the training
    support
    z_source = (np.asarray(x_source, dtype=float) -
    ctx["centre"]) / ctx["scale"]
    z_anchor = (np.asarray(quantile_anchor_storage,
    dtype=float) - ctx["centre"]) / ctx["scale"]
    terms = proxy_linear_terms(ctx, x_source, L.STALE)
    arow, intercept_c, a_dot_source = terms["a"],
    terms["c"], terms["a_dot_z"]
    prediction = source_predictions(ctx, x_source)
    stale_cap = a_dot_source + STALE_TOL          # b
    = a'z_source + tol = f(z_source) - c + tol
    if abs(stale_cap - (terms["prediction"] -
    intercept_c + STALE_TOL)) > 1e-9:
        raise AssertionError("half-space intercept is
    inconsistent: b != f(z_source) - c + tol")
    if abs(prediction["stale_hops"] -
    terms["prediction"]) > 1e-8:
        raise AssertionError("proxy.predict_storage
    and a'z+c disagree: the constraint row would not be
    the proxy")
    box_min_proxy = float(np.sum(np.where(arow > 0,
    arow * low, arow * up)))
    return {"low": low, "up": up, "z_source":
    z_source, "z_anchor": z_anchor,
            "caps": caps, "cap_states": cap_states,
    "zref": ctx["z_mean"].copy(),
            "stale_cap": stale_cap, "arow": arow,
    "source_prediction": prediction,
            "stale_intercept_c": intercept_c,
    "stale_a_dot_source": a_dot_source,
            "stale_prediction_source":
    terms["prediction"],
            "stale_cap_old_definition":
    float(prediction["stale_hops"] + STALE_TOL),
            "box_min_proxy": box_min_proxy,
            "feasible_set_nonempty":
    bool(box_min_proxy <= stale_cap)}
def delivered_metrics(ctx, x_source, x_new, problem,
    strategy, meta):
    centre = np.asarray(ctx["centre"], dtype=float)
    scale = np.asarray(ctx["scale"], dtype=float)
    z_new = (np.asarray(x_new, dtype=float) - centre)
    / scale
    prediction = source_predictions(ctx, x_new)
    arow, halfspace_b = problem["arow"],
    problem["stale_cap"]
    stale_halfspace_residual = float(arow @ z_new -
    halfspace_b)       # a'z_new - b, the declared row's
    residual
    stale_prediction_delta = float(
    prediction["stale_hops"] - problem["stale_prediction_s
    ource"])
    # f(z_new) - f(z_source) = a'z_new - a'z_source =
    (a'z_new - b) + tol, exactly; this is the identity
    the old
    # code violated by keeping the intercept inside b.
    identity_error = abs(stale_prediction_delta -
    (stale_halfspace_residual + STALE_TOL))
    if identity_error > 1e-7:
```

```
    raise AssertionError("half-space identity
broken: f(z_new)-f(z_source)=%r but a'z_new-b+tol=%r"
                         % (
stale_prediction_delta, stale_halfspace_residual +
STALE_TOL))
out = {
    "target_mahalanobis": mahalanobis(ctx, z_new),
    "target_mahalanobis_legacy_centering":
mahalanobis(ctx, z_new, legacy=True),
    "l1_change_z": float(np.sum(np.abs(z_new -
problem["z_source"]))),
    "mean_abs_change_z": float(np.mean(np.abs(
z_new - problem["z_source"]))),
    "stale_hops_source": problem["source_predictio
n"]["stale_hops"],
    "stale_hops_new": prediction["stale_hops"],
    "stale_halfspace_b": float(halfspace_b),
    "stale_intercept_c": float(
problem["stale_intercept_c"]),
    "stale_a_dot_source": float(
problem["stale_a_dot_source"]),
    "stale_halfspace_residual":
stale_halfspace_residual,
    "stale_prediction_delta":
stale_prediction_delta,
    "stale_halfspace_identity_error":
float(identity_error),
    "stale_hops_nonincreasing":
bool(stale_prediction_delta <= STALE_TOL + 1e-9),
    "t2n_source": problem["source_prediction"]["t2
n"],
    "t2n_new": prediction["t2n"],
    "t2n_increasing": bool(prediction["t2n"] >
problem["source_prediction"]["t2n"] + 1e-9),
    "chemistry": {},
}
for name in CAPPED_CHEMISTRY:
    j = L.FEATURE_COLS.index(name)
    before = float(L.physical_conc(name,
np.array([x_source[j]]), ctx["codes"])[0])
    after = float(L.physical_conc(name,
np.array([x_new[j]]), ctx["codes"])[0])
    state = problem["cap_states"][name]
    out["chemistry"][name] = {
        "stored_source": float(x_source[j]),
"stored_new": float(x_new[j]),
        "raw_source_value":
state["raw_source_value"], "is_na_source":
state["is_na"],
                               "is_floor_source": state["is_floor"],
"imputed_value": state["imputed_value"],
                               "imputation_basis":
state["imputation_basis"], "source_state":
state["source_state"],
                               "physical_source": before,
"physical_new": after,
                               "nonincreasing": bool(after <= before *
(1.0 + 1e-9) + 1e-12),
                           }
    out["nonincrease_all"] = bool(all(
entry["nonincreasing"] for entry in
out["chemistry"].values())
                                  and
out["stale_hops_nonincreasing"])
    out["n_constraint_violations"] = int(
        sum(1 for entry in out["chemistry"].values()
if not entry["nonincreasing"])
        + (0 if out["stale_hops_nonincreasing"] else
1))
    return out
def source_nonincreasing_verdict(residual, state,
name):
    if residual > 1e-12:
        return "failed"
    if state["is_na"] or state["is_floor"]:
        return "not_identifiable"
    if name in L.DEFECT_KEYS:
        return "verified"
    return "conditional"
def bound_check_verdict(residual, state):
    if residual > 1e-12:
        return "failed"
    if state["is_na"]:
        return "conditional_on_imputation"
    if state["is_floor"]:
        return "conditional_floor_code"
    return "verified"
STRATEGIES = ("source-fixed", "target-mean-projection"
, "target-mean-projection-full",
                  "qp-source-anchor",
    "qp-quantile-anchor")
QP_STRATEGIES = ("qp-source-anchor",
    "qp-quantile-anchor")
```